

Falcon
Developer Guide

Beta Revision
v2.04277

Usage Restrictions

Your right to access and use the following "**PDN Materials**" is subject to and governed by the following terms and the terms and conditions of the Palm Developer Network Program License Agreement, available at https://pdn.palm.com/regac/pdn/page?Page_Name=terms.

Disclaimer and Limitation of Liability

Palm, Inc. ("**Palm**") provides these PDN Materials as a courtesy to members of its Palm Developer Network ("**PDN**"). Palm provides the PDM Materials on an "AS IS" basis and does not guarantee nor assume any liability for the accuracy or completeness of these PDN Materials, nor does Palm guarantee that the PDN Materials will be without errors. You assume the entire risk of using the PDN Materials, including any damage or loss resulting from your use of the PDN Materials or any loss or claims by third parties which may arise from your use of the PDN Materials.

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, THE PDM MATERIALS ARE PROVIDED TO YOU ON AN "AS IS" BASIS AND WITHOUT ANY WARRANTY OF ANY KIND OR NATURE. TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, PALM AND ITS SUPPLIERS EXPRESSLY DISCLAIM ANY IMPLIED OR STATUTORY WARRANTIES, INCLUDING WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE, ACCURACY, CORRESPONDENCE WITH DESCRIPTION, SATISFACTORY QUALITY AND NON-INFRINGEMENT.

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, NEITHER PALM NOR ITS SUPPLIERS SHALL BE LIABLE FOR INCIDENTAL, CONSEQUENTIAL, INDIRECT, SPECIAL, OR PUNITIVE DAMAGES OF ANY KIND, LOSS OF INFORMATION OR DATA, LOSS OF REVENUE, LOSS OF BUSINESS OR OTHER FINANCIAL LOSS ARISING OUT OF OR IN CONNECTION WITH THE USE OF THE PDN MATERIALS, WHETHER BASED IN CONTRACT, TORT (INCLUDING NEGLIGENCE), STRICT PRODUCT LIABILITY OR ANY OTHER THEORY, EVEN IF PALM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES AND EVEN IF ANY LIMITED REMEDY IS DEEMED TO HAVE FAILED OF ITS ESSENTIAL PURPOSE. Some countries, states, or provinces do not allow the limitation of incidental or consequential damages so the above provisions may be limited in their application to you.

Copyright and Trademark
© 2004-2007 Palm, Inc. All rights reserved.

Palm, LifeDrive, Zire, Tungsten, Treo, Blazer, Handspring, Graffiti, HotSync, VersaMail, Palm Powered, and Palm OS are among the trademarks or registered trademarks owned by or licensed to Palm, Inc. Adobe and Acrobat are either registered trademarks or trademarks of Adobe Systems Incorporated in the United States and/or other countries. All other brand and product names are or may be trademarks of, and are used to identify products or services of, their respective owners.

The foregoing provisions shall be governed by the laws of the State of California and by the federal laws of the United States, excluding their conflicts of laws provisions. The United Nations Convention on Contracts for the International Sale of Goods (1980) is hereby excluded in its entirety from application to the foregoing provisions. In the event any provision is found to be invalid, illegal or unenforceable, the validity, legality and enforceability of any of the remaining provisions shall not in any way be affected or impaired.

Please read the Palm Developer Network Program License Agreement before using the PDN Materials. If you do not agree to the terms and conditions of the Developer Network Program License Agreement, you may not access or use the PDN Materials. USING OR ACCESSING ANY PART OF THE PDN MATERIALS INDICATES THAT YOU ACCEPT THE TERMS OF THE DEVELOPER NETWORK PROGRAM LICENSE AGREEMENT.

Contents

What's New . . . . . **ix**

Chapter 1: Developer Guide Overview **3**
1.1 Intended audience 3
1.2 How this guide is organized 3
1.3 Finding API documentation 3

Chapter 2: Falcon Overview **5**
2.1 Product overview 5
2.2 Falcon features 6
2.3 Falcon system software design 8
2.4 Falcon operation 11
2.5 File organization 13

Chapter 3: Hardware Developers Kit **15**
3.1 Hardware features 15

Chapter 4: Getting Started Overview **21**
4.1 Intended audience 21
4.2 Falcon development 21
4.3 Linux operating system 21
4.4 Prior Linux experience 22
4.5 New to Linux 22

Chapter 5: Ubuntu Overview **23**
5.1 Ubuntu overview 23
5.2 Major features 23
5.3 Command Line Tools 24

Chapter 6: SDK System Requirements **27**
6.1 Hardware requirements 27
6.2 Software requirements 27

Chapter 7: Installing Ubuntu **29**
7.1 Create the Ubuntu CD 29
7.2 Booting the install CD 30
7.3 The install process 30

7.4 Update system software 32
7.5 Download kernel headers 33

Chapter 8: Falcon SDK 35
8.1 Install the Falcon SDK 35
8.2 Download the Falcon SDK 35
8.3 Install the Falcon SDK 35
8.4 Reboot the development station 37
8.5 Launch the simulator 37

Chapter 9: Flashing the Falcon 39
9.1 Falcon ROM images 39
9.2 Acquiring the ROM image 39
9.3 Prepare the SD card 41
9.4 Copy ROM files to SD card 41
9.5 Flash the Falcon 43

Chapter 10: Mailsync Setup 45
10.1 What is Mailsync? 45
10.2 Mailsync compatiblity 45
10.3 Acquiring the Mailsync library 47
10.4 Install Versamail 4.0 47
10.5 Install the Mailsync library for Windows Mobile 49
10.6 Installed files 50

Chapter 11: Falcon Setup 53
11.1 Initial configuration 53
11.2 Language and license 53
11.3 Date and time 53
11.4 Phone setup 53

Chapter 12: Falcon SDK 61
12.1 The development process 61
12.2 Requirements 61
12.3 Downloading the SDK 62
12.4 Installing the SDK 62
12.5 How the SDK is organized 62
12.6 PDN online support 65

Chapter 13: Building Applications on the i386 Development System . . . 67
13.1 Creating the app source file 67
13.2 Compiling and linking the app 68
13.3 Executing the app from the command line 69

13.4 Creating the app.hxi file 70
13.5 Expanded Makefile 71
13.6 Running the application in the simulator 72
13.7 Debugging i386 applications 73

Chapter 14: Building Applications for ARM 75
14.1 Creating the app source file 75
14.2 Compiling and linking the app 75
14.3 Packaging the app 77
14.4 Expanded Makefile 78
14.5 Installing the ARM app on Falcon 80
14.6 Remote debugging ARM apps 82

Chapter 15: Debugging 85
15.1 GDB 85
15.2 Remote debugging with GDB 87
15.3 Configure Falcon for remote debugging 87
15.4 Debugging DirectFB 90
15.5 Debugging tips 90

Chapter 16: Tutorial 91
16.1 From hxhello.c 91
16.2 Events 91
16.3 From hxcommandbutton.c 92
16.4 From hxquit.c 92
16.5 Sending messages 93
16.6 Building with resources 93
16.7 From hxdrawstring.c 93
16.8 From hxdrawstring.c (also) 94
16.9 Memory Management 94
16.10 Interrupts 94
16.11 From hxmenu.c 95

Chapter 17: Graphics and UI 101
17.1 DirectFB 101
17.2 Lite 101
17.3 HxUI 101
17.4 Localization 101

Chapter 18: HxUI XML Format 103
18.1 HxUI XML overview 103
18.2 Top Level Element (hxui) 103
18.3 Meta Elements 104

18.4 UI Elements 104
18.5 String Resources 123
18.6 Localization 124
18.7 Editing Tips 125
18.8 HxUI APIs 125

Chapter 19: Sound 127
19.1 AC97 Support 127

Chapter 20: Messaging System 129
20.1 Messaging 129

Chapter 21: Wireless 131
21.1 Bluetooth 131
21.2 WiFi 131

Chapter 22: Storage and File Management 133
22.1 Default storage 133
22.2 Compact Flash card 133
22.3 SD/MMC card 138
22.4 USB drive 142
22.5 Filesystems 143
22.6 Default storage location 143

Chapter 23: System Features and Widgets 145
23.1 System features 145
23.2 Widgets 145

Chapter 24: System Support 147
24.1 Power 147

Chapter 25: Support Libraries 151
25.1 Applications 151
25.2 Daemons 152
25.3 App Frameworks 152
25.4 App libraries 152
25.5 Utility libraries 153
25.6 LiTE UI toolbox framework 153
25.7 Graphics library 153
25.8 Font libraries 154
25.9 Graphics Codecs 154
25.10 File Systems 154
25.11 Kernel modules 154

25.12 Kernel 154
25.13 Hardware drivers 154
25.14 Boot time only 155
25.15 Non-ROM components 155

Chapter 26: Style Guide 159
26.1 Applications 159
26.2 File management 160
26.3 Navigation 163
26.4 Wireless connectivity 164
26.5 Security 166
26.6 Error handling 167
26.7 Media support 168
26.8 System services 168
26.9 System level functions 173
26.10 Display Lock application 176
26.11 Localization 177
26.12 Multiple-language ROM support 177

Appendix A: HxUI XML DTD 181
A.1 HxUI XML Format Verification 181

Appendix B: FAQs 187
B.1 Falcon 187
B.2 SDK 189
B.3 API 190
B.4 Linux 193

What's New

This section provides an overview of new features and product developments that have been implemented since the last release of this guide.

The following items have been added or modified in the *Developer Guide*:

-
-

PART I

Overview

This section of the *Developer Guide* provides a high level overview of the Falcon device and a look at the hardware developer's kit (HDK).

CHAPTER 1

Developer Guide Overview

This chapter discusses how this guide is organized and how to locate API documentation for the Palm® Falcon SDK.

1.1 Intended audience

The Falcon SDK targets experienced developers with a firm knowledge of Linux. It is not meant as a general introduction to embedded Linux programming. Rather it stresses the features unique to developing hardware and software for the Falcon.

1.2 How this guide is organized

This developer guide contains an overview of the Falcon device including hardware specifications. It concentrates on the software development environment, including the system software on Falcon, the tools, interfaces, and libraries of the SDK, and a general discussion of Falcon APIs. This guide also contains debugging information, as well as style conventions on how certain features should be used.

IMPORTANT This guide is a current snapshot of Falcon. As information changes over the duration of the project, it will be communicated in subsequent versions of this guide.

WARNING! This guide is Palm® Confidential. DO NOT distribute it.

1.3 Finding API documentation

A detailed description of Falcon APIs can be found in the API documentation. Complete API documentation is generated directly from Palm® source code. Refer to the separate Falcon API Guide either in compressed HTML format (.chm) or HTML format available in the Falcon section of the Palm Developer Network at **http://pdn.palm.com**.

CHAPTER 2

Falcon Overview

This chapter describes the Falcon features.

2.1 Product overview

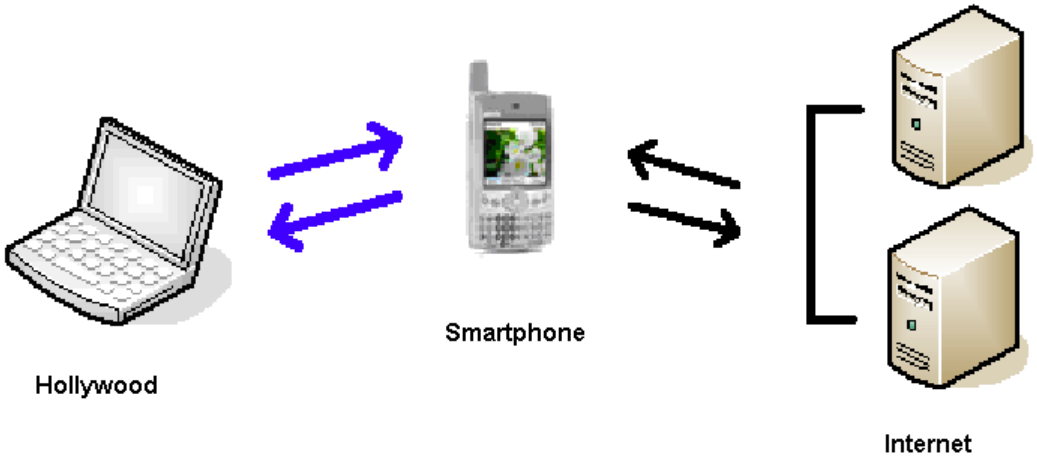
Falcon is a companion device to smartphones. It communicates with Palm Treo and Windows Mobile 5 smartphones, seamlessly synchronizing email while providing an enhanced online experience. Falcon turns a smartphone into a complete mobile computing solution.

Falcon design features:

- A big screen and keyboard
- Large color display
- Full size keyboard
- PDA style instant on
- Included suite of Office-compatible applications
- Standards compliant WWW browser



To use Falcon, users simply turn on the device. Email is automatically synchronized with the smartphone with no intervention required by the user. If the smartphone is provisioned with dial-up networking (DUN) capabilities, the Falcon can use it as a modem to communicate with the Internet for web browsing. Alternatively, a user can also use WiFi to connect to the Internet, however WiFi cannot be used for syncing email.



2.2 Falcon features

2.2.1 Smartphone integration

- Falcon is a companion device for smartphones that:
  - Provides the responsiveness and reliability of a PDA.
  - Integrates tightly with the Palm Treo line of smartphones as well as third party Windows Mobile smartphones. Falcon automatically wakes the smartphone and establishes data communication with no user interaction.
  - Provides a complete mobile computing solution for common tasks for mobile professionals.
  - Provides a flexible architecture that is easily customized to support vertical solutions.

2.2.2 Instant response

The primary goal for Falcon is that all functions be perceived as instantaneous. The product has an overall feel of real-time response and immediacy. The user should feel like everything is always ready to go, and it's only a matter of pressing the right button to get there. This means:

- Instant on and off: Even a 5-second boot period is unacceptable. To preserve battery power, some delay may be required for extended periods of inactivity. Similarly, when the user turns Falcon off, the screen goes off immediately.

- Perception of instant app launching: Launching happens with the same speed as screen redraw. The difference between switching running apps and launching an app to the previous state is imperceptible.
- Eliminate as many wait states and "blocking UI" as possible: Goal is never to see a wait cursor and minimize progress dialogs for downloading data. Whenever possible, handle processes in the background and do not require user intervention.
- Minimum steps for common actions: The speed of an instant process is compromised if you have to take too many steps or think too much to execute it.

2.2.3 Simplified Windows standards

- Eliminate full-blown windowing environment.
- Simplify file structure.
- Replace complex systems with Treo standards, for example, Menu commands.

2.2.4 Elegant keyboard navigation

Minimize track point pain.

Today's laptops provide two options:

- Finicky track point/pad navigation, or
- Inefficient keyboard access (technically, you can access everything but it's not convenient)

Falcon users will typically use a trackpoint as they do on a laptop. System conventions eliminate the most egregious pain cases, primarily the targeting of small objects such as scroll bar elevators and minimize/maximize icons.

The Falcon keyboard also contains a built-in scroll wheel to allow scrolling without needing to position the cursor directly over the scroll bar elevators.

2.2.5 Power management

Falcon has two goals that affect power management: rapid startup (from off in less than half a second) and a long battery life. This provides a PDA-like instant on but with (near) zero power drain.

- Falcon simplifies power states and presents a simple on/off model to the user.
- To preserve power, different levels of power states are implemented behind the scenes (intermediate sleep and ready states).
- The user may notice slight differences in boot time but the general behavior is a binary on/off.
- There are several power states:
  - On: The display is on. The user can use all the input devices, etc.
  - Ready: The display is off. The WiFi and Bluetooth radios may be active, but broadcasting and receiving at fewer times to conserve power. The device turns on instantly but draws power to maintain readiness.

- Sleep: The device turns on after a slight pause. The OS state is saved in memory.
  - Entering sleep mode means shut down all apps first; hibernation mode only has to be a quick way to boot the OS.
  - Apps save their own state.
  - OS saves state with no apps running.
  - Hibernation is not necessarily a copy of all system memory, just a quick boot that then quickly loads the one app that was in the foreground when the sleep state was initiated.

IMPORTANT There is no API to change these power states. The only thing developers may notice is a longer latency for some operations.

2.2.6 User accounts

- Falcon simplifies the development effort by avoiding a full-blown accounts model.
- Falcon will come with a single user account named User. As part of the boot process, the Application Manager automatically logs in as User with no login name or password required. So there is no user concept of logging on to an account. User runs with normal Linux operating permissions, although it does have an entry in the sudoers file.
- The Application Manager itself runs with system operating permissions. However, whenever it launches an application for the user, that application will run only with normal permissions.
- There is a root account on the Falcon, however it cannot be directly accessed. You can only login as root by using the custom *sume* command. This command is a custom version of the traditional *su* and *sudo* commands. *Sume* requires a password only if the user has set one on the device.

NOTE Remote login is enabled by default in the development ROMs, however it will be disabled by default in the Product ROM that ships to retail. This feature can be manually enabled.

2.3 Falcon system software design

Falcon uses open source components. In keeping with the spirit of open source, Falcon system software sources will also be made available. However it is not yet known exactly what format that open source will take. There are some portions of the system, such as the sync model, which are proprietary throughout the line of Palm devices and therefore will not be released.

Below are some of the considerations of open source software.

2.3.1 Linux basis

- Falcon is based on Linux kernel version 2.4.21.

- For optimum graphics performance, Falcon uses DirectFB to provide direct access to the graphics frame buffers.
- LiTE is used to hold the custom Falcon System UI onto DirectFB. LiTE stands for *LiTE is a Toolbox Enabler*. Hxui is the toolbox that LiTE enables. For more information, see **LiTE UI toolbox framework**.
- To tie the various widgets created from DirectFB and LiTE together, Fusion is used for IPC and message passing. It is built into the Falcon arm kernel, but installed as a kernel module for the i386 based development system.
- Commercial Bluetooth and WiFi drivers provide communication with the outside world.

2.3.1.1 Open Source Initiative

- Term "Open Source" coined in 1998 by programmers describing Netscape initiative to "open" the source code for its browser.
- "Open Source Initiative" ("OSI") founded in 1998 to encourage publishers to distribute software on terms that
  - Make source code available;
  - Do not restrict copying and redistribution;
  - Permit modification and distribution of modified works;
  - Do not restrict uses or classes of user
- Certifies Licenses as "Open Source" if they meet those criteria.
- At least 35 separate licenses certified

2.3.1.2 Licensing Concepts: Redistribution

- Right to redistribute is central to the definition of "open source"
  - Microsoft Shared Source License does not allow
- Most problematic aspect of open source licenses
  - Especially for derivative works and modifications under the GPL and LGPL.
- Right to distribute verbatim code and derivative works/modifications conditioned on compliance with:
  - Copyleft/forcing provisions
  - Attribution Requirements
  - Advertising Requirements
  - Disclaimers of Warranty

2.3.1.3 Licensing Concepts: Copyleft

- FSF term for requiring that any redistribution of code licensed under an open source agreement, and any derivatives, must be pursuant to the same open source license terms: "Forcing" the code to remain open

License	Copyleft (Forcing Clause)	GPL Compatible
Apache License 1.1	No	No
Modified BSD	No	Yes
GPL	Yes	Yes
LGPL & Library GPL	Yes	Yes
SISSL	Yes	No
IBM Public License	Yes	No
Mozilla Public License	Yes	No

2.3.1.4 The GPL: Modified Works and Linking

- Do both static linking (linked at compile time) and dynamic linking (linked at run-time) create a "derivative or collective work based on the Program"?
- Richard Stallman (GNU): Yes, all linking creates a derivative work
- Linus Torvalds (Linux): Certain types of dynamic linking may not be covered (as when a proprietary program makes run time calls on Linux OS functions).

2.3.1.5 The GPL: Conclusion

To minimize the risk of "viral" effect of GPL on proprietary code:

- Design proprietary software separately to run on multiple platforms, not reliant on GPL'd code.
  - Do not use any GPL'd code within proprietary software
  - Ship proprietary software separately from GPL'd code, or at least on separate storage medium;
  - Avoid static linking (linking at compilation);
  - Avoid appearance of single "work"
  - Use only APIs or similar "arms length" means to transfer data between proprietary software and GPL'd code;
  - If GPL'd code is modified, distribute modifications in accordance with terms

2.3.2 Falcon additions

- Tying the low-level Falcon libraries together is done by our native Hx libs.
- The Falcon look and feel is provided by our own HxUi libraries.
- The Palm® heritage of instant on and ease of use can be found in Falcon. Part of that is responsible for our own version of Sync ML.

2.4 Falcon operation

2.4.1 Normal operation

Normal Falcon operation is much like a PDA. Hitting the power button lights up the screen, which shows the last running application. Pressing the Application key displays the Application Launcher so the user can launch a new application. The new application takes over the entire screen and the previous application is shifted to the background. That application does not actually quit. There is no quit or exit menu item on a Falcon application.

The previous application saves its current state for when it is reselected from the Application Launcher to return to the foreground. It may ask to explicitly save files before switching to the background. Hitting the power button again turns off the display and the foreground application goes to sleep, ready to wake up at the next touch of the power button.

- Tap power button to turn Falcon off.

  The application will be notified to save it's state.

- Tap power button to instantly turn Falcon back on.

  The screen was off, but Falcon was only sleeping. To give the appearance of instant on, the screen is turned back on with the previous contents still displaying even as the sleeping app is being reawakened.

- What happens if an application is hung?

  Pressing the Apps and backspace keys at the same time will kill the application.

- How do you reset Falcon?

  Since there is no reset key on Falcon, hold down the power button for 5 seconds. Falcon will reboot and automatically start the Application Manager, which will, in turn, launch `HxPreferences`.

2.4.2 Boot process

During the normal course of operation the Falcon will rarely go through the boot process. There is an initial boot the first time the device is turned on, however all subsequent on/off cycles will normally be suspend operations unless a manual reboot is performed.

It is possible to manually reboot the Falcon by holding down the power button for five seconds. This is equivalent to issuing the "shutdown -r now" command on a standard *nix system.

The first process to initialize during the boot process is the ARM boot loader, known as the blob. This runs in the background and cannot be manipulated by the user. While the boot loader is initializing the system, the Falcon displays a Palm logo on a black screen. A white progress bar underneath the logo displays the current state of the process.

For normal users, the kernel will continue on to launch the Application Manager, which will, in turn, launch `HxPreferences`.

For developers, the boot process can be made to stop in the kernel where you can issue normal Linux commands.

The Linux startup process is controlled by `/etc/inittab`. Since the R3 Falcon release, a convenience script, `boot_hxtek`, has been provided that can toggle `/etc/inittab` between auto-launching the Application Launcher or stopping in the shell. If you are already running in the Application Launcher, launch the Terminal program and type `boot_hxtek`. The next time you reboot, you will stop in the shell.

When the developer using the console wishes to proceed to normal Falcon operation, they can do so by typing the `hxtek` command. That will relaunch the Application Manager into its normal startup routine.

IMPORTANT If you want to get console output from your application, including that from DirectFB, you will have to do so from the startup shell. The Terminal program runs after the Application Launcher has already started making it impossible to get direct output.

2.4.3 Missing programmer keys

To be less intimidating than a regular computer, the Falcon keyboard is missing some typical programmer keys. Some of these missing keys are expected by normal Linux software. If traditional Linux programs are launched, such as vi, which require the escape key to switch editing modes, or another Linux process that you wish to kill with control c, special key combinations can provide the same functionality. (This is true except for DirectFB applications where DirectFB handles the keystrokes directly and does not provide these translations.) The special key combinations require pressing the menu key at the same time as the other key to select the action:

- Escape - Hold down the menu key and then press the [key.
- Control C - Hold down the menu key and c key simultaneously.

2.4.4 Application Manager

Once the Application Manager is running, whether by typing `hxtek` to launch it or just allowing it to autostart from boot, the Application Manager is responsible for providing the system toolbar portion of the menu bar at the top right of the Falcon screen.

The application menu bar may or may not be hidden while an application is running. Pressing the menu key will cause it to reappear so that menu items may be selected.

The Application Manager also provides the ability to launch a new application or choose a previously running application. The Application Manager detects the pressing of the Apps key and displays the Application Launcher to select the next application to run.

The Application Manager builds up a list of available applications to choose from by checking the app.hxi file associated with each application in the /Applications directory. The app.hxi file is explained in **Creating the app.hxi file**. The application file structure is explained below.

2.5 File organization

Most Falcon files are organized exactly as they would be in a normal Linux OS. For Falcon applications, however, there are two special locations.

/Applications/domain folder/bundle folder/app.hxi
/Home
/root
/User/Data/domain folder/application folder/data file

2.5.1 Applications

Falcon applications are based in the /Applications directory. The application itself may be located in various locations, but the Application Manager will always base its idea of where the application is on the api.hxi file that must be located two directories deep from /Applications.

The first level of directories under /Applications is known as domain directories. The domain directory is usually named using reverse DNS lookup, such as com.palm, to ensure uniqueness. The domain directory can contain multiple applications from the same developer or hold shared resources such as libraries or other resources used by the multiple applications.

The second level of directories usually holds the application itself. It must contain a file named app.hxi in order for the Application Manager to find the application. For more information on app.hxi, see **Creating the app.hxi file**. This directory in turn holds other directories that will be used by the application. Because this directory bundles them all together, it is referred to as the *bundle* directory.

The bundle directory may also contain a Resource subdirectory. While Falcon does not have a resource manager like Palm OS or Macintosh, the Resource subdirectory is a convenient place to store such things as special fonts or images used by the application.

Furthermore, if the application uses the `HxDBOpenPreferences` provided by Falcon, the system will create an sqlite database within a Preferences subdirectory. which it also creates within the bundle directory. The exact path to these preferences is {bundle directory}/Preferences/User/.

NOTE While Falcon is a single user system, the groundwork is in the design for multiple users, who will not all be named User.

System support to locate all of these specialized folders is provided by the `hxsystempaths` calls.

NOTE Spaces in pathnames are discouraged within Linux. However, the name displayed by the Application Launcher may have spaces. That is specified by the display name entry of the app.hxi file.

2.5.2 /Home/User/

The second special location provide by the Falcon OS is /Home/User. The main thing to note is that while Falcon does not require you to have a special login account, that

is accomplished by giving the user an account named User, which has normal access privileges. This is where data unique to the user is stored.

Data for each application should be stored here in the Data subdirectory. Within the Data subdirectory, each application will have the same domain and application subdirectories found in the /Applications directory.

2.5.3 System support

While the app.hxi file may have an application's pathname specified, it is not necessary to embed that within your code. The Falcon OS provides system calls for finding the application, data, and other directories for the application. See the `HxSystemPath` calls in the Falcon API Guide.

CHAPTER 3

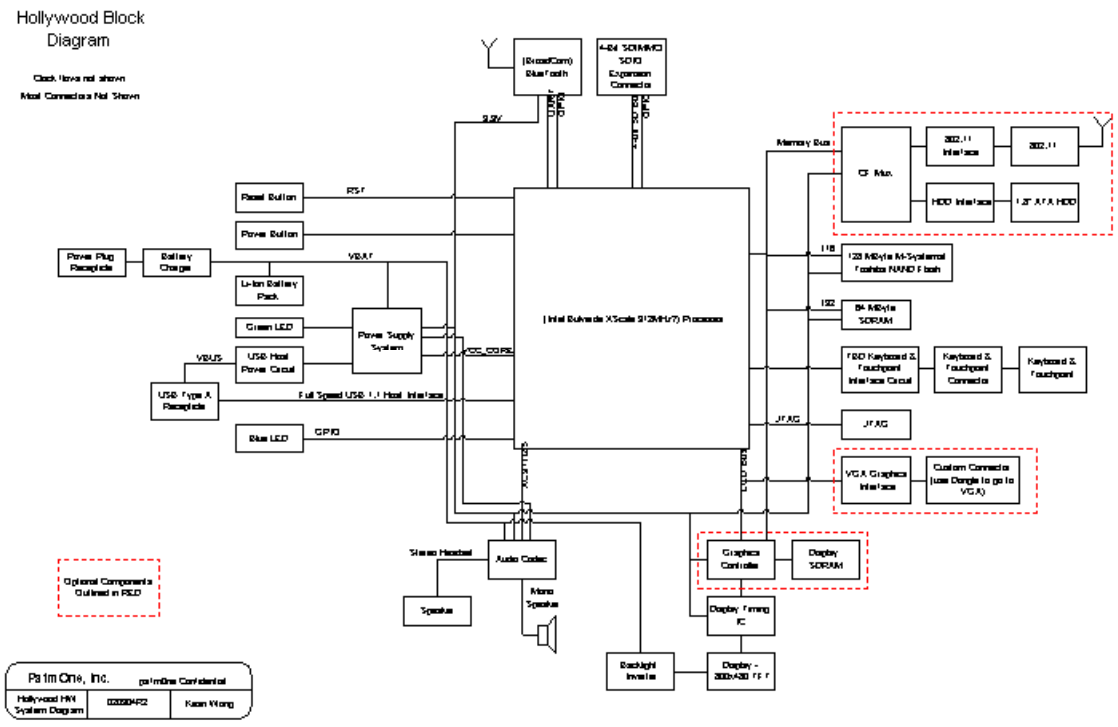
Hardware Developers Kit

This chapter provides details on how to develop for hardware on Falcon devices.

NOTE This information is currently incomplete. If you intend to primarily develop software for Falcon, you may skip this chapter.

3.1 Hardware features

3.1.1 Microprocessor - Intel Bulverde Arm 416MHz



3.1.2 Battery

Falcon's battery is an 8.4 volt Lithium ion battery.

To remove the battery and install a new one:

- Loosen the self-retaining screws until they rotate freely.
- Slide the battery cover forward, and then remove the cover.

- Remove the battery and insert the new one.
- Slide the cover back on, and then retighten the screws.

3.1.3 Memory

Falcon devices have NAND flash memory which is nonvolatile. This NAND Flash houses the storage heap and other data so it is not erased even if the power is drained from the device. Note that NAND memory is serial, whereas NOR memory is random access.

The main difference between NOR and NAND flash memory is that NOR flash is completely XIP (execute in place) and all addresses in a NOR flash are mapped using address pins so that each and every byte can be accessed. In NAND flash memory, data is accessed in blocks. This means that the whole ROM cannot remain in NAND flash memory and execute. However, NAND flash is much better for write and erase functions as it is much faster than NOR flash and the life of NAND flash is longer than that of NOR flash.

Since NOR memory is execute in place, Falcon boots from this memory. Once up and running, programs are loaded from NAND memory into SDRAM for execution.

Falcon has 64KB NOR to boot from, 64MB SDRAM to execute applications, and 128MB NAND for storing the system and applications.

3.1.4 Additional storage

- SD/MMC Card - Place to store user documents
- Compact Flash - HDD hard disk
- USB - The Falcon supports the USB mass storage profile. Any devices such as USB keys, removeable hard drives or optical drives which conform to this profile can be used.

3.1.5 Communication

3.1.5.1 Bluetooth® v1.2

Bluetooth is used to communicate with the paired smartphone. Email is synchronized via the Bluetooth connection and the Falcon can use the DUN (Dial-up Networking) profile to establish an Internet connection via the paired smartphone.

NOTE Bluetooth PAN connections are NOT currently supported.

3.1.5.2 WiFi (802.11b)

The Falcon uses industry standard 802.11b in order to establish a wireless Ethernet connection. This allows the Falcon to connect to the Internet, corporate and personal networks at high speed whenever you are within range of a WiFi access point.

WiFi on the Falcon supports infrastructure and ad-hoc networks. Open, WEP, WPA and 802.1X Enterprise security configurations are available.

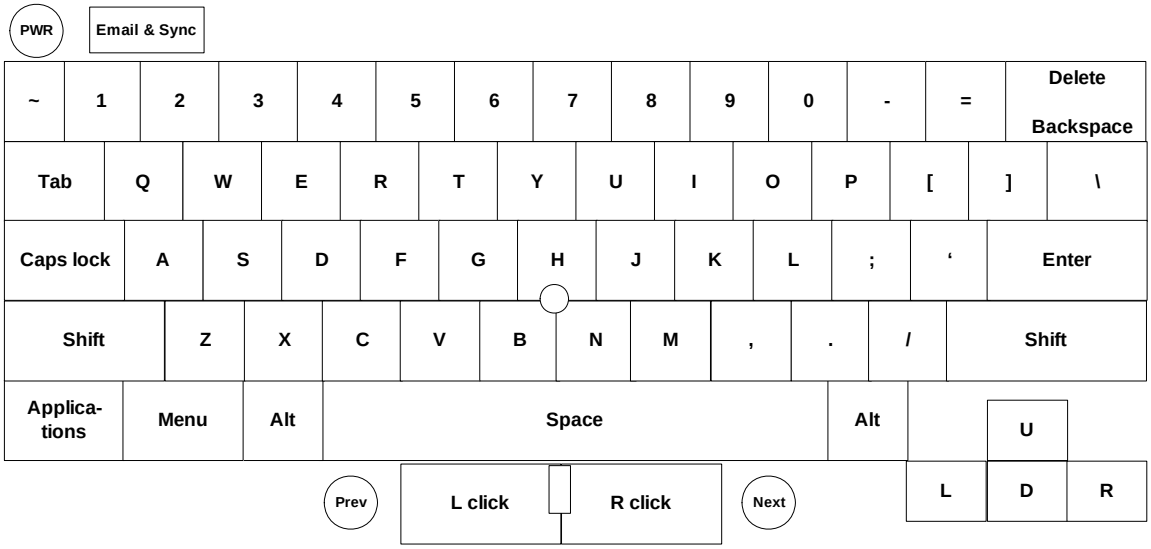
NOTE WPA2 is NOT currently supported.

3.1.5.3 USB

- TCP over USB Ethernet - USB to Ethernet for file transfer
- USB keyboard- and mouse-ready
- USB drives (flash and standard hard drive) serve as removeable storage devices

3.1.6 Direct connect keyboard

- Darfon keyboard with trackpoint cursor control



3.1.7 Video out

Falcon can display to the built-in LCD screen, to XVGA video out, or to a combination of the two. Use <Alt>--<8> to switch between different refresh rates. Both displays use the same screen buffer but with the following different resolution:

- LCD panel - 1024 x 600
- XVGA video out - 1024 x 728

NOTE The Falcon does not stretch the displayed image when using the XVGA output. Instead an extra 128 pixels are added to the bottom of the image. Application developers should take this into consideration when planning for video output scenarios.

3.1.8 HotSync®

The Falcon does not support HotSync operations with the Palm Desktop software. It only supports syncing directly with a supported Treo smartphone or Windows Mobile device.

PART II

Getting Started / Setup

This part of the guide walks you though the initial configuration and setup of a development PC running Ubuntu Linux and the Falcon SDK. It also describes the configuration and setup of a Falcon device.

CHAPTER 4

Getting Started Overview

This chapter discusses the layout of the *Getting Started* guide.

WARNING! This guide is Palm® Confidential. DO NOT distribute it.

4.1 Intended audience

The *Getting Started* guide is intended for anyone new to Falcon development. The guide provides a step-by-step explanation to configuring a Linux-based development system and installing the Falcon SDK. Advanced users will be able to use this guide as a quick reference manual for installing the SDK and setting up a development environment.

4.2 Falcon development

Developing for the Falcon is a different process than developing for PalmOS or Windows Mobile devices. Although Falcon operates like a PDA it is powered by a highly customized version of the open source operating system known as Linux. The current revision of the Falcon runs v2.4 of the Linux kernel.

4.3 Linux operating system

The term Linux doesn't refer to any one specific operating system, rather it is a general term for any open-source, UNIX-like operating system. There are many different vendors, each offering different Linux distributions such as Red Hat, SUSE and Yellowdog. Though the distributions are generally compatible, each one is tuned slightly differently and what works on one distribution may not work directly on another.

IMPORTANT This guide uses Ubuntu 6.06.1 (Dapper Drake) for the base operating system. Advanced users may choose to use alternative Linux operating systems, however, be aware that variances between distributions may require modification of some install and configuration procedures. Alternative operating systems are not fully supported by Palm.

4.4 Prior Linux experience

Developers with Linux experience will find the Falcon immediately familiar and can start writing code right away, simply by installing the Falcon SDK and the GCC tools. Porting an existing app to the device is fairly straightforward, though any programs using a graphical user interface (as opposed to the terminal) will have to use Falcon's HxUI as opposed to existing systems such as GTK.

4.5 New to Linux

For developers without Linux experience, this Getting Started guide will familiarize you with some basic Linux commands as well as walk you through setting up a Linux workstation, installing the Falcon tools, and loading the simulator for the first time.

CHAPTER 5

Ubuntu Overview

This chapter describes the basic features of Ubuntu 6.06.1 (Dapper Drake).

5.1 Ubuntu overview

Ubuntu 6.06 is a complete operating system and is designed to serve as a workstation. It can be installed by itself on a computer or alongside another operating system (such as Windows XP Professional) in a multi-boot environment. While it will not run programs designed for Windows XP directly, many of the programs you are familiar with either have Linux versions or equivalents.

5.2 Major features

Detailing every one of Ubuntu's features is well beyond the scope of this guide, however the following major features will be used while installing and configuring the Falcon SDK. For further information on these, or any other Ubuntu features please refer to the Unofficial Ubuntu 6.06 Starter Guide at http://ubuntuguide.org/wiki/Dapper.

5.2.1 GNOME Desktop

GNOME is shorthand for GNU Object Model Environment. It comprises the graphical user interface (GUI) that you see when Ubuntu 6.06 boots up. The developers behind the project define GNOME as, "the popular, multi-platform free desktop environment, providing all the tools a user needs for everyday work, and a platform for developers to write new software."

In many ways the GNOME desktop is similar to the interface offered by Windows XP or MacOS X. Individual options may be in slightly different locations, however the general user experience matches both Microsoft's and Apple's offerings quite well. If you know how to use a mouse, you can easily navigate around a GNOME desktop.

5.2.2 Terminal

The terminal is also known as the command line. This interface allows users to quickly issue commands without having to use the GUI. You will use the terminal while installing the Falcon SDK and when using the command line GCC tools. After the SDK has been installed, developers have their choice of using command line editors to author code or fancier GUI-based code editors such as Eclipse.

5.2.3 Root user

As a Linux distribution, Ubuntu 6.06 is a permission based operating system. Normal users are limited in their actions in order to enforce various security restrictions. The Root user is a special "superuser" that does not have to obey the standard security rules. When running as Root you can do anything, including wipe out the system with a single command. Root should only be used when it is needed to administer the system. You should never login as the Root user.

5.2.4 Case sensitive

Unlike Windows and MacOS, Ubuntu uses a case sensitive file system by default. As a result, it is possible to have two files with what appear to be the “same” name. For example, Readme.txt, ReadMe.txt and readme.txt would all be different files.

Case sensitivity also applies to commands. Using a capital letter where there should be a lowercase letter, and vice versa, will often result in an error.

5.3 Command Line Tools

Though most of your interaction with the operating system will be done via the GNOME Desktop, certain parts of the install process require the use of command line tools within the terminal. It is beyond the scope of this document to document every single supported command in Ubuntu 6.06, however the most common are listed below:

5.3.1 apt-get

The easiest way to ensure your copy of Ubuntu 6.06 is fully patched and up-to-date is with the command line tool apt-get. We will be using apt-get to update the operating system as well as installing optional build packages that are required for the Falcon SDK.

5.3.2 cd

Short for "change directory" cd allows you to quickly navigate through the various folders on a Linux system when using the terminal. Type "cd" followed by the name of the folder you want to open.

5.3.3 cd ~

The tilde is a special character which means "home directory." No matter where you are currently located in the system, if you type "cd ~" at the terminal prompt and press enter you will be returned to the current user's home directory.

5.3.4 cp

Short for "copy," the cp command is an easy way to copy files from one location to another. Type "cp" followed by the file you want to copy and then the destination.

5.3.5 ls

Short for "list," ls is the Linux equivalent of Microsoft's "dir" command. When used with the -al option, the command gives a detailed description of every file in the current directory. Type "ls -al" to execute the list command.

5.3.6 mkdir

The "mkdir" command is used to create new folders on the system. Type "mkdir" followed by the name of the folder you wish to create.

5.3.7 pwd

Many new users mistake the "pwd" command for the password command. Instead, "pwd" displays your current location, or path, on the system. Type "pwd" at the terminal prompt at any time.

5.3.8 rm

Short for "remove," rm is the command to use when you want to delete a file or folder on the system.

5.3.9 su

The "su" command is used to switch from the current user to the superuser, or root, user account. This requires the root password.

5.3.10 sudo

Similar to the "su" command, "sudo" grants root privileges, but only for the associated command. "Sudo" should normally be used in place of "su" when something must be run as root.

5.3.11 tar -xvf

The "tar" command is analogous to the "pkzip" program for Windows. Issuing the "tar -xvf" command followed by the name of a compressed file tells the machine to decompress the archive and unpack all of the stored files.

5.3.12 uname -a

The "uname -a" command is used to identify the current kernel version of a given Linux system. This is important to know as the kernel version can determine if an application is compatible with a given distribution.

CHAPTER 23

System Features and Widgets

This chapter provides an overview of the Falcon's system features and custom widgets.

23.1 System features

The

23.2 Widgets

CHAPTER 6

SDK System Requirements

This chapter details the requirements for configuring a Falcon development station.

In order to develop applications for the Falcon, you will need a development computer that can successfully run Ubuntu 6.06 (or a compatible version of the Linux operating system). While it is possible to configure a machine for dual-booting Windows XP and Ubuntu, it is recommended that you dedicate a system to Falcon development in order to minimize any possible conflicts.

6.1 Hardware requirements

The development station must meet or exceed the following specifications:

- CPU: Intel Pentium D (or equivalent)
- RAM: 512 MB (1 GB recommended)
- Hard drive: 20 GB or larger
- VESA compatible video card capable of 1280 x 1024 resolution or better
- Ethernet card #1 (for Internet connectivity)
- Ethernet card #2 (for initiating a debugging session with the Falcon)
- Windows Keyboard (must have a Windows button)
- Mouse
- Active Internet connection
- Crossover Ethernet cable

6.2 Software requirements

Although you can download individual software packages as needed, many developers find it helpful to download much of the required software in advance and either store it on a removable hard drive or CD-R until needed.

The following software packages are needed to fully configure the Falcon development environment:

- Ubuntu 6.06 Installation CD - Download from http://www.ubuntu.com/products/GetUbuntu/download#lts
- Falcon SDK - Download the latest Falcon SDK from http://pdn.palm.com

CHAPTER 7

Installing Ubuntu

This chapter provides a detailed guide to install the Ubuntu 6.06 operating system and prepare it for the Falcon SDK.

IMPORTANT By default this process WILL COMPLETELY WIPE THE HARD DRIVE OF THE DEVELOPMENT MACHINE and configure it exclusively for Falcon development. Any pre-existing data WILL BE LOST.

7.1 Create the Ubuntu CD

Before you can install Ubuntu 6.06, you must first create an installation CD. To do so, you will need a computer with an Internet connection, a CD-R drive and approximately 1 GB of free hard drive space.

1. Open a web browser and navigate to http://www.ubuntu.com/products/GetUbuntu/download#lts
2. Click a local mirror (listed by country)
3. Click the “PC (Intel x86) desktop CD”
4. Save the file “ubuntu-6.06.1-desktop-i386.iso” to the local hard drive
5. Open a CD burning program (such as Nero)
6. Choose the “Burn CD Image” option
7. Load the file “ubuntu-6.06.1-desktop-i386.iso”
8. Place a blank CD-R disc in the CD-R drive
9. Confirm your selections and burn the CD

NOTE Depending on the speed of your Internet connection and CD-R drive, the creation of the Ubuntu 6.06 install CD can take anywhere from 20 minutes to an hour or more.

Once the process is completed, label the new install CD and bring it over to the development machine.

7.2 Booting the install CD

The Ubuntu 6.06 Install CD is pre-configured as a Live CD. This means that the disc will boot into a fully functioning operating system on any computer with compatible hardware. Booting the Live CD is an easy way to verify that your hardware is compatible with Ubuntu 6.06. If the CD boots into the GNOME Desktop, Ubuntu 6.06 will work on your hardware.

IMPORTANT If you have not yet done so, configure the development machine to boot from the CD-ROM instead of the hard drive. One some machines this is done in the BIOS. On others, it is a key command that must be pressed during POST. See your hardware documentation for details on your specific system.

7.3 The install process

7.3.1 Live CD menu

When the Ubuntu Live CD boots, it will present you with a system menu offering various diagnostic and configuration options. Choose the first option, “Start or install Ubuntu” and press <enter> to confirm.

Wait for the GNOME Desktop to load.

NOTE If the desktop does not load for any reason (garbled screen, system crash, etc.) it usually means either the created CD is defective or the hardware is not fully compatible with the default Ubuntu 6.06 configuration. Most incompatibilities can be resolved by booting into the "safe" configuration from the Ubuntu 6.06 splash screen. If that does not resolve the issue, refer to http://www.ubuntu.com for further documentation.

7.3.2 Start the installer program

Once the GNOME Desktop has loaded, look for the “Install” icon in the upper left-hand-corner of the screen. Using the mouse, double-click on the “Install” icon to launch the Ubuntu installer program.

7.3.3 Language selection

Select “English” from the menu, then click on “Forward” to continue.

7.3.4 Set the system clock

Select your current time zone from the menu. If your city is not indicated, choose another major city within the same time zone. For example, Palm is near San Francisco, however San Francisco is not an option. Instead, we chose Los Angeles.

After setting the time zone, be sure to double check the system time. If it is incorrect, choose the “Set TIme” option to configure it.

Click “Forward” to continue.

7.3.5 Select a keyboard layout

Select your keyboard layout from the menu and click “Forward” to continue.

7.3.6 Create the default account

The default account is the main account on the system. Most of the time it will run as a normal user, however the default account also has administrator privileges. It is very important that you safeguard the password to this account, especially if the development station is going to be on a network.

Enter your full name in the first box.

Enter your desired username in the second box.

Enter your desired password in the third and fourth boxes. The password must be entered twice to ensure that it was not mistyped.

Enter a name for the computer in the fifth box.

Click “Forward” to continue.

7.3.7 Configure the hard drive

This step in the process tells the operating system how much of the computer’s hard drive it is allowed to use. If you are configuring the development station as a dedicated system, select “Erase entire disk.”

If you plan to dual-boot between Ubuntu and another operating system, select “Manually edit partition table.”

Click “Forward” to continue.

NOTE Configuring a multi-boot system requires advanced knowledge of all operating systems to be used on the machine and is beyond the scope of this guide. It is recommended that the development machine be dedicated to Ubuntu 6.06.

7.3.8 Confirm configuration settings

This is the last step before changes are actually made to the machine. Carefully review all system settings on the summary page and ensure they are correct. If there are any errors click the “Back” button and correct them.

If all of the system settings are correct, click “Install” to begin copying Ubuntu 6.06 to the development machine. This section of the install will take approximately 10-20 minutes to complete.

IMPORTANT Once you click "Install" the system will start writing to the selected partitions on the hard drive and any pre-existing data WILL BE LOST. If you are installing Ubuntu 6.06 on a machine that was previously in use be sure to BACK UP ANY IMPORTANT DATA before beginning the install process.

7.3.9 Reboot the system

Once the install prcess has finished, an “Installation Complete” window appears on screen. Click “Restart now” and wait for the system to shut down.

After the shutdown process completes, the message “Please remove the disc, close the tray (if any) and press ENTER to continue” appears on the screen. The CD should have automatically ejected by now. If it has not, press the eject button on the CD/DVD-ROM drive.

Remove the CD from the system. If the drive tray is open, close it.

Press <enter> to reboot the system.

7.4 Update system software

Ubuntu 6.06 is installed, but now it has to be updated. Much like the automated systems used in Windows XP and MacOS X, updating Ubuntu 6.06 is an easy task, however it does REQUIRE an active connection to the Internet. The easiest way to configure such a connection is via broadband Ethernet as Ubuntu 6.06 is designed to work seamlessly on a network with DHCP.

NOTE Many systems ship with 802.11b/g wireless Ethernet, otherwise known as WiFi. While this configuration is supported in Ubuntu 6.06, configuring WiFi is beyond the scope of this Getting Started guide due to the many different adapters that are on the market. If you wish to use WiFi, refer to the Ubuntu 6.06 documentation for more details.

After the system has rebooted you are presented with a login screen. Enter your username and password to bring up the desktop.

7.4.1 Upgrading the core system

1. Click the Applications menu
2. Click Accessories
3. Click Terminal to start a terminal session
4. Type “sudo apt-get update” and press <enter>
5. Enter your password and press <enter>
6. Wait for the update to finish
7. Type “sudo apt-get upgrade” and press <enter>
8. Type Y at the prompt and press <enter>
9. Wait for the upgrade to finish

NOTE The system may display a prompt asking you to reboot the system. This prompt can safely be ignored as no reboot is required at this time.

7.4.2 Install xCHM viewer

Compressed HTML, or CHM, is a standard file format for packaging help files across various platforms. In order to view CHM files on the development PC a viewer application is needed.

1. Open a terminal session
2. Type “cd /etc/apt” and press <enter>
3. Type “sudo nano sources.list” and press <enter>
4. If prompted, enter your password and press <enter>
5. Look for the following block of text:

```
## Uncomment the following two lines to add software from the 'universe'
## repository.
## N.B. software from this repository is ENTIRELY UNSUPPORTED by the Ubuntu
## team, and may not be under a free licence. Please satisfy yourself as to
## your rights to use the software. Also, please note that software in
## universe WILL NOT receive any review or updates from the Ubuntu security
## team.
# deb http://us.archive.ubuntu.com/ubuntu/ dapper universe
# deb-src http://us.archive.ubuntu.com/ubuntu/ dapper universe
```

6. Delete the # symbol from the last two lines. The edited block should look like this:

```
## Uncomment the following two lines to add software from the 'universe'
## repository.
## N.B. software from this repository is ENTIRELY UNSUPPORTED by the Ubuntu
## team, and may not be under a free licence. Please satisfy yourself as to
## your rights to use the software. Also, please note that software in
## universe WILL NOT receive any review or updates from the Ubuntu security
## team.
deb http://us.archive.ubuntu.com/ubuntu/ dapper universe
deb-src http://us.archive.ubuntu.com/ubuntu/ dapper universe
```

7. Press <ctrl>-o followed by <enter> to save the edited file
8. Press <ctrl>-x to exit nano
9. Type “sudo apt-get update” and press <enter>
10. Type “sudo apt-get install xchm” and press <enter>
11. If prompted, enter your password and press <enter>
12. At the “Do you want to continue [Y/n]?” prompt, type Y and press <enter>

7.5 Download kernel headers

As the core of your operating system, the kernel is the most important item on the development machine. Any application that is going to run on the system (such as the Fusion component of the Falcon SDK) needs to be properly configured against the running kernel. While most of this is done automatically, the current kernel source files need to be installed and prepped for the automatic configuration to happen during the SDK install.

7.5.1 Verify current kernel version

1. Open a terminal session
2. Type “uname –a” and press <enter>

3. The resulting output lists the current kernel version (ex. 2.6.15-26-386)

7.5.2 Download current kernel headers

1. Make a note of the EXACT kernel version reported by uname (ex 2.6.15-26-386)
2. Type “sudo apt-get install build-essential linux-headers-2.6.15-xx-xxx” (where xx-xxx represents the kernel version running on your machine) and press <enter>
3. If prompted, enter your password and press <enter>
4. At the “Do you want to continue [Y/n]?” prompt, type Y and press <enter>

IMPORTANT It is very important that the proper version of the kernel source is downloaded. In the listed example, the kernel source was 2.6.15-26-386 so the file installed was linux-headers-2.6.15-26-386. If uname reports a different kernel version on your system during the verification step be sure to edit the linux-headers file name so that the proper version of the source files are downloaded.

CHAPTER 8

Falcon SDK

8.1 Install the Falcon SDK

Now that the development system is prepped and ready to go, you need to download and install the Falcon SDK. The Falcon SDK includes the necessary header files, and API guide and a full system simulator. The Falcon SDK also contains updated product ROM images which should be used to update your Falcon device to the latest software revision.

NOTE Don't forget, all commands are case sensitive. If you receive an error after entering a command be sure to double check the spelling.

8.2 Download the Falcon SDK

1. Click the Applications menu
2. Click Internet
3. Click Firefox Web Browser to open Firefox
4. Navigate to http://pdn.palm.com
5. Login to the site with your email and password
6. Navigate to the Falcon section
7. Download the current SDK package
8. Click "Save to Disk"
9. Click "OK"
10. Close any Firefox windows

8.3 Install the Falcon SDK

8.3.1 Unpack the Falcon SDK

1. The SDK package you downloaded should be visible on the desktop
2. Open a terminal session
3. Type "cd ~" and press <enter>
4. Type "cd Desktop" and press <enter>
5. Type "tar –xvf ./*name of SDK*" and press <enter>

8.3.2 Configure Fusion

1. Open a terminal session
2. Open a terminal session
3. Type “cd ~” and press <enter>
4. Type “cd Desktop” and press <enter>
5. Type “cd Build” and press <enter>
6. Type “sudo copy 99-fusion.rules /etc/udev/rules.d” and press <enter>
7. If prompted, type your password and press <enter>
8. Type “sudo chmod 644 /etc/udev/rules.d/99-fusion-rules” and press <enter>
9. Type “cd /etc” and press <enter>
10. Type “sudo nano modules” and press <enter>
11. Add the following text to the end of the file:

```
# Run Fusion at boot

fusion
```

12. Press <ctrl>-o followed by <enter> to save the edited file
13. Press <ctrl>-x to exit nano

8.3.3 Install core Falcon SDK

1. Open a terminal session
2. Type “cd ~” and press <enter>
3. Type “cd Desktop” and press <enter>
4. Type “cd Build” and press <enter>
5. Type “sudo chmod 777 /opt” and press <enter>
6. If prompted, type your password and press <enter>
7. Type “./sdk-installer.sh” and press <enter>
8. Midway through the install, the fusion module will ask for the path to the kernel source. It should default to the proper location, however if your kernel headers are located elsewhere enter the correct path now. For example, “/usr/src/linux-source-2.6.15”
9. Press <enter> to confirm the selection and finish installation

NOTE The most common issue when installing the Falcon SDK is the fusion module failing to build properly. This often happens when trying to install the SDK on an existing system which has a number of custom environment variables set.

If the fusion module fails to build ensure that the ARCH, CROSS_COMPILE and CC variables are set to the appropriate values for your development PC.

8.3.4 Edit .bashrc file

1. Open a terminal session
2. Type “cd ~” and press <enter>
3. Type “nano .bashrc” and press <enter>
4. Scroll down to the bottom of the file
5. Press <ctrl>-r to open the read file menu
6. You will see a “File to insert [from ./] :” prompt
7. Type “/opt/holly/docs/bash_functions” and press <enter>
8. Press <ctrl>-o followed by <enter> to save the edited file
9. Press <ctrl>-x to exit nano

8.4 Reboot the development station

1. Open a terminal session
2. Type “sudo shutdown –r now” and press <enter>
3. Wait for the machine to shutdown and reboot
4. Enter your username and password at the login screen

8.5 Launch the simulator

The Falcon simualtor requires the SDK to be properly installed in order to run, so launching the simulator is an easy way to ensure that everything has been installed properly.

There are two ways to launch the simulator. One is a manual process, the other is via a shortcut in the .bashrc file.

8.5.1 Manually launch the simulator

1. Open a terminal session
2. Type “LD_LIBRARY_PATH=/opt/holly/sysroot/i386-linux/usr/lib /opt/holly/sysroot/i386-linux/usr/bin/hxtek” and press <enter>
3. If the Falcon SDK is installed, the simulator starts
4. Type <ctrl>-c in the terminal window to close the simulator

8.5.2 Launch via a shortcut

1. Open a terminal session
2. Type “dohxtek” and press <enter>
3. Type <ctrl>-c in the terminal window to close the simulator

CHAPTER 9

Flashing the Falcon

9.1 Falcon ROM images

The Falcon system software is provided as a ROM image which can be flashed to the device via a SD card. Two major versions of the system software are provided under NDA, a *Core* ROM and a *Product* ROM. Each of these is also available in a debug version and a release version.

The Core ROM is a bare system image that does not contain any bundled software. This image is only available to developers and will NOT be provided to end users.

The Product ROM is a complete system image which includes all bunded software. This image will be loaded on the Falcon when it ships to end users.

IMPORTANT Because the Falcon does not have the ability to perform a "hard reset" it is important to always have a copy of the current ROM image on SD card. Flashing the device from the SD card is the ONLY WAY to revert to a clean system image.

9.1.1 System requirements

In order to flash the Falcon with a new ROM image you will need the following:

- Computer with Internet access
- Active PDN developer account
- SD card reader
- SD card (64 MB or larger)

9.2 Acquiring the ROM image

Falcon ROM images are provided exclusively through PDN. ROM images will NOT be provided to anyone outside of PDN.

If you do not yet have an account on PDN, open a web browser and nagivate to http://pdn.palm.com and sign up for an account. After registering on PDN, contact your Palm Internal Sponsor in order to gain access to the secure Falcon section.

NOTE You can always check the build version of the currently installed ROM by turning on the Falcon and going to the System Preferences screen. Click on System, then click on the "Details" button next to the System Info header. The System Info Details window displays the currently installed build version.

9.2.1 Core ROM

The Falcon Core ROM is the basis for the entire system. While every application must pass testing with the Product ROM, the Core ROM can be useful when you want to test your application with a minimum amount of services running. Because the Core ROM does not contain bundled applications, there is more available user space for storing core dumps and log files.

9.2.1.1 Download the Core ROM

1. Login to the PDN site at http://pdn.palm.com
2. Click “develop”
3. Click “platform SDKs”
4. Click “Falcon”
5. Look for the “Falcon Resources” box in the upper right hand corner
6. Click “Core ROM”
7. If you want the standard ROM, click “Falcon_CoreRom-Release-*XXXX*.tar.gz”
8. If you want the debug ROM, click “Falcon_CoreRom-Debug-*XXXX*.tar.gz”
9. Save the selected file to your desktop

9.2.2 Product ROM

The Falcon Product ROM is the complete system image. Along with the operating system, the Product ROM also contains the full suite of Palm applications, including Documents To Go, Email, PDF Viewer, PDGF Viewer, Photo Viewer and Web Browser.

9.2.2.1 Download the Product ROM

1. Login to the PDN site at http://pdn.palm.com
2. Click “develop”
3. Click “platform SDKs”
4. Click “Falcon”
5. Look for the “Falcon Resources” box in the upper right hand corner
6. Click “Product ROM”
7. If you want the standard ROM, click “Falcon_ProductRom-Release-*XXXX*.tar.gz”
8. If you want the debug ROM, click “Falcon_ProductRom-Debug-*XXXX*.tar.gz”
9. Save the selected file to your desktop

NOTE The Core ROM and Product ROM will NOT have the same build numbers. Check the Product ROM release notes to determine the exact Core ROM used as a base for a particular build.

9.3 Prepare the SD card

The Falcon can recognize a number of filesystems, including FAT16 which is what the device uses for SD cards. If your SD card is not formatted as FAT16 you should do so before using it. This can either be done on your desktop computer or on the Falcon itself.

9.3.1 Format the SD card

1. Turn on the Falcon
2. Press the Apps button to bring up the Application Launcher
3. Click File Manager to start the File Manager
4. Insert the SD card into the Falcon
5. Navigate to the menu bar at the top of the screen
6. Highlight the Tools menu and click Format
7. Select SD Card and click OK
8. Enter a name for the SD card and click OK
9. When you see the “Formatting was successful” message click OK
10. The SD card is now ready for use

9.4 Copy ROM files to SD card

9.4.1 Unpack the ROM image

No matter which version of the Falcon ROM you decide to use, Core or Product, release or debug, the ROM image is provided as a compressed TAR.GZ file. This is a native Unix format, however many common file compression utilities for Windows can manipulate TAR.GZ files.

NOTE Palm does not officially recommend any particular software, however both WinZip (http://www.winzip.com) and WinRAR (http://www.rarlab.com) are known to work with the TAR.GZ file format under Windows XP.

1. Locate the Falcon ROM file you downloaded
2. Decompress the file into a temporary folder
3. You should see the following files:

```
BUILD_VERSION
blob-holly-evt2
blob-holly-evt2.md5
blob-holly-evt3
blob-holly-evt3.md5
blob-holly-evt4
blob-holly-evt4.md5
blob-holly-evt5
blob-holly-evt5-128M
blob-holly-evt5-128M.md5
blob-holly-evt5.md5
blob-holly-evt6-128M
blob-holly-evt6-128M.md5
```

```
mtdpt.128MB
mtdpt.128MB.md5
mtdpt.256MB
mtdpt.256MB.md5
rootfs.tar.gz
rootfs.tar.gz.md5
zImage
zImage.md5
```

9.4.2 Copy ROM files to SD card

1. Insert the FAT16 formatted SD card into your card reader
2. If necessary, mount the SD card as a drive on your system
3. Copy the Falcon ROM files from the temporary folder to the SD card
4. Eject the SD card

IMPORTANT The Falcon ROM files must be located in the root folder of the SD card. If the SD card has more than one partition, the Falcon ROM files must be located in the root folder of the first partition.

9.5 Flash the Falcon

Once the SD card is prepared, flashing the Falcon is a simple process. If an SD card is inserted into the device before a cold boot, the system automatically checks for the presence of a ROM update. If one is present, the user is presented with the flash interface.

IMPORTANT Flashing the Falcon with a ROM update WILL DELETE ALL DATA CURRENTLY ON THE DEVICE. Be sure to backup any stored data before performing a ROM update as it will be deleted if left on the Falcon.

---

1. Insert the SD card into the Falcon
2. Hold down the power button until you see the Palm logo and the "Rebooting..." text on the screen
3. Wait for the Falcon to restart
4. After the system finishes initializing, it will scan the SD card for ROM files
5. If the system finds ROM files on the SD card, the following dialogue is displayed:

```
*****************************************************************
*
*                       FALCON FLASH UPDATER
*
*****************************************************************

Waiting for NAND Flash to become available.
Found Build version XXXXX on SD/MMC Card.

Checking the checksum for zImage...done.
Checking the checksum for rootfs.tar.gz...done.
Checking the checksum for mtdpt.256MB...done.
Checking the checksum for blob-holly-XXXX-128M...done.

The following files will be installed:
 blob-holly-XXXX-128M
 zImage
 rootfs.tar.gz
 mtdpt.256MB

Reflash device with this Build version? (y/n):
```

NOTE If the Falcon boots into the standard user interface it did not find the necessary ROM files on the SD card. Return to the previous step and ensure you have properly created the SD card.

6. Type "y" and press <enter> to start the flash process
7. Wait for the flash process to finish. You will see the individual files scrolling across the screen while the flash is in progress

8. When you see the following screen, the update process is complete:

```
*****************************************************************
*
*                          UPDATE COMPLETE
*
*         Remove the SD/MMC Card and HIT RETURN to reboot.
*
*****************************************************************
```

9. Eject the SD card from the device
10. Press the <enter> key to reboot the Falcon with the new ROM

CHAPTER 10

Mailsync Setup

10.1 What is Mailsync?

The Falcon Mailsync library is a utility that allows the Falcon to stay in sync with the active e-mail accounts on the linked smartphone. No complicated e-mail setup is necessary on the Falcon itself. So long as the Mailsync library is installed on a compatibile Palm OS Treo or Windows Mobile 5.0 smartphone all existing email accounts automatically appear on the Falcon.

10.2 Mailsync compatiblity

When the Falcon ships to retail, the Mailsync library will be officially supported on the following devices:

- Palm Treo 680 (Palm OS)
- Palm Treo 700p (Palm OS)
- Palm Treo 700w (Windows Mobile 5.0)
- Palm Treo 700wx (Windows Mobile 5.0)
- Palm Treo 750 (Windows Mobile 5.0)
- Palm Treo 750v (Windows Mobile 5.0)

IMPORTANT The Palm Treo 650 (and earlier devices) is NOT supported by the Falcon Mailsync library. Under no circumstances will Palm provide any support for the Falcon Mailsync library and the Palm Treo 650.

IMPORTANT No developer access to the Mailsync library is provided in the SDK. This should be considered a closed system with no third party access.

10.2.1 Current compatibility status

Because the Mailsync library is still in the beta stage, developers may experience issues with the software on the 700 and 750 series devices. Currently the Palm Treo 680 is the only device fully working with the Mailsync library. If developers wish to test the Falcon's Mailsync feature you should acquire a Cingular-branded Palm Treo 680.

Developers who have a pre-release version of the Palm Treo 680 will need to ensure that the device is upgraded to the Gold Master Cingular ROM. If you do not know how to do this, please contact devsupport@palm.com.

The Gold Master Cingular ROM has the following build numbers:

- ROM Build: 3131
- Firmware: R01.31

NOTE If you have a retail version of the Cingular-branded Palm Treo 680 it already has the Gold Master ROM loaded.

10.2.2 Checking the Palm Treo 680 ROM version

10.2.2.1 Check the ROM build number

1. Press the "Applications" button on the Treo
2. Press the "Menu" button to bring up the menu bar
3. Tap the "Options" menu
4. Using the stylus, tap and hold "About Applications"
5. Press and hold down on the 5-way button
6. Release the stylus
7. Release the 5-way button
8. The current ROM Build number is displayed on screen

10.2.2.2 Check the Firmware build number

1. Press the "Phone" button on the Treo
2. Press the "Menu" button to bring up the menu bar
3. Tap the "Options" menu
4. Tap "Phone Info"
5. The current Firmware is displayed on screen

10.3 Acquiring the Mailsync library

When the Falcon ships to end users, the Mailsync library will be included in the retail box. In the interim, developers can acquire the software by accessing the Falcon section of PDN and downloading the appropriate software.

If you are using a Palm OS Treo smartphone, the Mailsync library is integrated into Versamail 4.0. If you are using a Windows Mobile 5.0 smartphone the Mailsync library is a separate application.

10.3.1 Versamail 4.0 for Palm OS

6. Login to the PDN site at http://pdn.palm.com
7. Click “develop”
8. Click “platform SDKs”
9. Click “Falcon”
10. Look for the “Falcon Resources” box in the upper right hand corner
11. Click “Product ROM”
12. Click “VM40_Installer.prc”
13. Save the selected file to your desktop

10.3.2 Mailsync library for Windows Mobile 5.0

1. Login to the PDN site at http://pdn.palm.com
2. Click “develop”
3. Click “platform SDKs”
4. Click “Falcon”
5. Look for the “Falcon Resources” box in the upper right hand corner
6. Click “Product ROM”
7. Click “PalmSyncSvc.ARMV4I.CAB”
8. Save the selected file to your desktop

10.4 Install Versamail 4.0

Installing Versamail 4.0 is just like installing any other Palm OS application. It can be installed via Hotsync or SD card.

IMPORTANT If you are installing Versamail 4.0 for the first time, use the clean install instructions. If you are upgrading an existing copy of Versamail 4.0 use the upgrade install instructions.

10.4.1 Clean install via SD card

1. Insert a SD card into the computer
2. If a “PALM” folder does not exist on the SD card, create one
3. Double click on the “PALM” folder
4. If a “Launcher” folder does not exist within the “PALM” folder, create one
5. Copy the file “VM40_Installer.prc” into the “Launcher” folder on the SD card
6. Eject the SD card from the computer
7. Insert the SD card into the Palm Treo 680
8. The SD card should automatically appear on screen. If it does not, press the “Applications” button until the contents of the SD card are displayed
9. Tap “Install Email” to start the Versamail 4.0 installer
10. Tap “Update Now” to install Versamail 4.0
11. Reset the device

10.4.2 Upgrade install via SD card

1. Insert a SD card into the computer
2. If a “PALM” folder does not exist on the SD card, create one
3. Double click on the “PALM” folder
4. If a “Launcher” folder does not exist within the “PALM” folder, create one
5. Copy the file “VM40_Installer.prc” into the “Launcher” folder on the SD card
6. Eject the SD card from the computer
7. Insert the SD card into the Palm Treo 680
8. The SD card should automatically appear on screen. If it does not, press the “Applications” button until the contents of the SD card are displayed
9. Tap “Install Email” to start the Versamail 4.0 installer
10. Press the “r” button on the keyboard to upgrade to the current build of Versamail 4.0
11. Reset the device

10.4.3 Configure email accounts

Before syncing with the Falcon, you will need to properly setup your email accounts within Versamail 4.0.

10.5 Install the Mailsync library for Windows Mobile

The Mailsync library for Windows Mobile ships as a CAB file which can be installed via ActiveSync or miniSD/SD card.

IMPORTANT Your email account settings should be configured in Pocket Outlook before installing the Mailsync library.

10.5.1 Clean install via miniSD/SD card

1. Insert a miniSD/SD card into the computer
2. Copy the file “PalmSyncSvc.ARMV4I.CAB” onto the miniSD/SD card
3. Eject the miniSD/SD card from the computer
4. Insert the miniSD/SD card into the Windows Mobile 5.0 smartphone
5. Tap “Start”
6. Tap “Programs”
7. Tap “File Explorer”
8. Select “Storage Card” from the drop-down menu on the top of the screen
9. Tap the file “PalmSyncSvc.ARMV4I.CAB” to install the Mailsync library
10. The device will display a warning asking if you want to continue. Tap “Yes”
11. Select “Device” when prompted for an install location
12. Tap “Install”
13. Tap “Yes”
14. As soon as the install completes, the device automatically performs a soft reset

10.6 Installed files

10.6.1 Versamail 4.0

The following files are installed by Versamail 4.0:

- "AboutBoxLib.prc
- "Address.prc
- "AddressLib.prc
- "AirSAM.prc
- "AirSAMPrv.prc
- "ALIData.prc
- "ALIDataViewer.prc
- "BGService.prc
- "CategoryLib.prc
- "Datebook.prc
- "DatebookLib.prc
- "DebugPrefs.prc
- "DmTxn.prc
- "EBS.prc
- "HollySAM.prc
- "HxSML.prc
- "HxSync.prc
- "IIDataLib.prc
- "Interceptor.prc
- "Memo.prc
- "MIDataLib.prc
- "MMConduit.prc
- "MMConfigFPI.prc
- "MMDiskLib.prc
- "MMHtmlPlugin.prc
- "MMMailTo.prc
- "MMNotify.prc
- "MMPhotos.prc
- "MMPluginMGR.prc
- "MMPRCPlugin.prc
- "MMSDCard.prc

- "MMSmartAdd.prc
- "MMTextPlugin.prc
- "MMUnzipPlugin.prc
- "MMUpgrade.prc
- "MMVCal.prc
- "MMVCardPlugin.prc
- "MMWave.prc
- "MMWizard.prc
- "MMWordPlugin.prc
- "MultiMail.prc
- "PalmVMFontLibrary.prc
- "PassThruSync.prc
- "PopSAM.prc
- "SAMgr.prc
- "TextLookupFPI.prc
- "ToDo.prc
- "TxnLogLib.prc
- "VMContactDA.prc
- "VMEmailDA.prc
- "VMNetLogging.prc

10.6.2 Mailsync library for Windows Mobile 5.0

The following files are installed by the Mailsync library:

- "\Windows\PalmSyncSvc.dll
- "\Windows\EmailDataAdapter.dll
- "\Windows\ContactDataAdapter.dll
- "\Windows\CalendarDataAdapter.dll
- "\Windows\vCard_vCalendar.dll
- "\Program Files\HxSync\hxsml.dll
- "\Program Files\HxSync\hxsync.dll
- "\Program Files\HxSync\Data\devinf11.xml
- "\Program Files\HxSync\Log\PalmTxnLog.vol
- "\Program Files\HxSync\Log\SyncLog.txt
- "\Program Files\HxSync\BUILD.xtx
- "\Program Files\HxSync\SimpleLog.exe
- "\Program Files\HxSync\DebugSettings.exe
- "\Program Files\HxSync\SyncedOnce.exe
- "\Program Files\HxSync\TestDAs.exe

CHAPTER 11

Falcon Setup

11.1 Initial configuration

Flashing the Falcon ROM has much the same effect as a "hard reset" on a Treo. All of the current system settings, custom preferences and user data are wiped clean and replaced with a new copy of the operating system. This chapter walks you through the initial Falcon configuration and prepares the device for use.

11.2 Language and license

After the initial boot, the language selection and license agreement screen appears.

Select your language by clicking on English or Español (Spanish).

Check the box which indiactes agreement to the terms and conditions of the Falcon software license agreement. Once this is done, the "Next" button lights up.

Click "Next" to proceed.

NOTE You can read the full text of the Falcon software license agreement by clicking on the blue "Software License Agreement" text.

11.3 Date and time

This screen allows you to configure the Falcon's locale (important for testing localized software), the date, time and current time zone. All time zone settings are relative to GMT.

Click "Next" to proceed.

11.4 Phone setup

The phone setup section allows you to easily configure a compatible Palm OS Treo or Windows Mobile device for e-mail sync as well as Internet access.

If you do not wish to setup a phone at this time, click "Skip" to finish setup. Otherwise click on "Next" to proceed.

NOTE Standard phones (such as the Motorola RAZR) can also be paired with the Falcon, however they will only provide dial-up networking (DUN) access functionality and will not be able to sync. These types of phones cannot be setup through the wizard. They must be configured manually in the Falcon System Preferences.

11.4.1 Choose device and carrier

1. Select the type of phone you wish to use by choosing “Palm OS smartphone” or “Windows Mobile phone” from the drop-down menu
2. Select your carrier from the drop-down menu
3. Click “Next” to proceed

11.4.1.1 Custom carrier settings

If your carrier is not listed in the menu, you will need to enter the data connection settings manually.

1. Select “Other” from the drop-down menu
2. Click “Edit” to open the “Other Carrier Settings” dialogue box
3. Choose your connection type, GPRS or CDMA
4. Enter the APN, or access point name, used by your carrier. This is not required for CDMA carriers
5. Enter the username provided by your carrier. This is not required for CDMA carriers
6. Enter the password provided by your carrier. This is not required for CDMA carriers
7. Click “Ok”

IMPORTANT The APN, username and password information is provided by your carrier. Without the correction information, the Falcon will not be able to go online. Some GPRS carriers require a username and password, others require the fields stay blank. Please check with your carrier before entering custom settings.

11.4.2 Bluetooth pairing

In order for the Falcon to communication with your smart phone, Bluetooth must be enabled on both devices. If you are familiar with Bluetooth pairing, click “Advanced” to proceed directly to the pairing screen. If you are not familiar with Bluetooth pairing, click “Next” to proceed through the walkthough.

11.4.2.1 Advanced pairing method

Clicking on “Advanced” brings up a quick-connect Bluetooth screen which lists the Falcon’s device name, along with a default passcode. You need to use this name and passcode when pairing the Falcon with your phone.

1. Enable the Bluetooth radio on your phone
2. Initiate a new pairing from your phone
3. Wait for the Falcon to show up in the device list. By default the Falcon has a device name of “Palm”
4. Connect to “Palm” on your phone

5. When the wizard asks for a security key enter the six digit passcode that is displayed on the Falcon's screen
6. The "Pairing successful" screen appears when the Bluetooth pairing has successfully completed
7. Click "Next" to continue
8. After pairing, the Falcon automatically starts downloading e-mail from the Treo
9. Click "Next" to continue
10. Press the Mail button on the Falcon to launch the Mail application and complete setup

11.4.2.2 Regular pairing with Palm OS Treo

If you selected a Palm OS smartphone as your device type, the Bluetooth pairing wizard will automatically walk through the steps necessary to pair the Palm OS Treo with the Falcon.

1. Turn on the Palm OS Treo
2. Press the Applications (Home) button on the Treo to open the main menu
3. Click "Next" on the Falcon
4. Tap the Bluetooth icon on the Treo
5. Tap "Setup Devices" on the Treo
6. Click "Next" on the Falcon
7. Tap "Trusted Devices" on the Treo
8. Tap "Add Device" on the Treo
9. Click "Next" on the Falcon
10. Tap "Nearby Devices" on the Treo
11. Highlight the device "Palm" in the device list on the Treo. This is the default name for the Falcon
12. Tap "OK" on the Treo
13. Click "Next" on the Falcon
14. Find the six digit Bluetooth security key on Falcon screen
15. Enter the security key on the Treo
16. Tap "OK" on the Treo
17. The "Pairing successful" screen appears on the Falcon when the Bluetooth pairing has successfully completed
18. Click "Next" to continue
19. After pairing, the Falcon automatically starts downloading e-mail from the Treo
20. Click "Next" to continue
21. Press the Mail button on the Falcon to launch the Mail application and complete the setup

11.4.2.3 Regular pairing with Windows Mobile 5.0

If you selected a Windows Mobile 5.0 smartphone as your device type, the Bluetooth pairing wizard will automatically walk through the steps necessary to pair the Windows Mobile 5.0 smartphone with the Falcon.

1. Turn on the Windows Mobile 5.0 smartphone
2. Tap “Start” on the Windows Mobile 5.0 smartphone
3. Click “Next” on the Falcon
4. Tap “Settings” on the Windows Mobile 5.0 smartphone
5. Tap the “Connections” tab on the Windows Mobile 5.0 smartphone
6. Click “Next” on the Falcon
7. Tap “Bluetooth” on the Windows Mobile 5.0 smartphone
8. Ensure the check box next to “Turn on Bluetooth” is checked on the Windows Mobile 5.0 smartphone
9. Click “Next” on the Falcon
10. Tap the “Devices” tab on the Windows Mobile 5.0 smartphone
11. Tap “New Partnership” on the Windows Mobile 5.0 smartphone
12. Click “Next” on the Falcon
13. Highlight the device “Palm” in the device list on the Windows Mobile 5.0 smartphone. This is the default name for the Falcon
14. Tap “Next” on the Windows Mobile 5.0 smartphone
15. Click “Next” on the Falcon
16. Ensure the Windows Mobile 5.0 smartphone is configured for number entry by pressing the option key twice
17. Find the six digit Bluetooth security key on Falcon screen
18. Enter the security key on the Windows Mobile 5.0 smartphone
19. Tap “Next” on the Windows Mobile 5.0 smartphone
20. Tap “Finish” on the Windows Mobile 5.0 smartphone
21. The “Pairing successful” screen appears on the Falcon when the Bluetooth pairing has successfully completed
22. Click “Next” to continue
23. After pairing, the Falcon automatically starts downloading e-mail from the Windows Mobile 5.0 smartphone
24. Click “Next” to continue
25. Press the Mail button on the Falcon to launch the Mail application and complete setup

11.4.2.4 Regular pairing with Windows Mobile “Crossbow”

Palm does not officially support beta Operating Systems, however if you wish to experiment with the Falcon Mailsync library and Windows Mobile “Crossbow” setup is very similar to Windows Mobile 5.0 devices.

In order to use a Windows Mobile “Crossbow” device you must have selected a Windows Mobile 5.0 smartphone as your device type in the initial setup. The Bluetooth pairing wizard will display the steps necessary to pair a Windows Mobile 5.0 smartphone with the Falcon. While similar, there are some minor differences when pairing with a Windows Mobile “Crossbow” device.

1. Turn on the Windows Mobile “Crossbow” smartphone
2. Tap “Start” on the Windows Mobile “Crossbow” smartphone
3. Click “Next” on the Falcon
4. Tap “Settings” on the Windows Mobile “Crossbow” smartphone
5. Tap the “Connections” tab on the Windows Mobile “Crossbow” smartphone
6. Click “Next” on the Falcon
7. Tap “Bluetooth” on the Windows Mobile 5.0 smartphone
8. Tap the “Mode” tab on the Windows Mobile “Crossbow” smartphone
9. Ensure the check box next to “Turn on Bluetooth” is checked on the Windows Mobile “Crossbow” smartphone
10. Click “Next” on the Falcon
11. Tap the “Devices” tab on the Windows Mobile “Crossbow” smartphone
12. Tap “Add new device...” on the Windows Mobile “Crossbow” smartphone
13. Click “Next” on the Falcon
14. Highlight the device “Palm” in the device list on the Windows Mobile “Crossbow” smartphone. This is the default name for the Falcon
15. Tap “Next” on the Windows Mobile “Crossbow” smartphone
16. Click “Next” on the Falcon
17. Ensure the Windows Mobile “Crossbow” smartphone is configured for number entry by pressing the option key twice
18. Find the six digit Bluetooth security key on Falcon screen
19. Enter the security key on the Windows Mobile “Crossbow” smartphone
20. Tap “Next” on the Windows Mobile “Crossbow” smartphone
21. Tap “Finish” on the Windows Mobile “Crossbow” smartphone
22. The “Pairing successful” screen appears on the Falcon when the Bluetooth pairing has successfully completed
23. Click “Next” to continue
24. After pairing, the Falcon automatically starts downloading e-mail from the Windows Mobile “Crossbow” smartphone
25. Click “Next” to continue

26. Press the Mail button on the Falcon to launch the Mail application and complete setup

PART III

Developing Applications

This part of the guide provides an overview of the Falcon Software Development Kit (SDK) and how to build applications using the SDK.

CHAPTER 12

Falcon SDK

This chapter provides a general overview of the Falcon Software Development Kit (SDK). The SDK contains the tools necessary to develop native Falcon applications as well as develop and simulate them on i386-based Linux computers. The SDK also includes interfaces and libraries of the released Falcon APIs referenced in this guide, and sample code and instructions.

Complete API documentation is generated directly from Palm® source code. Refer to the separate Falcon API Guide either in compressed HTML format (.chm) or HTML format available from **http://pdn.palm.com**.

12.1 The development process

The normal development process uses the command line tools provided in the Falcon SDK. The two primary development steps are:

- Create an application on the i386 development system and execute it there with the simulator. For information, see **Building Applications on the i386 Development System**
- Migrate the app to work with the ARM processor on the Falcon device. For information, see **Building Applications for ARM**.

12.2 Requirements

12.2.1 Hardware

The Falcon device is based on the arm microprocessor. However, the development environment is intended to run on a Pentium IV-based (or similar) PC. The faster the CPU, the faster the compilation time: 512MB of RAM (or more) is recommended. The build environment works with less memory and slower CPUs, but that will mean increased build time.

12.2.2 Software

- Linux

  Since the underlying Falcon OS is Linux, the SDK runs on an x86 Linux computer. Most of the development work on Falcon is done on a Linux workstation running Ubuntu 6.06.01 (Dapper Drake). The tools and instructions included in this Developer Guide are tuned for this specific Linux distribution.

  If you don't already have a Linux system running on your PC, you should install all the Linux development and build tools. The exception is that the GNOME/GTK+ and KDE components are not needed for Falcon development. To install

the Fusion module (see **Installing the Fusion module**), it is also necessary to have the kernel sources available at the time the SDK is installed.

NOTE It is possible to use other Linux variants for Falcon development, however this developer's guide is targeted for Ubuntu. If you do use a different Linux distribution, you might need to make accommodations between what is detailed here and what works on your system.

- Use your favorite code editor (vi, vim, emacs, eclipse)
- Use our tools: gcc, ld, gdb
- When installing, you must have root access or your account must be listed in the sudoers file with no password required.
- The SDK installer will install the Fusion module necessary for the simulator to function. The installation of the Fusion module will require recompiling your Linux kernel, so sources for your kernel must be available. The fusion-installer.sh script assumes the sources are available in the default location. However, to accommodate alternate installations, the script will pause for input about where the sources may be. If nothing is entered, the script will continue with normal installation.

12.3 Downloading the SDK

The latest SDK is available for download from **http://pdn.palm.com**.

If you do not have access to PDN, please register for an account on the site. Once access to Plugged In's Falcon resources has been granted, return to the website and use the fly-out menu to navigate to "develop -> platform SDKs -> Falcon." The Falcon SDK is available under "Falcon Resources."

12.4 Installing the SDK

To install the Falcon SDK simply untar the SDK file and run *sdk-installer.sh* as a normal user. The installer will automatically configure the Fusion module and install the SDK in the /opt/holly directory.

For detailed installation instructions please refer to the **Falcon Getting Started Guide** which is included in the Falcon SDK.

NOTE The SDK installer will not remove an existing /opt/holly environment. Before upgrading to a new version of the SDK you must manually delete or rename the existing /opt/holly tree.

12.4.1 For further installation help

If further help is necessary, please refer to **http://www.kernelnewbies.org/** or **http://www.directfb.org**.

12.5 How the SDK is organized

In **Installing the SDK**, it was mentioned that the SDK is installed at /opt/holly.

The SDK allows for development of native apps compiled for the arm processor on the Falcon device and also for x86 so the application can be simulated on the development PC.

To provide both options, the SDK installs two complete sets of tools and libraries. The tools are located in /opt/holly/toolroot, while the libraries and includes are in /opt/holly/sysroot.

12.5.1 toolroot

toolroot has the needed tools such as the compilers for the specific target. The /opt/holly/toolroot/arm-linux/ has the arm cross-compile tools, and /opt/holly/toolroot/i386-linux/ has the Linux desktop compilation tools. The reason there's both a target compiler and the desktop build compiler is that both compilers are the same – they use the same libraries and headers. So you should use the i386-linux tools when building and testing code in the Linux environment.

To setup the environment to find the specific tools, use the PATH to point at the location for the tools.

```
export PATH=/opt/holly/toolroot/bin:$PATH
```

The bin subdirectory has the compilers and linkers with a specific prefix name, such as: arm-linux-gcc or i386-linux-ld. In many cases, you could set the default compiler with the CC global variable, as in:

```
$ CC=i386-linux-gcc make mycode
```

	toolroot
arm-linux	arm-linux-gcc
i386-linux	386-linux-gcc

The compilers and tools for the platforms are also part of the individual directories with plain names: gcc, nm and so on. These are in /opt/holly/toolroot/<architecture>/bin/. However, Palm recommends not using these to avoid confusion with existing platform tools under /usr/bin/ and so on. The architecture directory also has a local/bin directory with GNU tools, pkg-config, and similar build tools. Place this into your path in case you are using pkg-config in your build setup.

12.5.2 sysroot

Compilers and tools assume that they find needed header files, libraries, and other configuration tools under sysroot, and not from the system in general. This makes it possible to isolate specific library versions and headers from desktop versions.

It's important to know that the tools are compiled to expect that / (slash) starts at /opt/holly/sysroot/<architecture>/. There are two architecture sets: i386-linux and arm-linux. In these two sets, most of the libraries and headers are the same. An example of an exception is that SDL support to simulate the hardware device is only in the i386-linux architecture sysroot.

- i386-linux libraries built with debug on

- arm-linux libraries built for release

	sysroot
arm-linux	arm-linux/usr/includes
	arm-linux/usr/lib
i386-linux	i386-linux/usr/includes
	i386-linux/usr/lib

If you want to use your own system-wide libraries, you need to install them under sysroot, either in /usr/lib/, /usr/local/lib/, or a similar place. Treat headers similarly, installing them under sysroot's /usr/include/, /usr/local/include/, or a similar place. Or you could just keep the libraries and headers as part of your source project directory.

12.5.3 Graphics and DirectFB

The Falcon platform is running on top of DirectFB. On the target, this is done by the underlying Linux frame buffer driver. You could also run this combination on the Linux development system by using the SDL DirectFB layer. The SDL DirectFB layer runs transparently inside an X11 window.

IMPORTANT This is not the same as the X11GTK+ library that was previously provided by the Falcon SDK and there is no programmer access provided to X11 or GTK. Instead, the Falcon app is using DirectFB as its low-level drawing engine.

12.5.4 pkg-config

This SDK is using pkg-config as the primary system for exporting the correct include and library components for building code relying on headers and libraries in the sysroot section. pkg-config returns the correct include and library flags for the compiler. For example:

```
$ gcc mycode.c `pkg-config -libs -cflags directfb`
```

Not all components have so called .pc files that have the definitions. Look inside the sysroot's usr/lib/pkgconfig directory for all the current .pc files.

You should use the SDK pkg-config tool, not the platform version, as the platform pkg-config file is using the /lib/pkgconfig/ as it's default directory for .pc files, not the sysroot. Make sure your PATH is pointing at the directory where the SDK pkg-config file resides:

For arm-linux

```
export PATH=/opt/holly/toolroot/arm-linux/local/bin: \
/opt/holly/toolroot/bin:$PATH
```

For i386-linux

```
Export PATH=/opt/holly/toolroot/i386-linux/local/bin: \
/opt/holly/toolroot/bin:$PATH
```

NOTE This local/bin directory has many GNU make-related tools ported as well, in case you want to cross-compile open-source related projects. For example, libtool does not work correctly with a sysroot related cross-compiler setup, so the SDK has a specific libtool that handles this case.

12.5.5 Samples

Sample code for Falcon is included in the Samples directory found at the main level of the SDK. They do not have to be built within this location, so the samples can be moved elsewhere. Instructions on how to use the SDK, including building some of the sample code, are in **Building Applications on the i386 Development System**. The ReadMe included with the samples provides current information on the samples and how to build them.

12.5.6 Documentation

Documentation for development on Falcon can be found in several places:

- Foremost is this Falcon Developer Guide, which gives an overview of the entire Falcon development process.
- The Falcon Getting Started Guide details the process of installing the Falcon SDK, pairing the Falcon with a Treo for Mailsync and upgrading the ROM image on the Falcon hardware
- More detailed documentation on individual Falcon functions can be found in the Falcon API Guide. The API Guide is compiled from comments found in the Falcon header files. It represents the very latest and most detailed programming information for Falcon. Complete API documentation is generated directly from Falcon source code. The API Guide is available in either compressed HTML format (.CHM) or HTML format.
- Release Notes and the ReadMe file for the SDK can be found at the top level of the SDK. They provide the most recent information about the SDK. Additionally, the samples download contains its own ReadMe file and Release Notes with up-to-date instructions on what is included in the samples and how to build and use them. These are typically text files that can be read by any development environment editor.

12.6 PDN online support

As noted, the place to find and download the SDK or the samples and documentation is the Palm® PDN website: **http://pdn.palm.com**.

If you have trouble accessing the online Falcon resources, email devsupport@palm.com. If you have questions about Falcon programming or bugs in Falcon, email devsupport@palm.com with Falcon mentioned in the subject line.

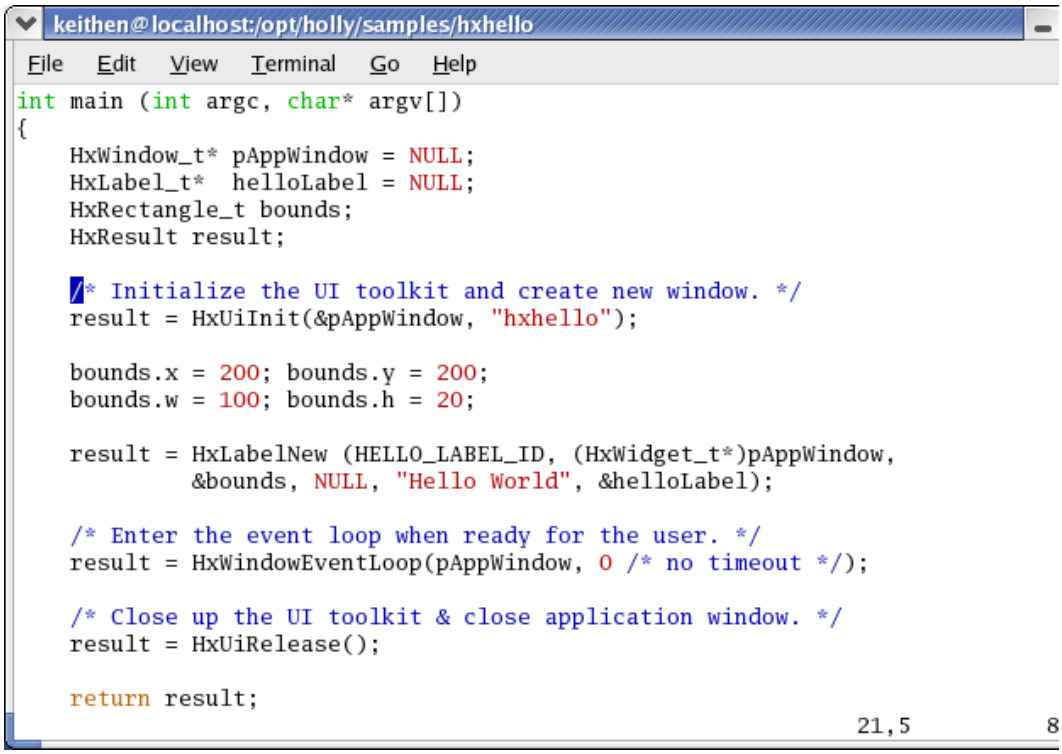
NOTE Palm has created a Falcon-specific forum on the PDN site for discussion and collaboration between developers. This is an excellent resource for exchanging Falcon tips and tricks with fellow Falcon developers.

CHAPTER 13

Building Applications on the i386 Development System

13.1 Creating the app source file

Developers can use any editor they are comfortable with to open and modify source files. Below is an example using vim, the improved vi editor.



To start programming for Falcon, you need to create source files. Falcon applications are written in C. Below is a simple sample of C code for a Falcon application, `HxHello`.

```
/*  File hxhello.c
 *
 *  Falcon SDK sample code.
 *
 *  copyright 2005, All rights reserved
 *  Palm, Inc.
 */
#include <hx/ui/hxui.h>
#include <local.h>
```

```
int main (int argc, char* argv[])
{
    HxWindow_t* pAppWindow = NULL;
    HxLabel_t*  helloLabel = NULL;
    HxRectangle_t bounds;
    HxResult result;


    /* initialize the UI toolbox and create new window */
    result = HxUiInit(&pAppWindow, "hxhello");
     HX_EXIT_IF_ERROR(HX_OK == result, result, error);

    bounds.x = 200;
    bounds.y = 200;
    bounds.w = 100;
    bounds.h = 20;

    result = HxLabelNew (kHxWidgetAutoID, (HxWidget_t*)pAppWindow,
             &bounds, NULL, "Hello World", &helloLabel);
    HX_ASSERT(HX_OK == result);

    /* enter the event loop when ready for the user */
    result = HxWinEventLoop(pAppWindow, 0 /* no timeout */);
    HX_ASSERT(HX_OK == result);

error:

    /* close up the UI toolbox & close application window*/
    result = HxUiRelease();

    return result;
}
```

13.2 Compiling and linking the app

13.2.1 Creating a Makefile

A Makefile provides instructions to the Falcon SDK to compile and link an application. Below is an example of the simple Makefile used to build `HxHello`.

```
# hxhello is a minimal Falcon sample. It can be built

# right in place and run with no other resources.

hxhello: hxhello.c

   $(CC) hxhello.c -o hxhello `pkg-config --libs --cflags hx-1.0 hxui-1.0
directfb`
```

The environment variable `CC` specifies the build operation. Examples of exporting this are in our shell's build session. You can also specify it inline with the `make` command.

Set CC to `arm-linux-gcc` for building native apps for the Falcon device, or `i386-linux-gcc` for running on the simulator in our development PC. Each version of the compiler knows the correct version of the `ld` tool for linking the code libraries together.

`pkg-config` is called to specify build options for the `CC` build command.

`-cflags` specifies which options, such as include paths, should be given to the gcc compiler.

`-libs` specifies which options will be passed on to `ld`.

Makefiles for larger Falcon applications are more elaborate to specify cpu architecture, debug or profiling options, and which components or subcomponents to build.

13.2.2 Setting the path

Before building, make sure you have the PATH pointing at the correct tools in the toolroot.

```
export PATH=/opt/holly/toolroot/i386-linux/local/bin:/opt/holly/toolroot/bin:$PATH
```

13.2.3 Specifying the compiler

In addition to setting the path, you need to tell the Makefile which platform you are compiling for by specifying the compiler to use in the `CC` variable. This can be done by setting it just before the `make` command as shown below:

```
$ CC=i386-linux-gcc make hxhello
```

While Falcon APIs are written in C, it is also possible to use a C++ compiler by using either `CC=arm-linux-g++` or `CC=i386-linux-g++`.

Instead of always typing the compiler option into the command line, you can set CC as an environment variable and pass it to `make`:

```
export CC=i386-linux-gcc
make hxhello
```

13.3 Executing the app from the command line

The arm-linux binaries must be installed on a Falcon device to execute. However, the final i386-linux binaries can be executed directly on the development machine. It is necessary to let the system know where the Falcon libraries are stored so they can be used by the app. To do this, set `LD_LIBRARY_PATH`:

```
export LD_LIBRARY_PATH=/opt/holly/sysroot/i386-linux/usr/lib
hxhello
```

It is also possible to set `LD_LIBRARY_PATH` on the same line that launches the app:

```
LD_LIBRARY_PATH=/opt/holly/sysroot/i386-linux/usr/lib ./hxhello
```

You could write a small bash function that does this automatically each time the app is started. An example is shown in **Post-installation instructions**.

When a command is issued to run an app, such as `hxhello`, in the i386 environment, the app must call the same routines that are in ROM on the Falcon device. In this respect, the simulator is running. The frame buffer being provided by the hardware on the Falcon is being simulated by the Simple Direct Media Layer, SDL, provided by DirectFB. Above that base layer, all the system code is the same for both ROM and simulator.

When running on the PC, some keys that are on Falcon don't exist. Here is a list of PC keyboard keys that provide the Falcon key functions.

Falcon	PC Keyboard
Email	F12
BACK	F10
FORWARD	F11
MENU	Left or right control
APP	Windows

If `hxhello` is launched from the command line, there is no Application Launcher to return to when the app button is pressed. In that case, use control c to kill the app.

13.4 Creating the app.hxi file

Falcon applications can be built and launched manually following the steps above. The app.hxi file resides in the bundle directory, usually alongside the executable. Below is a sample app.hxi file from one of the code examples:

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Comment -->
<hx>
        <application>
                <display_name>
                        myapp
                </display_name>
                <executablePath>
                        /Home/User/Applications/com.mycompany/myapp
                </executablePath>
        </application>
<hx>
```

The display name is what will be displayed by the Application Launcher and may contain spaces. The executable path must be an absolute path to the file.

NOTE All applications should install within the `/Home/User/Applications/` folder on the device. The `executablePath` value should always be `/Home/User/Applications/com.mycompany/myapp`.

13.4.1 Modifying an existing app.hxi file

It is an easy task to modify an existing app.hxi file rather than starting from scratch.

13.4.2 Using the hxify script

You can also use the hxify script contained in the SDK to generate the app.hxi file as part of the build process.

```
hxify [-d displayName] [-f fullPath] [-o outputPath] [-h] appname
```

Where:

-d displayName: displayName may contain spaces if quoted.

-f fullPath: fullPath not including final "/appname"

-o outputPath: Creates app.hxi in outputPath directory

-h: Outputs this help menu

13.5 Expanded Makefile

You can combine all of the steps in the expanded Makefile shown below:

```
#  File Makefile
#
#  copyright 2005, All rights reserved
#  Palm, Inc.
COMPONENT   = hollyview
DOMAIN      = com.palmsdk
SOURCES     = $(COMPONENT).c
INCLUDES    =
PACKAGES    +=  hx-1.0 \
                hxui-1.0 \
                lite \
                directfb \
                glib-2.0
# Expected values:
#   ARCH    = arm | i386
#   DEBREL  = debug | release | profile
ifndef $(ARCH)
   ARCH = i386
endif
ifndef $(CC)
   CC=$(ARCH)-linux-gcc
endif
ifndef $(DEBREL)
    ifeq ($(ARCH),i386)
        DEBREL  = debug
    else
        DEBREL  = release
    endif
endif
ifeq ($(DEBREL),debug)
    CFLAGS  += -g3
else
    ifeq ($(DEBREL),profile)
        CFLAGS  += -p
    endif
endif
SDK_BASE    = /opt/holly
SYS_BASE    = $(SDK_BASE)/sysroot
TOOL_BASE   = $(SDK_BASE)/toolroot
HOLLY_BASE  = $(SYS_BASE)/$(ARCH)-linux
APP_BASE    = $(HOLLY_BASE)/Applications
TARGET_DIR  = $(APP_BASE)/$(DOMAIN)/$(COMPONENT)
TARGET      = $(TARGET_DIR)/$(COMPONENT)
CFLAGS      += `pkg-config --cflags $(PACKAGES)`
LDFLAGS     += `pkg-config --libs $(PACKAGES)`
all: $(TARGET)
$(TARGET): $(SOURCES) $(INCLUDES)
   mkdir -p $(TARGET_DIR)
   $(CC) $(SOURCES) -o $(TARGET) $(CFLAGS) $(LDFLAGS)
```

```
    hxify -d "Holly View" -o $(TARGET_DIR) $(COMPONENT)
clean:
    rm -r $(TARGET_DIR)
```

Simply typing `make` on the command line now results in the application getting built and installed at:

```
/opt/holly/sysroot/i386-linux/Applications/${domain}/${bundle}
```

You could navigate the terminal window to that directory and once again launch the application from the command line. However, below is a better alternative.

13.6 Running the application in the simulator

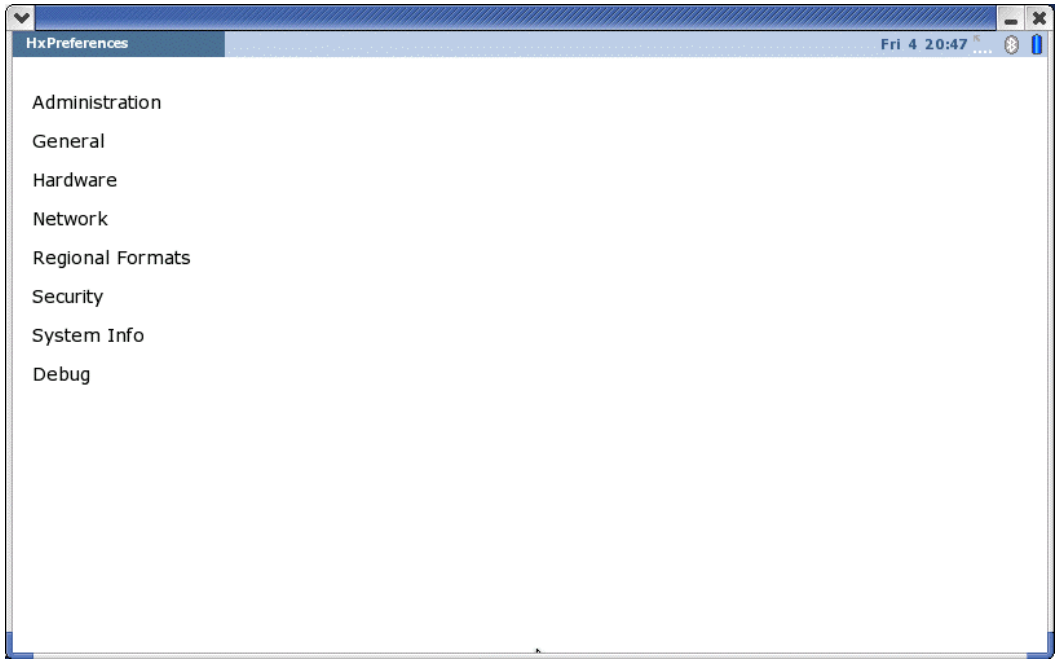
1. Use `LD_LIBRARY_PATH` to ensure that the simulator uses the correct i386 libraries. This may have already been done in a previous step but, if not, do so before running the simulator.

   ```
   export LD_LIBRARY_PATH=/opt/holly/sysroot/i386-linux/usr/lib
   ```

2. Launch the simulator with the command:

   ```
   /opt/holly/systroot/i386-linux/usr/bin/hxtek
   ```

   The simulator then executes the same startup code as is run on the native Falcon, which in turn leads to the `hxpreferences` application shown below:



3. While in the simulator, press the escape key to produce a list of applications the simulator can run. The application you just built should be listed.
4. When applications are running in the simulator, pressing either control key simulates pressing the menu key on the real device.

13.7 Debugging i386 applications

The default build rules from the Makefile create an i386 build. They also default to building i386 as a debug build. For that reason, the Makefile adds the `-g3 cflag`, which tells the compiler to keep the debugger symbols with the build.

At this time, it is easiest to do your debugging in the simulator. Start the debugging session by launching gdb with the name of the program you wish to debug.

```
[keithen@keithen hxhellodrawstring]$ gdb hxhellodrawstring
GNU gdb
...
library "/lib/tls/libthread_db.so.1".

(gdb) b DrawHelloWorld
Breakpoint 1 at 0x80485f6
(gdb) run
Starting program: /opt/holly/samples/hxhellodrawstring/hxhellodrawstring
[Thread debugging using libthread_db enabled]
[New Thread -1218570624 (LWP 3911)]
(*) DirectFB/Config: Parsing config file '/opt/holly/sysroot/i386-linux/etc/directfbrc'.
[New Thread 26192816 (LWP 3914)]
[New Thread 55311280 (LWP 3915)]
DEBUG: HXTEK: PID: 3911 - Initialized
Hxinit Initialized Entered
Error: Error opening directory '/opt/holly/sysroot/i386-linux/Applications': No such
file or directory
(!) [  NO NAME         0.235] ( 3911) *** Assumption [core != NULL] failed *** [../../
../../DirectFB-0.9.21/src/core/core.c:424 in dfb_core_create_surface()]
[Switching to Thread -1218570624 (LWP 3911)]

Breakpoint 1, 0x080485f6 in DrawHelloWorld ()
(gdb) s
(gdb) x pColor
0xbfffad9c:     0x0000aaff
(gdb) p pWidget
$1 = (HxWidget_t *) 0x93a3218
(gdb) p *pWidget
$2 = {box = {parent = 0x0, n_children = 0, children = 0x0,
    type = LITE_TYPE_WINDOW, rect = {x = 0, y = 0, w = 800, h = 454},
    surface = 0x93a9750, user_data = 0x0, is_focused = 0, is_visible = 1,
    handle_keys = 0, OnFocusIn = 0, OnFocusOut = 0, OnEnter = 0, OnLeave = 0,
    OnMotion = 0, OnButtonDown = 0, OnButtonUp = 0, OnKeyDown = 0,
    OnKeyUp = 0, OnWheel = 0, Draw = 0xbb35da <draw_window>, Destroy = 0},
  id = 800, magic = 454, pParent = 0x2, type = 154808232, attr = 154809336,
  borderWidth = 154809584, SetFocus = 0xc, SetEnabled = 0, SetVisible = 0,
  pScrollbarH = 0x0, pScrollbarV = 0x0, pInsertionPt = 0x0,
  inFocusArray = false, hasFocusRing = false, enabled = false}
(gdb) c
Continuing.

<cntrl 'c'>
Program received signal SIGINT, Interrupt.
0x0022b27d in pthread_cond_wait@@GLIBC_2.3.2 () from /lib/tls/libpthread.so.0
(gdb) quit
The program is running.  Exit anyway? (y or n)
```

CHAPTER 14

Building Applications for ARM

14.1 Creating the app source file

This step identical to the material in **Building Applications on the i386 Development System**.

14.2 Compiling and linking the app

14.2.1 Creating a Makefile

A Makefile provides instructions to the Falcon SDK to compile and link an application. Below is an example of the simple Makefile used to build `HxHello`.

```
# hxhello is a minimal Falcon sample. It can be built
# right in place and run with no other resources.
hxhello: hxhello.c
   $(CC) hxhello.c -o hxhello `pkg-config --libs --cflags hx-1.0 hxui-1.0 directfb`
```

The environment variable `CC` specifies the build operation. You can export this in the the shell's build session or specify it inline with the `make` command.

Set CC to `arm-linux-gcc` for building native apps for the Falcon device, or to `i386-linux-gcc` for running on the simulator in our development PC. Each version of the compiler knows the correct version of the `ld` tool for linking the code libraries together.

`pkg-config` is called to specify build options for the `CC` build command.

`-cflags` specifies which options, such as include paths, should be given to the gcc compiler.

`-libs` specifies which options will be passed on to `ld`.

Makefiles for larger Falcon applications will be more elaborate to specify cpu architecture, debug or profiling options, and which components or subcomponents to build.

14.2.2 Setting the path

Before building, make sure you have the PATH pointing at the correct tools in the toolroot.

```
export PATH=/opt/holly/toolroot/arm-linux/local/bin:/opt/holly/toolroot/bin:$PATH
```

14.2.3 Specifying the compiler

Tell the Makefile which platform you are compiling for by specifying the compiler to use in the `CC` variable. You can set this just before the `make` command as shown below:

```
$ CC=arm-linux-gcc make hxhello
```

While Falcon APIs are written in C, it is also possible to use a C++ compiler by using either `CC=arm-linux-g++` or `CC=i386-linux-g++`.

Instead of always typing the compiler option into the command line, you can set CC as an environment variable and pass it to `make`:

```
export CC=arm-linux-gcc
make hxhello
Libraries
```

14.2.4 Setting the library path

The arm-linux binaries must be installed on a Falcon device to execute. However, the final i386-linux binaries can be executed directly on the development machine. It is necessary to let the system know where the Falcon libraries are stored so they can be used the app. To do this, set `LD_LIBRARY_PATH`:

```
export LD_LIBRARY_PATH=/opt/holly/sysroot/i386-linux/usr/lib
hxhello
```

It is also possible to set `LD_LIBRARY_PATH` on the same line that launches the app:

```
LD_LIBRARY_PATH=/opt/holly/sysroot/i386-linux/usr/lib ./hxhello
```

You could write a small bash function that does this automatically each time the app is started. An example is shown in **Post-installation instructions**.

Shared libraries

- Normal approach is to use ldlib to create library.
- How to Create and Load Libraries
  - In same folder
  - In domain folder
- Then libconfig using symbolic links to control versioning.
- Libraries would normally go in …/usr/lib
  - but not good for library cleanup
- Put in <domain> directory
  - Share with symbolic link or LD_LIBRARY_PATH.
  - Better: use linker “-rpath ../Common”.

14.3 Packaging the app

14.3.1 Creating the app.hxi file

Falcon applications can be built and launched manually following the steps above. The app.hxi file resides in the bundle directory, usually alongside the executable. Below is a sample app.hxi file from one of the code examples:

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Comment -->
<hx>
        <application>
                <display_name>
                        myapp
                </display_name>
                <executablePath>
                        /Home/User/Applications/com.mycompany/myapp
                </executablePath>
        </application>
<hx>
```

The display name is what will be displayed by the Application Launcher and may contain spaces. The executable path must be an absolute path to the file.

NOTE All applications should install within the `/Home/User/Applications/` folder on the device. The `executablePath` value should always be `/Home/User/Applications/com.mycompany/myapp`.

14.3.2 Modifying an existing app.hxi file

It is an easy task to modify an existing app.hxi file rather than starting from scratch.

14.3.3 Using the hxify script

You can also use the hxify script contained in the SDK to generate the app.hxi file as part of the build process.

```
hxify [-d displayName] [-f fullPath] [-o outputPath] [-h] appname
```

Where:

-d displayName: displayName may contain spaces if quoted.

-f fullPath: fullPath not including final "/appname"

-o outputPath: Creates app.hxi in outputPath directory

-h: Outputs this help menu

14.3.4 Default icon

Every application must have a default icon which will be displayed in the AppLauncher menu. This can be specified in the app.hxi file.

If a default icon is NOT specified the system will use a generic icon.

14.4 Expanded Makefile

You can combine all of the steps in the expanded Makefile shown below:

```
#  File Makefile
#
#  copyright 2005, All rights reserved
#  Palm, Inc.
COMPONENT   = hollyview
DOMAIN      = com.palmsdk
SOURCES     = $(COMPONENT).c
INCLUDES    =
PACKAGES    +=  hx-1.0 \
                hxui-1.0 \
                lite \
                directfb \
                glib-2.0
# Expected values:
#   ARCH    = arm | i386
#   DEBREL  = debug | release | profile
ifndef $(ARCH)
   ARCH = i386
endif
ifndef $(CC)
   CC=$(ARCH)-linux-gcc
endif
ifndef $(DEBREL)
    ifeq ($(ARCH),i386)
        DEBREL  = debug
    else
        DEBREL  = release
    endif
endif
ifeq ($(DEBREL),debug)
    CFLAGS  += -g3
else
    ifeq ($(DEBREL),profile)
        CFLAGS  += -p
    endif
endif
SDK_BASE    = /opt/holly
SYS_BASE    = $(SDK_BASE)/sysroot
TOOL_BASE   = $(SDK_BASE)/toolroot
HOLLY_BASE  = $(SYS_BASE)/$(ARCH)-linux
APP_BASE    = $(HOLLY_BASE)/Applications
TARGET_DIR  = $(APP_BASE)/$(DOMAIN)/$(COMPONENT)
TARGET      = $(TARGET_DIR)/$(COMPONENT)
CFLAGS      += `pkg-config --cflags $(PACKAGES)`
LDFLAGS     += `pkg-config --libs $(PACKAGES)`
all: $(TARGET)
$(TARGET): $(SOURCES) $(INCLUDES)
   mkdir -p $(TARGET_DIR)
   $(CC) $(SOURCES) -o $(TARGET) $(CFLAGS) $(LDFLAGS)
   hxify -d "Holly View" -o $(TARGET_DIR) $(COMPONENT)
clean:
   rm -r $(TARGET_DIR)
```

14.4.1 Creating the .hxp file

Falcon applications and supporting files are packaged in .hxp files and placed in Falcon Package files. This is how you package an app for distribution to Falcon devices at the end of the development and debugging cycle. *It is also how you place the app on Falcon for testing. [??]*

14.4.1.1 .hxp format specification

.hxp files are Falcon Packages, designed to transport a program bundle to and from the device.

14.4.1.2 .hxp naming conventions

The .hxp naming convention is inherited from the RPM specification:

```
name-version-release.arch.hxp
```

- `name` is the name of the package. This may be derived from the company name and program name, but it's up to the .hxp file producer to determine.
- `version` is another string containing the version number of the package. This is again an unstructured number. It might incorporate information about alpha/beta/release status and a build date.
- `release` is an integral number. The first release should be numbered 1, with any repacks or updates incrementing the number. A substantial change in the program would change the version number and reset this back to 1.
- `arch` is a string from the set "`arm`", "`armdbg`", "`x86rel`", "`x86dbg`"", and "`noarch`". All apps released to users will be "`arm`". The additional options are useful during development and for supplying packaged apps to the Falcon system group for integration into the system ROM or SDK. "`noarch`" is reserved for apps written in a non-compiled scripting language, but is not used. If Palm updates its ARM ABI, a new architecture string will be defined.

14.4.1.3 .hxp contents

An .hxp file is a GZIPd TAR archive. It was produced so that it is rooted in the /apps folder of the Falcon device or SDK tree. All files should have user and group ID 0, stored as a numeric entry or as the word "root". When installed into the SDK, this is changed to ID 500, but on the device, the ID cannot be easily adjusted when unpacking.

The tools that manipulate .hxp files will look for embedded app.hxi files to get information about the packaged app. For security reasons, when unpacking the .hxp file, the system will be chrooted to the /apps folder, disallowing the unpack process from affecting files outside the apps hierarchy.

The tar tool on the device does not support GNU tar extensions, so the `-sparse` option is not available. Since .hxp files must be creatable on the device as well to support app archiving and sending, features of tar files that are not available in the busybox tar program must be avoided.

Packaging app to ship

- Falcon linuX Archive: .hxp
  - Finalize application directory layout

- Application, app.hxi, ReadMe, [Resource dir], [libraries]

– Must install applications with owner and group as root

```
sudo chown -R 0 appdir
sudo chgrp -R 0 appdir
```

– Use .hxp naming conventions

- App name (no spaces in name)
- Version
- Release
- Type: arm, armdbg, x86rel, x86dbg, or noarch.
- Tar it up using GZiP compression

```
tar czf AppName-1.0-1.arm.hxp appdir
```

14.4.1.4 Example

For the demo app `HxField`, the .hxp file name is:

```
HxField-1.0-1.arm.hxp
```

When expanded with tar, tzvf would look like:

```
drwxrwxrwx 0/0      0 2005-04-13 09:22:46 palm.com/HxField/
-r--r--r-- 0/0    350 2005-04-13 09:22:46 palm.com/HxField/app.hxi
-rwxrwxrwx 0/0   7768 2005-04-13 09:22:46 palm.com/HxField/HxField
```

14.5 Installing the ARM app on Falcon

14.5.1 Using the network

- The Default Transfer Setup in the Falcon Group
- Add second Ethernet card
- System Settings Network:
- Activate eth1 Device
- Double click to bring up Ethernet Device
  – Select General Tab
  – Select Statically set IP addresses
- Address: 192.168.1.100
- Subnet Mask: 255.255.255
- Default Gateway Address: (leave blank)
  – OK
  – Close
  – Yes to save changes

NOTE Shows our 2nd Ethernet card as eth1.

- Development PC ifconfig

```
[keithen@keithen include]$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0F:1F:87:5F:C7
          inet addr:148.92.214.93  Bcast:148.92.215.255  Mask:255.255.252.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:818543 errors:0 dropped:0 overruns:0 frame:0
          TX packets:560138 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:345750691 (329.7 Mb)  TX bytes:594326501 (566.7 Mb)
          Base address:0xdcc0 Memory:dfee0000-dff00000

eth1      Link encap:Ethernet  HWaddr 00:12:17:50:01:71
          inet addr:192.168.1.100  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:7732 errors:0 dropped:0 overruns:0 frame:0
          TX packets:8285 errors:702 dropped:0 overruns:0 carrier:940
          collisions:0 txqueuelen:1000
          RX bytes:1212106 (1.1 Mb)  TX bytes:2154178 (2.0 Mb)
          Interrupt:10 Base address:0xe400
```

- Must use Linksys USB200M USB to Ethernet adapter with a second Ethernet card and crossover cable
- Falcon Default ifconfig:

```
~ # ifconfig
eth0      Link encap:Ethernet  HWaddr 00:10:60:84:71:9F
          inet addr:192.168.1.102  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:52 errors:0 dropped:0 overruns:0 frame:0
          TX packets:39 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:100
          RX bytes:4305 (4.2 KiB)  TX bytes:3038 (2.9 KiB)
mount /mnt/nfs
```

14.5.2 Alternative using internet without extra Ethernet card

- Must use Linksys USB200M USB to Ethernet adapter
- and standard Ethernet cable connected to DHCP port.

On Falcon:

- Edit `/etc/fstab` to match ip address of Development PC.
- Must connect via DHCP to internet

```
$ udhcpc -i eth0
```

- Use ifconfig to find Falcon ip address for Development PC to use for telnet into Falcon.

```
$ mount /mnt/nfs
```

14.5.3 Option using the SD/MMC card

Optionally, you can install the ARM app on Falcon using a SD/MMC card. The file system automatically mounts SD/MMC cards:

```
~ # cd /Storage/0
~ # ls
dcim
hxcheckbox
palm
~ # mkdir -p /Applications/hxcheckbox/
~ # cp -r /mnt/mmca1/hxcheckbox/ /Applications/hxcheckbox/
~ # ls /Applications/hxcheckbox/hxcheckbox/
hxcheckbox  app.hxi
```

14.6 Remote debugging ARM apps

Debugging remains a topic under investigation. Below are some debugging notes provided for internal Palm Falcon developers. These notes are here to provide inspiration to experienced Linux programmers who want to charge on. Good luck!

There is data on debugging DirectFB at **directfb.com**. You will find a dfbrc (or similar) file that contains flags that DirectFB will check. You can change some of the variables to enable such actions as providing a stack trace.

Same issue for debugging fusionNotes about DirectFB debugging:

- directfbrc:
- no-trace?
- no-debug?
- quiet?

You can debug on either the development system itself or on ARM.

14.6.0.1 MAKEFILE (the same as it was for i386)

```
#  File Makefile
hxhello: hxhello.c
$(CC) hxhello.c -o hxhello `pkg-config --libs --cflags hx-1.0 hxui-1.0
directfb`
```

- Change commands to run before make

```
export CC=arm-linux-gcc
export PATH=     /opt/holly/toolroot/arm-linux/local/bin; \
             /opt/holly/toolroot/local/bin:$PATH
```

- Make the same pkg-config call to set flags and include paths
  - The export PATH command above will steer pkg-config to proper config files.
  - hx-1.0.pc
  - hxui-1.0.pc
  - directfb.pc

14.6.1 Logging in

The best approach is to log in from another system, using ssh or telnet, and then start the `gdb`, `kdbg` or similar debugging sessions.

There are limitations when debugging on a single system:

- DirectFB takes over all of the memory RAM on the system, so you can't have multiple non-DirectFB and DirectFB windows running at the same time. You could run DFTerm as a terminal in this environment, or use any of the DirectFB ported window environments (XDirectFB, just now broken)...
- Deadlocks could occur in a single-system environment. These are caused if the debugged program is stopped while it's holding a global lock. The only way to recover from this is to kill the process remotely. You could avoid these deadlocks if you run the slave in a different session from the debugger. Start another master, switch back, and then start your program as a slave specifying the second session.

If the program hangs, you could also issue the `SysRq-R` to unraw the keyboard and switch to the debugger. Make sure you run the master with `no-graphics-vt` and `no-vt switching`. Otherwise you can't switch to the debugger until you do `SysRq-K` (that makes the debugger useless).

Debugging an app launched by the Application Launcher will add complications. It is easiest to debug an app if it is launched from the command line.

14.6.2 Multi-App Core

You need to use a multi-app core if you want to use `gdb`. Otherwise you can't see the debugger when your app halts. The magic sysrc key should get you out from any freezes in this case. Note this is not the case with SDL. In this case, you don't need to enable the multi-app core.

14.6.3 Stack Traces

Stack traces are controlled by settings in the etc/directfbrc file.

14.6.4 References

- **http://www.directfb.org/mailinglists/directfb-users/2003/08-2003/msg00109.html**
- **http://www.directfb.org/mailinglists/directfb-users/2003/08-2003/msg00113.html**
- **http://www.directfb.org/mailinglists/directfb-users/2003/08-2003/msg00114.html**
- **http://www.directfb.org/mailinglists/directfb-dev/2004/03-2004/msg00002.html**

CHAPTER 15

Debugging

Ideally, every program would compile and run perfectly on the very first try. Unfortunately the chances of this happening decrease as the complexity of your program increases so debugging becomes a necessity. Debugging a Falcon application has much in common with debugging a PalmOS or Windows Mobile application, however the tools to do so are a bit different.

In this chapter we'll take a look at the tools available for debugging a Falcon application in the simulator as well as connecting to a Falcon device and remotely debugging a native ARM process.

15.1 GDB

GDB is an open source debugger, provided as part of the open source GNU toolset. Because it is a command line debugger, GDB is flexible and portable, working on many systems. However its portability comes at the cost of a GUI.

GDB is an entirely command line driven program. Everything is done via the command line and all output is displayed in text. For a developer used to working in Codewarrior or Visual Studio, debugging with GDB may seem a bit jarring at first but once you've mastered the basic commands GDB quickly becomes second nature.

There are many different versions of GDB and most share the same commands. In order to avoid confusion, this guide focuses on GDB v6.5. Earlier and later versions of the tool should work, but have not been tested.

15.1.1 Key GDB commands

The following commands can be entered at any GDB prompt.

1. r - run: Instruct the program to run as normal until it reaches the next breakpoint.
2. p - print variable: Display the contents of the listed variable on screen.
3. c - continue: Continue running the program.
4. s - step: Steps forward through the program, line-by-line, including all subroutines.
5. n - next: Steps forward through the program, line-by-line, bypassing all subroutines.
6. bt - backtrace: Display the current stack.
7. info threads - information: Display a summary of the current threads.
8. help - help: Display the help screen.
9. q - quit: Quit the debugger.

15.1.2 Start a GDB session on the simulator

To start a GDB debugging session on the Falcon simulator, simply type “gdb” followed by the name of your program and then press <enter>. For example, to debug the hxhello sample program, you would type:

```
[testmachine]$ gdb hxhello
```

This command launches the debugger and prepares the sample program. After initializing, GDB displays some legal text before prompting you with the (gdb) prompt. Here is where you will want to set the first breakpoint. This can be done by typing “b” followed by the name of the function to break at and press <enter>. For example:

```
(gdb) b main
```

This sets a breakpoint at the function main.

You can also set a breakpoint by line number:

```
(gdb) b 95
```

Now that the breakpoint is set, type “run” and press <enter> to start the sample program. For example:

```
(gdb) run
```

Because the simualtor behaves very much like a real device, debugging your applications on the simulator is recommended.

15.1.3 GDB documentation

Like any free software, there is plenty of documentation for GDB. Sorting through all of it can be a time consuming task, so we’ve listed some of the best GDB reference manuals right here.

15.1.3.1 Peter’s GDB tutorial

Written by Peter Jay Salzman, this tutorial is easily the best GDB overview available. Salzman does an excellent job of breaking GDB down into logical bits and then covers each section in-depth.

http://www.dirac.org/linux/gdb/index.php

15.1.3.2 GDB official manual

Perhaps the most detailed source of GDB information on the Internet is the official documentation. Available in both HTML and PDF formats, the official manual tells you everything you need to know about GDB and then some. Although it can be a useful reference tool, the official manual is not the best way to learn the program.

http://sourceware.org/gdb/documentation/

15.1.3.3 GDB reference card

Sometimes you just need a quick reference card with a list of key commands and that's exactly what you'll get at Ben Jasik's web site. Jasik has assembled a GDB reference sheet that is viewable as HTML or available as a ready-to-print PostScript file.

http://www.cs.princeton.edu/~benjasik/gdb/gdbtut.html

15.2 Remote debugging with GDB

Debugging on the simulator is a great first pass, but there is no comparison for debugging directly on the Falcon itself. Because the Falcon does not ship with a version of GDB in its system software, the only way to debug a running application is via remote debugging.

In remote debugging configuration the Falcon is connected to the development PC. GDB runs on the development PC, while the application being debugged runs on the Falcon. Even though it is on a remote machine, GDB maintains control of the program on the Falcon and allows a standard debugging session.

15.2.1 System requirements

In order to setup a remote debugging session the following items are required:

- Development PC with the Falcon SDK installed
- Dedicated Ethernet card (for debugging only) in the development PC
- Crossover Ethernet cable
- Falcon device with current ROM image
- Certified USB-to-Ethernet adapter
- NFS server running on development PC

15.2.2 Recommended USB-to-Ethernet adapters

Currently only the following USB-to-Ethernet adapters have been tested by Palm and are supported on the Falcon. Other adapters may work, but are not supported

- Linksys USB100TX
- TrendNet TU-ET100C

15.3 Configure Falcon for remote debugging

Before the development PC can connect to the Falcon it must be configured for remote debugging. This is a multi-step process, however it is relatively straightforward.

1. Reboot the Falcon.
2. Plug the USB-to-Ethernet adapter into the Falcon's open USB slot.
3. Open a terminal prompt.
4. Type "ifconfig" and press <enter> to bring up the Falcon's Ethernet display. you should see a section that looks like the following:

```
~ # ifconfig
eth0      Link encap:Ethernet  HWaddr 00:10:60:84:71:9F
          inet addr:192.168.1.102  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:52 errors:0 dropped:0 overruns:0 frame:0
          TX packets:39 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:100
          RX bytes:4305 (4.2 KiB)  TX bytes:3038 (2.9 KiB)
```

NOTE The Falcon's pre-configured IP address is 192.168.1.102. This is hard-coded into the system and **CANNOT BE CHANGED**.

5. Configure the dedicated debugging Ethernet card on the development PC so that it has an IP address of 192.168.1.100.
6. Plug one end of the crossover cable into the development PC.
7. Plug the other end of the crossover cable into the USB-to-Ethernet adapter.
8. Open a terminal prompt on the Falcon.
9. Verify the connection by typing "ping 192.168.1.100" and pressing <enter>.
10. The display will show a response from the PC. If you do not see a response restart the process at step 1).

15.3.1 Mount the arm-linux tree of the Falcon SDK via NFS

After you have verified that the development PC and Falcon are connected via Ethernet you need to mount the holly toolchain on the Falcon.

1. Ensure the NFS server is running on the development PC.
2. Open a terminal prompt on the Falcon.
3. Type "mount /mnt/nfs" and press <enter>. on the Falcon.
4. After a moment the Falcon will mount the /opt/holly/sysroot/arm-linux/ directory from the development PC.
5. Verify the mount by listing the contents of the /mnt/nfs directory on the Falcon. It should match the contents of the /opt/holly/sysroot/arm-linux/ directory on the development PC.

15.3.2 Copy the debug application to the Falcon

In order to debug your application, it must first be copied to the Falcon device. This can easily be done via the NFS connection.

Place the application in the following directory on the Falcon:

```
/Applications/com.MyCompanyName/MyApplication
```

NOTE Don't forget to compile your application with debug flags enabled!

15.3.3 Start the gdbserver

In order for GDB to connect to the Falcon and debug your application, gdbserver must be launched on the device.

1. Open a terminal prompt on the Falcon.

2. Type “sume” and press <enter>.
3. At the Password: prompt press <enter>.
1. Type “cd /Applications/com.MyCompanyName/MyApplication” and press <enter>.
2. Type “gdbserver port:2345 *MyApp*” where “MyApp” is the name of the application you want to debug. Press <enter>.
3. The terminal should display text similar to the following:

```
Process MyApp created; pid = 12345

Listening on port 2345
```

15.3.4 Start GDB on the development PC

1. Open a terminal prompt on the development PC.
2. Type “cd /opt/holly/toolroot/bin/arm-linux/” and press <enter>.
3. Type “gdb *MyApp*” where “MyApp” is the name of the application you want to debug. Press <enter>.
4. Set a breakpoint in main. At the (gdb) prompt type “b main” and press <enter>
5. Type “r” and press <enter> to start remote execution of the program.
6. Wait for GDB to stop at the breakpoint.
7. Type “set solib-absolute-prefix /opt/holly/sysroot/arm-linux/usr/lib” and press <enter>.
8. The terminal will display text similar to the following as it resolves the various symbols:

```
Reading symbols from /lib/libc.so.6...done.

Loaded symbols for /lib/libc.so.6

Error while reading shared library symbols:

/usr/lib/libfusionsound-0.9.so.25: No such file or directory.

Reading symbols from /lib/libm.so.6...done.

Loaded symbols for /lib/libm.so.6

Error while reading shared library symbols:

/usr/lib/libssl.so.0.9.7: No such file or directory.

Error while reading shared library symbols:

/usr/lib/libcrypto.so.0.9.7: No such file or directory.

Reading symbols from /usr/lib/libexpat.so.0...done.

Loaded symbols for /usr/lib/libexpat.so.0

Reading symbols from /lib/libcrypt.so.1...done.

Loaded symbols for /lib/libcrypt.so.1

Reading symbols from /usr/lib/libfreetype.so.6...done.

Loaded symbols for /usr/lib/libfreetype.so.6

Reading symbols from /lib/ld-linux.so.2...done.

Loaded symbols for /lib/ld-linux.so.2
```

15.3.5 Continue debugging

You are now running GDB on the development PC while executing the native application on the Falcon device. Futher breakpoints can be set and full debugging functionality is available.

NOTE If GDB is trapped by a signal handler simply ignore the event and enter “c” at the (gdb) prompt to continue the debugging session.

15.4 Debugging DirectFB

Debugging DirectFB differs slightly from debugging a standard Falcon application as there are additional flags that are specific to DirectFB and can be set as breakpoints. These flags are specified in the *directfbrc* file which is located in */etc/directfbrc*.

Currently, the following DirectFB flags are supported on the Falcon:

- session
- bg-color
- suspend-color
- mode
- vt-switch
- vt-switching
- wmmx
- mmx-opts
- bg-image
- quiet

More information about the directfbrc configuration options can be found at:

http://directfb.org/docs/directfbrc.5.html

15.5 Debugging tips

- Always launch applications via gdb and gdbserver on the command line. Trying to debug an application launched from the Application Launcher will add complications.
- Don’t forget to compile your application with the debug flags enabled.

CHAPTER 16

Tutorial

16.1 From hxhello.c

> **NOTE** `HxUiInit` creates app window and initialized graphics. `HxWindowEventLoop` handles all events automatically.

```
int main (int argc, char* argv[])
{

    /* Initialize the UI toolbox and create new window. */
    result = HxUiInit(&pAppWindow, "hxhello");

    bounds.x = 200; bounds.y = 200;
    bounds.w = 100; bounds.h = 20;

    result = HxLabelNew (HELLO_LABEL_ID, (HxWidget_t*)pAppWindow,
             &bounds, NULL, "Hello World", &helloLabel);

    /* Enter the event loop when ready for the user. */
    result = HxWindowEventLoop(pAppWindow, 0 /* no timeout */);

    /* Close up the UI toolbox & close application window. */
    result = HxUiRelease();

    return result;
}
```

16.2 Events

> **NOTE** `HxUiInit` creates app window and initialized graphic. `HxWindowEventLoop` handles all events automatically.

```
int main (int argc, char* argv[])
{

    /* Initialize the UI toolbox and create new window. */
    result = HxUiInit(&pAppWindow, "hxhello");

    bounds.x = 200; bounds.y = 200;
    bounds.w = 100; bounds.h = 20;

    result = HxLabelNew (HELLO_LABEL_ID, (HxWidget_t*)pAppWindow,
             &bounds, NULL, "Hello World", &helloLabel);
```

```
    /* Enter the event loop when ready for the user. */
    result = HxWindowEventLoop(pAppWindow, 0 /* no timeout */);

    /* Close up the UI toolbox & close application window. */
    result = HxUiRelease();

    return result;
}
```

16.3 From hxcommandbutton.c

NOTE Need listener to detect button presses can call its callback function.

```
/* Create the message listener. */
    result = HxNewMessageListener(&spListener);
    /* Create the command buttons. */
    result = HxCmdButtonNew(kCIRCLE_CBID, (HxWidget_t*)pAppWindow,
                "Circle", 100, 100, &circleCmdButton);
    /* Listen for all command button messages. */
    result = HxListenToMessage(spListener, kHxActionMessageClass,
                kHxControlButtonChanged, CmdButtonCallback, NULL);

static HxResult CmdButtonCallback(const HxMessage_t* pMessage, void* pData)
{
    result = HxGetMessageSenderID (pMessage, &commandButtonID);
    switch (commandButtonID)
    {
        case kCIRCLE_CBID:
            printf("Circle Command button was called...\n");
            break;
    }
    return result;
}
```

16.4 From hxquit.c

NOTE Drop out of event loop to see if time to quit. Quit set by command button in this example.

```
result = HxNewMessageListener(&spListener);
    result = HxCmdButtonNew(kQUIT_CBID, (HxWidget_t*)pAppWindow,
                "Quit", 700, 300, &quitCmdButton);
    result = HxListenToMessage(spListener, kHxActionMessageClass,
                kHxControlButtonChanged, CmdButtonCallback, NULL);

    while (bRun)
    {
        result = HxWindowEventLoop(pAppWindow, 100);
    }

static HxResult CmdButtonCallback(const HxMessage_t* pMessage, void* pData)
{
    switch (commandButtonID)
```

```
    {
        case kQUIT_CBID:
            bRun = false;
            break;
    }
```

16.5 Sending messages

- `HxSendMessage`
  - Synchronous
  - Can't be used between threads
    - Data may be out of context between broadcaster and listener.
- `HxPostMessage`
  - Asynchronous

16.6 Building with resources

- Rather than individual calls to `HxWhateverNew` use resources.
- Create xml resource descriptions

```
<command_button id=1000 is_default="yes">
   <pos x="100" y="100" />
   <text>OK</text>
   </command_button>
```

- Add to Resources subdirectory

`/Applications/com.domain/AppName/Resources`

- At runtime, create new windows, dialogs, or menus from xml.

`HxResFileNewAppWindow, HxResFileNewDialog, HxResFileNewMenu`

- Can edit xml text files even after program compiled
  - Needed for localization
- Just added to Falcon OS.

16.7 From hxdrawstring.c

Falcon includes a full suite of graphics primitives that can be drawn.

```
HxFontDefinition_t helloFont;
    HxColor_t maroonColor = {0xff, 0xaa, 0x00, 0x00};

    result = HxGfxSetColor((HxWidget_t*)pAppWindow, &maroonColor);

    /* We have 3 choices for font family:
     *  kHxFontStandard Default system font.
     *  kHxFontSystem Built in font specifed by pFontName
     *                  (use HxFontGetNames to discover).
```

```
     *  kHxFontCustom   Provide your own font pointed to by
     *                  pFontName.
     */
    helloFont.family = kHxFontStandard;
    helloFont.size = 20;
    helloFont.weight = kHxFontPlain;
    helloFont.attr = kHxFontAttrNone;
    helloFont.pFontName = NULL;
    result = HxGfxDrawString ((HxWidget_t*)pAppWindow,
       &helloFont, "Hey there World!", 200, 200);
```

16.8 From hxdrawstring.c (also)

NOTE Replace the app window's draw callback with our custom one. This prevents the default drawing routines from blasting our graphics primitives.

```
result = HxWidgetSetCallback((HxWidget_t*)pAppWindow,
                kHxWidgetDraw, DrawAppWindow);
    result = HxGfxDrawString ((HxWidget_t*)pAppWindow,
       &helloFont, "Hey there World!", 200, 200);
    result = HxWidgetUpdate((HxWidget_t*)pAppWindow, NULL);
    result = HxWindowEventLoop(pAppWindow, 0 /* no timeout */);


static HxResult DrawAppWindow(HxWindow_t* pWindow,
                    const HxRegion_t* pRegion, bool clear)
{
    result = HxWindowLockSurface(NULL, (void**)&pDst);

    /* Transfer display memory to the hardware. */
    result = HxGfxForceUpdate((HxWidget_t*)pWindow, NULL);

    result = HxWindowUnlockSurface(pWindow);

    return result;
}
```

16.9 Memory Management

- Standard Unix
- Use malloc and free
  - No garbage collection
  - No reference counting
  - No special app partitions
  - If process dies, all memory cleaned up

16.10 Interrupts

- Standard Linux
  - For kernel see

- Use malloc and free
  - No garbage collection
  - No reference counting
  - No special app partitions
  - If process dies, all memory cleaned up.

16.11 From hxmenu.c

```
static HxMenuItem_t hxmenu_menu[] = {
    {NULL, "About", 0x00, kHxMenuAction, HXMENU_ABOUT, NULL}
};

static HxMenuItem_t file_menu[] = {
    {NULL, "Open", 0x00, kHxMenuAction, HXMENU_FILE_OPEN, NULL},
    {NULL, "Close", 0x00, kHxMenuAction, HXMENU_FILE_CLOSE, NULL},
    {NULL, NULL, 0x00, kHxMenuSeparator, 0, NULL},
    {NULL, "Exit", 0x00, kHxMenuAction, HXMENU_FILE_EXIT, NULL}
};
static HxResult MenuCallback(const HxMessage_t* pMessage, void* data);

 /* create 3 dropdown menus */
    result = HxMenuNew (HXMENU_HXMENU, hxmenu_menu,
                    sizeof(hxmenu_menu) / sizeof (HxMenuItem_t), &hxmenuMenu);
    HxTestPassIfTrue (HX_OK == result, "HxMenuNew");
    result = HxMenuNew (HXMENU_FILEMENU, file_menu,
                        sizeof(file_menu) / sizeof (HxMenuItem_t), &fileMenu);

 /* we need to initialize submenu pointers before creating root menu */
    root_menu[0].pData = hxmenuMenu;
    hxmenuMenu = NULL;
    root_menu[1].pData = fileMenu;
    fileMenu = NULL;
    root_menu[2].pData = editMenu;
    editMenu = NULL;

    /* creating root menu */
    result = HxRootMenuNew (HXMENU_ROOTMENU, root_menu,
                            sizeof(root_menu) / sizeof (HxMenuItem_t), &rootMenu);
    HxTestPassIfTrue (HX_OK == result, "HxRootMenuNew");

    /* install menu listener */
    result = HxNewMessageListener(&listener);
    HX_ASSERT(HX_OK == result);

    result = HxListenToMessage(listener, kHxActionMessageClass,
                                        kHxMenuSelected, MenuCallback, NULL);
if (rootMenu)
    {
        HxMenuDelete (rootMenu);
        HxTestPassIfTrue (HX_OK == result, "HxMenuDelete");
    }

    if (hxmenuMenu)
    {
```

```
        result = HxMenuDelete (hxmenuMenu);
    }

static HxResult
MenuCallback(const HxMessage_t* pMessage, void* data)
{
    HxResult result;
    int32_t  menuid;
    int32_t  itemid;
    int32_t  size;

    result = HxGetMessageSenderID (pMessage, &menuid);
    if (HX_OK != result)
    {
        return result;
    }

    result = HxGetMessageData (pMessage, (void**)&itemid, &size);
    if (HX_OK != result)
    {
        return result;
    }
    printf ("Menu with widget id (%i) had an item (%i) selected.\n", menuid,
                itemid);

    if (itemid == HXMENU_FILE_EXIT)
    {
        printf ("Exit\n");
    }

    return HX_OK;
}
#define HXCOMMANDBUTTON_DIALOG 0x66
#define HXCOMMANDBUTTON_COLOR  0x67
#define HXCOMMANDBUTTON_OPEN   0x68
#define HXCOMMANDBUTTON_SAVE   0x69
#define HXCOMMANDBUTTON_SAVEAS 0x6a
#define HXCOMMANDBUTTON_OK     0x6b
#define HXCOMMANDBUTTON_CANCEL 0x6c

static HxMessageListener_t* dialogListener = NULL;
static HxWidgetDrawFnPtr sSavedCallback = NULL;

static HxResult ButtonCallback(const HxMessage_t* pMessage, void* pData);
static HxResult GenericDialogCallback(const HxMessage_t* pMessage, void* pData);
static HxResult ColorDialogCallback(const HxMessage_t* pMessage, void* pData);
static HxResult FileDialogCallback(const HxMessage_t* pMessage, void* pData);

 result = HxWidgetSetCallback((HxWidget_t*)pAppWindow, kHxWidgetDraw,
                              (HxWidgetDrawFnPtr)Draw, (void**)&sSavedCallback);
    HX_EXIT_IF_ERROR(HX_OK == result, result, error);

    result = HxCmdButtonNew(HXCOMMANDBUTTON_DIALOG, (HxWidget_t*)pAppWindow,
                              "Show Dialog", (800 - 70) / 2 , (454 - 22) / 2,
                              &dialog);
    result = HxCmdButtonNew(HXCOMMANDBUTTON_COLOR, (HxWidget_t*)pAppWindow,
                              "Color", (800 - 70) / 2 + 90, (454 - 22) / 2,
```

```
                        &color);

result = HxCmdButtonNew(HXCOMMANDBUTTON_OPEN, (HxWidget_t*)pAppWindow,
                        "Open", (800 - 70) / 2 , (454 - 22) / 2 + 30,
                        &open);
result = HxCmdButtonNew(HXCOMMANDBUTTON_SAVE, (HxWidget_t*)pAppWindow,
                        "Save", (800 - 70) / 2 + 90, (454 - 22) / 2 + 30,
                        &save);
result = HxCmdButtonNew(HXCOMMANDBUTTON_SAVEAS, (HxWidget_t*)pAppWindow,
                        "Save As", (800 - 70) / 2 + 180, (454 - 22) / 2 + 30 ,
                        &saveas);

result = HxNewMessageListener(&dialogListener);
if (HX_OK != result)
```

PART IV

Core Systems

This part of the guide details the Core Systems that make up the Falcon device.

CHAPTER 17

Graphics and UI

This chapter provides an overview of the Falcon's Graphics and UI features. It touches on the core layers of DirectFB and Lite, as well as covering the custom HxUI system that was created specifically for the Falcon. This chapter also provides information on localizing your application for different languages and locales.

17.1 DirectFB

The

17.2 Lite

17.3 HxUI

17.4 Localization

CHAPTER 18

HxUI XML Format

This chapter provides an overview of the HxUI XML format.

18.1 HxUI XML overview

Designing the UI of an application can be just as time consuming as developing the core technology that powers the application in the first place. The Foleo's HxUI system simplifies this process by specifiying a custom XML form with which a developer can easily define the look and feel of their application.

Using HxUI over direct calls to LiTE or DirectFB is highly recommended as it ensures a consistent overall style between native Foleo applications.

18.2 Top Level Element (hxui)

All HxUI files have a top-level "hxui" element that contains all of the UI descriptions and other resources. This has a single required attribute, "version" that is specified as "1.0" for the initial release. It can also have an optional "default_locale" attribute which sets the default locale for the entire document. This has a default value of "en_US".

A "start_id" attribute is used to set the starting number for ID generation. This defaults to 1000. New IDs are assigned by incrementing this number by one after every new ID string.

NOTE No conflict checking is done to prevent IDs from this list from interfering with other IDs in the system.

The "display" attribute specifies the screen width for the system. Currently two values are recognized "800" and "1024". The default value is "1024" which reflects the native resolution of current Foleo systems: 1024 x 600. The value “800” is maintained for compatibility reasons but should normally never be used.

18.2.1 hxui Element Attributes

- version, required to be "1.0"
- default_locale, string, default: "en_US"
- start_id, integer, default: 1000
- display, string, default: "1024"

18.3 Meta Elements

Meta elements are used to define custom items that are used elsewhere in the XML file. There are two types of meta elements: constant and identifier.

18.3.1 Constant Attributes

- name, string for constant name
- value, decimal or hex value

18.3.2 Identifier Attributes

- name, string for identifier name
- value, decimal or hex value

18.4 UI Elements

UI elements make up the core of the HxUI XML file. This is where you will specifically define the placement of every UI item in your application.

18.4.1 Common UI Attributes

These attributes can be used across all the standard UI elements.

18.4.1.1 enabled

This attribute defines the state of the control. It defaults to a “yes” value, which means a user can interact with it. A “no” value will allow the control to be displayed, however a user will not be able to interact with the control.

NOTE A control that is not enabled may be drawn differently than an enabled control.

18.4.1.2 id

This attribute is either an integer number, a C-style identifier, or the text "$AUTO".

When processing an authoring file into a finalized file, both $AUTO and C identifiers will be transformed into the next automatically-assigned number. C identifiers will also be added to a list of identifiers and values to output into a header file for use in software or that can be used for at-runtime lookup by an API call.

While ID numbers are internally 32-bit values, the top 16-bits of an ID are reserved for system use, so only values in the range 0-65535 are permitted for use by applications.

18.4.1.3 visible

This attribute defines whether or not the control is visible. It defaults to “yes.” Setting this value to “no” hides the control from the user.

18.4.2 Common UI Attributes (DEPRECATED)

Attributes listed here were supported in earlier versions of the Foleo SDK, but have been officially deprecated and should no longer be used. The attribute definitions are included here for historical accuracy.

IMPORTANT DO NOT use the following attributes when creating new HxUI XML files.

18.4.2.1 locale

This is an optional attribute used to identify the locale of the UI element. It inherits sub-elements, so the most common case is to set this on the top-level hxui element. However, it can also be used to mark windows, controls within a window, or even individual text strings as being for a specific locale. For example, a command button could have two text elements, one marked as "en_US" and another as "es_ES".

Two values are recognized by the locale attribute: "en_US" (US English) and "es_ES" (Spanish). The default is "en_US". This attribute corresponds with the LANG environment variable.

Items with non-matching locales will not be processed for form creation, however they will be parsed for finding IDs. This ensures that ID strings used in multiple locales retain the same value.

18.4.3 Common Elements

Much like the common attributes, common elements can appear in multiple controls throughout the HxUI XML file.

18.4.3.1 pos

This is the position element and is used to specify coordinates for a control. The element has two attributes, “x” and “y” using the following format:

```
[('$CENTER' | '$RIGHT' | '$BOTTOM') '@']
  (('$PREV' | '$PARENT' | '$APPBAR' | id) ':'
   ('TOP' | 'BOTTOM' | 'LEFT' | 'RIGHT' | 'CENTER') [('+' | '-') n]) | n
```

NOTE $PREV refers to the previous control defined with this parent, $PARENT refers to the bounding box of the parent container, and $APPBAR refers to region of the application bar between the title area and system bar (this is mainly used when loading a <container> element into the window returned by HxApplicationBarGetWindow).

This format allows an application to specify a coordinate in reference to one of the locations of another control rather than an absolute position on screen. You can use an implicit previous control supported using "$PREV", or an explicit control supported using it's ID value. You can also refer to the parent's container's dimensions using "$PARENT" (although $PARENT:TOP and $PARENT:LEFT would always be 0). There are no $LEFT@ or $TOP@ modifiers, as those are assumed if no modifier is provided.

The following code can be used to position a control right-aligned with the previous control but five pixels down from the previous control's lower edge:

```
<pos x="$RIGHT@$PREV:RIGHT" y="$PREV:BOTTOM+5" />
```

The same control can be given an absolute position just as easily. To center the control at coordinates (400, 240) you would use the following code:

```
<pos x="$CENTER@400" y="$CENTER@240" />
```

Another example involves placing a button inside a container. To place a button 10 pixels away from the lower right hand corner of a container you would use the following:

```
<pos x="$RIGHT@$PARENT:RIGHT-10" y="$BOTTOM@$PARENT:BOTTOM-10" />
```

If you wanted to place the same button in the center of the container, edit the code snippet to read as follows:

```
<pos x="$RIGHT@$PARENT:RIGHT-10" y="$BOTTOM@$PARENT:BOTTOM-10" />
```

18.4.3.2 size

Whereas the pos element used two coordinate attributes to define to location of a UI item, the size element uses two coordinate attributes to defines the relative size of a UI item.

The two attributes used for the size element are "w" and "h," which are shorthand for width and height respectively. The size element is defined by the following format:

```
'$PREV' | '$PARENT' | '$APPBAR' | id} ':'
   ('WIDTH' | 'HEIGHT' [('+' | '-') n ]) | n [ 'em' ]
```

NOTE $PREV refers to the previous control defined with this parent, $PARENT refers to the bounding box of the parent container, and $APPBAR refers to region of the application bar between the title area and system bar (this is mainly used when loading a <container> element into the window returned by HxApplicationBarGetWindow).

The size attribute format allows you to specify a size in absolute terms or relative to another control. You can use an implicit $PREV reference or an explicit "by ID" method.

The "em" modifier on an absolute size number modifies the height and width calculation to be multiples of the font height (also known as em). Most fonts are about twice as high as they are wide, so if you wanted to specify a field that is four lines high and approximately 80 characters wide, you would use the following code:

```
<size w="40em" h="4em" />
```

To use the same width and heith as the previous label, use the following code:

```
<size w="$PREV:WIDTH" h="$PREV:HEIGHT" />
```

To create a containter is that 10 pixels smaller than its parent container, you would use the following code:

```
<size w="$PARENT:WIDTH-10" h="$PARENT:HEIGHT=10" />
```

18.4.3.3 font

The font element has the following attributes:

- family - "standard", "monospaced", "serif", "proportional", "sans_serif"; default: "standard"
- point_size - integer, default: system default value (currently 10 pt)
- weight - one of "plain", "bold", "italic", or "bold_italic", default: plain
- name - UTF-8 string. Default is empty.
- is_kerned - "yes" or "no", default: yes
- is_hinted - "yes" or "no", default: yes
- is_antialiased - "yes" or "no", default: yes

The "name" attribute overrides the "family" attribute if it is used, changing it to system or custom depending on if the string has a filename extension and path information.

The following example will create14 point, monospaced, bold text:

```
<font weight="bold" point_size="14" family="monospaced" />
```

18.4.3.4 color

The color element has the following attributes:

- value - a color value using HTML format, hex values in the form "#rrggbb" or "#rgb"
- alpha - optional alpha channel value, integer from 0 to 255 where 255 means no transparency
- r - red, integer from 0 to 255
- g - green, integer from 0 to 255
- b - blue, integer from 0 to 255

NOTE Either value or the r/g/b triplet must be specified, but if both are supplied the value will be used over the triplet measurement.

18.4.3.5 background_color

The background_color element has the following attributes:

- value - a color value using HTML format, hex values in the form "#rrggbb" or "#rgb"
- alpha - optional alpha channel value, integer from 0 to 255 where 255 means no transparency

- r - red, integer from 0 to 255
- g - green, integer from 0 to 255
- b - blue, integer from 0 to 255

NOTE Either value or the r/g/b triplet must be specified, but if both are supplied the value will be used over the triplet measurement.

18.4.3.6 text

The text element is defined at a UTF-8 string. Note that whitespace IS registered in the string, so writing

```
<text>My text</text>
```

is different than writing

```
<text> My text </text>
```

as the latter has spaces both before and after the “My text” example text.

18.4.3.7 image

The image element has a single “file” attribute which stores a filename for the image. The filename location is relative to the location of the HxUI XML file that’s being parsed so if the image files are located in the same folder as the HxUI XML file, no path is needed. You can simply state the filename.

For example:

```
<image file="small_dot.png" />
```

18.4.3.8 system_image

The system_image element is used to display a standard system image based on a requested type. This allows all applications to use a uniform set of images for standard functions across the device.

The single image attribute can be set to “alert,” “info,” “question,” or “system.”

For example:

```
<system_image image="question" />
```

18.4.4 Windows

The order of the controls in a window definition determines the focus order when a user is interacting with an application.

18.4.4.1 application_window

Attributes:

- id

Elements:

- all controls

The application_windows element is a top-level only element that can hold controls. It has no properties -- those are defined by the system at application start time.

NOTE The “id” attribute is not used to create the window. It is only provided to allow multiple app window definitions in one HxUI XML file.

NOTE Calling `HxResFileLoadApplicationWindow` will load items into an existing application window (previously setup by `HxUIInit`), rather than create a new window.

18.4.4.2 window

Attributes:

- id
- level - "bottom,” "middle,” or "top"

Elements:

- pos
- size
- all controls

This is a top-level window. See `HxWindowNew` API for more information.

18.4.4.3 dialog

Attributes:

- id
- escape - This value should be set to the id of a command button that responds to the Backspace key, closing the current dialog with no change. Typically, this would be the "Cancel" command button. If no escape button is defined, then Backspace is ignored by the dialog's command buttons.

IMPORTANT While it is possible to set a null value for the escape attribute it is NOT recommended. Dialog boxes should always have an escape value.

Elements:

- pos - This is an optional value. If it is not provided, the element defaults to <centered>.
- centered - This element can be used instead of pos to indicate that the dialog should appear centered on screen. Only dialog and alert support this element type.
- size - This is an optional value. If it is not provided, the element defaults to using the default system dialog size.
- text - A UTF-8 string which is used as a title.
- all controls

The dialog box is a modal window with a custom title bar. See the `HxDialogNew` API call for more information.

18.4.4.4 alert

Attributes:

- id
- style - "ok", "cancel", "ok_cancel", "yes_no", or "custom"
- initial_focus - This is the numeric value of the command button that should have initial focus. By default this value is set to 0 (zero) which gives the leftmost button initial focus.

Elements:

- pos - This element is only used for custom-style alerts.
- centered - This element can be used instead of pos to indicate that the dialog should appear centered on screen. Only dialog and custom-style alert support this element type.
- size - only used for custom-style alerts
- title - A UTF-8 string.
- text - A UTF-8 string, used as the body of the alert. This can be a printf-style format string if the alert is shown using `HxResFileAlertFormat`.
- image - This is the image displayed in the top left corner of the alert.
- system_image - A system-standard icon shown in the top left corner of the alert.
- alert_button - This element is only used for custom-style alerts.

18.4.4.5 container

Attributes:

- id

Elements:

- all controls

When used at the top-level, container acts like application_window, a holder of controls. It has no properties, as those are defined by the pre-existing container widget or window that is loaded from this container. The id attribute is only used to identify the element in the HxUI XML file.

18.4.5 Controls

All controls have the “can_focus,” “id,” “enabled,” and “visible” attributes unless otherwise stated. As a result, these attributes are not listed individually on the following control definitions.

18.4.5.1 command_button

Elements:

- pos

- size - Optional. This is only used to override the width value. The height value is ignored.
- text - UTF-8 string.
- font - Optional.

For example:

```
<command_button id=1000>
  <pos x="100" y="100" />
  <size w="150" h="0" />
  <text>OK</text>
</command_button>
```

18.4.5.2 checkbox

Elements:

- pos
- text - UTF-8 string

Attributes:

- is_checked - "yes" or "no", default "no"

There is a mixed state, but this is a read-only state that represents a checkbox group with mixed states, it cannot be set at design time.

18.4.5.3 columnheader

Elements:

- pos
- size
- column

Attributes:

- sortable - yes or no, default: yes

For example:

```
<columnheader id="first" sortable="yes">
        <pos  x="10" y="10" />
        <size w="360" h="16" />
        <column width="100">
           <text>Column 1</text>
        </column>
        <column width="100">
           <text>Column 2</text>
        </column>
        <column width="kHxColumnHeaderUseRemainingWidth">
           <text>Column 1</text>
        </column>
     </columnheader>
```

18.4.5.4 column

Elements:

- text - UTF-8 string

Attributes:

- width
- offset

18.4.5.5 combobox

Elements:

- pos
- size - only width is used
- listitem - multiple

Attributes:

- max_count - Default value is -1. This will show as many items as possible on screen.

Any tool that processes these files should ensure no more than one listitem is selected.

18.4.5.6 container

Elements:

- pos
- size
- background_color - This is the background color of the container. The default value is “transparent.”
- all controls

Attributes:

- has_frame - "yes" or "no", default: no
- has_horizontal_resizer - "yes" or "no", default: no
- has_vertical_resizer - "yes" or "no", default: no
- focusable - "yes" or "no", default: no
- has_focus_ring - "yes" or "no", default: no
- keep_events - "yes" or "no", default: no
- has_rounded_bottom - "yes" or "no", default: no (This only takes effect if has_frame is "yes.")

A container can hold other controls within an application window or dialog box.

18.4.5.7 dropdownbox

Elements:

- pos
- size - only width is used

- listitem - multiple

Attributes:

- max_count - Default value is -1. This will show as many items as possible on screen.

Any tool that processes these files should ensure no more than one listitem is selected.

18.4.5.8 field

Elements:

- pos
- size
- text - UTF-8 string. This attribute defaults to empty. Normally you will want to leave it as such so a user can enter data in the field, however if you want to pre-fill fields this is where the data should be loaded.
- font, default: system font, standard size
- color - default: "#000000" (black)
- background_color - default: "#FFFFFF" (white)

Attributes:

- is_editable - "yes" or "no", default: yes
- is_multiline - "yes" or "no", default: no
- is_scrollable - "yes" or "no", default: no
- is_password - "yes" or "no", default: no
- is_numeric - "yes" or "no", default: no
- focusable - "yes" or "no", default: yes for editable fields, no for non-editable fields
- alignment - "left", "right", or "center", default: left
- has_frame - "yes" or "no", default: yes
- max_glyph_count - integer, default: 0 (unlimited glyphs)
- buffer_size - integer, default: 0 (unlimited buffer size)
- filter - "none", "integer", "float", "non-negative-integer", "non-negative-float", "hex", "printable-ascii", or "no-spaces", default: none

18.4.5.9 file_list

Elements:

- pos
- size

Attributes:

- allow_multiple_selection - "yes" or "no", default: no

18.4.5.10 gadget

Elements:

- pos
- size
- text_object (multiple)
  - pos
  - text - UTF-8 string
  - font, default: system font, standard size
- graphic_object (multiple)
  - pos
  - image
  - system_image
- button_object (multiple)
  - (see below for content)

Attributes:

- focusable - "yes" or "no", default: "no"
- type - "toggle" or "default", default: "default"
- has_frame - "yes" or "no", default: "no"
- has_focus_ring - "yes" or "no", default: "yes"
- has_button_effects - "yes" or "no", default: "yes"

A single gadget can have multiple text_objects, graphic_objects, and button_objects all positioned within its own bounding box. The default drawing callback will draw all graphics and text objects, as well as any button objects whose value matches the current value of the gadget, within the client area bounds of the gadget. The starting value of any gadget is 0 and is changeable via UI and API, but not yet via hxresfile.

The attribute “has_button_effects” is available to enable/disable special drawing of button-like gadgets by the system. If a gadget has a single button object and this feature is enabled, the button object will receive special effects for the hover, pressed, and disabled state where no image has been specifically provided. This enables a consistent look and feel for all 4 states various gadget-button objects across all programs.

18.4.5.11 button_object (inside <gadget>)

Elements:

- pos
- normal_state (required)
  - image
  - system_image
- pressed_state

  - image
  - system_image
- hilite_state
  - image
  - system_image
- disabled_state
  - image
  - system_image

Attributes:

- value - integer, default: 0

A button_object is drawn only when its value matches the current value of the parent gadget. For any image state not provided, the normal state will be shown.

For example:

```
<gadget id="kGadget4" type="toggle">
     <pos x="240" y="150"/>
     <size w="20" h="20"/>
     <button_object value="0">
       <pos x="0" y="0"/>
       <normal_state>
         <image file="checkbox_off_normal.png"/>
       </normal_state>
       <pressed_state>
         <image file="checkbox_off_pressed.png"/>
       </pressed_state>
       <hilite_state>
         <image file="checkbox_off_hilite.png"/>
       </hilite_state>
       <disabled_state>
         <image file="checkbox_off_disabled.png"/>
       </disabled_state>
     </button_object>
     <button_object value="1">
       <pos x="0" y="0"/>
       <normal_state>
         <image file="checkbox_on_normal.png"/>
       </normal_state>
       <pressed_state>
         <image file="checkbox_on_pressed.png"/>
       </pressed_state>
       <hilite_state>
         <image file="checkbox_on_hilite.png"/>
       </hilite_state>
       <disabled_state>
         <image file="checkbox_on_disabled.png"/>
       </disabled_state>
     </button_object>
   </gadget>
```

18.4.5.12 imagebox

Elements:

- pos
- size
- image
- system_image

Attributes:

- has_frame - "yes" or "no", default is "no"
- no enabled - imageboxes are inactive, so this attribute is not used
- no can_focus - imageboxes are inactive, so this attribute is not used

18.4.5.13 label

Elements:

- pos
- size (optional) - if not provided, toolkit will compute minimal bounding box
- font
- text - UTF-8 string
- color - default is "#000000" (black)

Attributes:

- alignment - left, center, or right, default is left
- origin - top, bottm, default is top

18.4.5.14 list

Elements:

- pos
- size
- listitem (multiple)

Attributes:

- allow_multiple_selection - "yes" or "no", default: no
- is_extended - "yes" or "no", default: no
- extended_item_height - integer, default: 14

Any tool that processes these files should ensure no more than one listitem is selected if allow_multiple_selection is "no". If a listbox is marked as extended, it can contain no <listitem> elements.

This is used often used with the columnheader item and a container.

18.4.5.15 listbox

Elements:

- pos
- size
- listitem (multiple)

Attributes:

- allow_multiple_selection - "yes" or "no", default: no
- is_extended - "yes" or "no", default: no
- extended_item_height - integer, default: 14
- has_frame - "yes" or "no", default: yes

Any tool that processes these files should ensure no more than one listitem is selected if allow_multiple_selection is "no". If a listbox is marked as extended, it can contain no <listitem> elements.

18.4.5.16 listitem

Elements:

- image (optional)
- system_image (optional)
- text (must be included)

Attributes:

- is_selected - "yes" or "no", default: no

These are used in listbox and dropdownbox controls to specify the items that are in the box. They never appear as top-level controls. They are not allowed in listbox elements that are marked as extended lists.

18.4.5.17 progressbar

Elements:

- pos
- size

Attributes:

- initial_value - decimal number, required
- max_value - decimal number, required
- Range is from 0 to max_value.

18.4.5.18 radio_button

Elements:

- pos
- text - UTF-8 string

Attributes:

- state - "on" or "off", default: off

This control can only exist inside a radio_button_group element.

18.4.5.19 radio_button_group

Elements:

- radio_button (multiple)

This is not a true control, but is instead a way to group radio buttons together. All radio_button elements inside this will be part of the same group. An error will be indicated if there is not exactly one radio_button within the group that is marked as "on."

18.4.5.20 scrollbar

Attributes:

- is_vertical - "yes" or "no", default: yes
- full_range - integer, no default
- visible_range - integer, no default
- initial_position - integer, default: 0
- step_size - integer, default: -1 (HxScrollbarSetButtonStep)
- Scrollbars are always children of other controls, so they have no position or size attributes, but are automatically placed with their parent.

18.4.5.21 separator

Elements:

- pos
- size
- color - default is "#000000" (black)

Attributes:

- none

18.4.5.22 slider

Elements:

- pos
- size - the height attribute is ignored for horizontal sliders, the width is ignored for vertical sliders

Attributes:

- orientation - either "horizontal" or "vertical", defaults to "horizontal"
- num_ticks - number of ticks, default is "0" for no tick marks
- initial_value - decimal number
- min_value - decimal number
- max_value - decimal number
- page_size - positive decimal number, default is "1"

18.4.5.23 spin_button

Elements:

- pos

Attributes:

- initial_value - decimal number
- min_value - decimal number
- max_value - decimal number
- increment - positive decimal number, default is "1.0"
- format - string, default is "%.0f"

18.4.5.24 storage_list

Elements:

- pos
- size

18.4.5.25 tab

Elements:

- text
- container - optional

Attributes:

- id
- is_selected - "yes" or "no", default: “no”

This is a tab that is used to decorate a tabbed_container. It can be associated with a container by including that as a child element. That container element should NOT include pos or size elements, as those will be inherited from the tabbed_container that holds the tabs.

18.4.5.26 container (inside <tab>)

Elements:

- background_color - background color of the container, default: transparent
- all controls

This container holds controls that are associated with a tab.

18.4.5.27 tabbed_container

Elements:

- pos
- size
- tab - multiple, but at least one required
- all controls

A tabbed container holds other controls within a application window or dialog. It differs from a normal container because it has a set of tabs on the top, and one must be selected at any time. This selection can be queried, and when the user clicks on a different tab, the appearance changes and a tab changed message is broadcast with the ID of the new tab.

18.4.6 Alerts

The following items can only appear as children of an alert.

18.4.6.1 alert_button

Elements:

- text - UTF-8 string

Attributes:

- value - the button number (0, 1, or 2)

For example:

```
<alert style="custom">
  <alert_button value="0">
    <text>Press Here</text>
  </alert_button>
  <text>A sample alert.</text>
</alert>
```

18.4.6.2 title

This is simply a UTF-8 string, shown in the title of an alert. This item is otherwise identical to the text element.

18.4.7 Meta-Information

18.4.7.1 focus_array

Elements:

- focus_item - multiple

Provide this as the last item in an application_window, window, or dialog to set a custom focus array which replaces the one built by the system. The array is a list of focus_items, with each item providing the ID of some item in the top-level container. This is not valid for top-level containers.

18.4.7.2 focus_item

Attributes:

- id - ID of previously defined control

18.4.8 Menus

IMPORTANT It is important to note that menus and menu items are not widgets. They do NOT have “enabled” or “visible” attributes.

18.4.8.1 menu

Elements:

- menu_item (multiple)
- menu_separator (multiple)
- submenu (multiple)

Attributes:

- id

This is a container for "menu_item" elements. It is used for sub-menus or top-level popup menus.

18.4.8.2 root_menu

Elements:

- submenu (multiple)

Attributes:

- id

This is a container for "submenu" elements. It is used for defining the top-level root menu shown when hitting the "Menu" key.

18.4.8.3 menu_item

Elements:

- image - optional
- system_image - optional
- text - UTF-8 string

Attributes:

- id
- shortcut - UTF-8 string, single character, default: empty
- enabled - can the item be selected by the user, default: "yes"

18.4.8.4 toggle_menu_item

Elements:

- image - optional
- system_image - optional

- text - UTF-8 string

Attributes:

- id
- shortcut - UTF-8 string, single character, default: empty
- enabled - can the item be selected by the user, default: "yes"
- selected - is the items checked?, no default

18.4.8.5 menu_separator

Attributes:

- id

18.4.8.6 submenu

Elements:

- image - optional
- system_image - optional
- text - UTF-8 string
- menu_item
- menu_separator
- submenu

Attributes:

- id
- shortcut - UTF-8 string, single character, default: empty
- enabled - state of the submenu, default: "yes"

For example:

```
<root_menu id="2000">
  <submenu id="2001">
    <text>File</text>
    <menu_item id="2010"> <text>Open</text> </menu_item>
    <menu_item id="2011"> <text>Close</text> </menu_item>
    <menu_separator id="2012"/>
    <menu_item id="2013"> <text>Exit</text> </menu_item>
  </submenu>
  <submenu id="2020" enabled="no">
    <text>Edit</text>
    <menu_item id="2021" shortcut="X"> <text>Cut</text> </menu_item>
    <menu_item id="2022" shortcut="C"> <text>Copy</text> </menu_item>
    <menu_item id="2023" shortcut="V"><text>Paste</text> </menu_item>
    <menu_separator id="2024"/>
    <menu_item id="2025"> <text>Select All</text> </menu_item>
  </submenu>
</root_menu>
```

18.5 String Resources

The HxUI XML format supports a simple string lookup mechanism. Strings are represented as top-level items, children of the hxui element. The text of the string is embedded directly within the element, there is no text sub-element. Each string has its own ID value.

All of the strings are loaded into a system hash table when the HxUI XML file is opened, and lookups are quick, not requiring an XML reparse.

18.5.0.1 string

Attributes:

- id

For example:

```
<string id="6001">This is my first string</string>
<string id="6002">This is my second string</string>
<string id="6003">The third string has a couple of
embedded line terminators.
</string>
```

18.5.1 String Example File

This is a basic HxUI XML file showing practical usage of string:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hxui SYSTEM "hxui_1.0.dtd">
<hxui version="1.0" default_locale="en_US">
  <application_window id="1000">
    <command_button id="1001">
      <pos x="100" y="100"/>
      <text>OK</text>
    </command_button>
    <field id="1002" is_multiline="yes">
      <pos x="100" y="120"/>
      <size h="400" w="300"/>
    </field>
    <listbox id="1003">
      <pos x="550" y="120"/>
      <size h="400" w="300"/>
      <listitem>
        <text>One</text>
      </listitem>
      <listitem is_selected="yes">
        <text>Two</text>
      </listitem>
      <listitem>
        <text>Three</text>
      </listitem>
      <listitem>
        <text>Four</text>
      </listitem>
    </listbox>
  </application_window>
  <root_menu id="2000">
    <submenu id="2001">
```

```
      <text>File</text>
      <menu_item id="2010">
        <text>Open</text>
      </menu_item>
      <menu_item id="2011">
        <text>Close</text>
      </menu_item>
      <menu_separator id="2012"/>
      <menu_item id="2013">
        <text>Exit</text>
      </menu_item>
    </submenu>
    <submenu id="2020">
      <text>Edit</text>
      <menu_item id="2021" shortcut="X" enabled="no">
        <text>Cut</text>
      </menu_item>
      <menu_item id="2022" shortcut="C">
        <text>Copy</text>
      </menu_item>
      <menu_item id="2023" shortcut="V">
        <text>Paste</text>
      </menu_item>
      <menu_separator id="2024"/>
      <menu_item id="2025" shortcut="A">
        <text>Select All</text>
      </menu_item>
    </submenu>
  </root_menu>
</hxui>
```

18.6 Localization

18.6.1 Locale-dependent File Lookup

When a HxUI file is opened, the library will get the current LC_MESSAGES locale from the system using "setlocale" and use that to modify the filename passed in. For example, if the current setting was "es_MX.UTF8" and you opened "Resources/foo.hxui", it would look for files to open in this order:

1. Resources/es_MX/foo.hxui
2. Resources/es/foo.hxui
3. Resources/foo.hxui

The system doesn't verify that the default_locale attribute of the hxui tag matches the language picked, but it's a good idea.

For any files referenced from the HxUI file, such as fonts and images, the same search mechanism is used, with the default directory being the directory of the HxUI file that was passed into the HxResFileOpen call. In this example, if you had loaded "Resources/es_MX/foo.hxui" and you referenced "bar.png", it would search the following:

1. Resources/es_MX/bar.png
2. Resources/es/bar.png

3. Resources/bar.png

to find the image.

Using parallel HxUI files is the preferred method of localization on the device.

18.7 Editing Tips

18.7.1 Custom colors in Emacs

If you use the Emacs text editor on your development PC, edit your .emacs file and insert the following code:

```
(push '("\\.hxui" . sgml-mode) auto-mode-alist)
```

This snippet enables syntax coloring making it much easier to read the XML tags.

18.8 HxUI APIs

The HxUI APIs are declared in `/hx/ui/hxresfile.h`. Please reference the Foleo API guide for more detailed information on the HxUI APIs used by the system.

CHAPTER 19

Sound

This chapter provides an overview of the Falcon's sound system.

19.1 AC97 Support

The

CHAPTER 20

Messaging System

This chapter provides an overview of the Falcon's messaging system.

20.1 Messaging

The

CHAPTER 21

Wireless

This chapter provides an overview of the Falcon's wireless capabilities, including Bluetooth and WiFi.

21.1 Bluetooth

The Falcon ships w

21.2 WiFi

The Falcon contains a

CHAPTER 22

Storage and File Management

This chapter provides an overview of the Falcon's data storage and file manament systems. It covers both the internal flash memory as well as user expandable options such as SD card, Compact Flash and USB drives.

22.1 Default storage

The Falcon ships with 128 MB of user accessible storage space in the internal flash memory. This storage space is currently divided into both a user area and a system area. When the system is flashed with a new ROM file, you are given the option of flashing both areas or only updating the system area.

Flashing both the user and system areas of the internal flash memory is equivalent to a "hard reset" on a Treo device. While the flash does not perform a secure wipe, it does erase everything on the device and returns it to a generic "out-of-the-box" state.

Flashing just the system area preserves any custom settings, installed programs and user-specific data while updating the core system software. It is expected that end-users will use this method to update their devices with official system ROM updates.

22.1.1 User storage

The user storage area contains any custom settings, installed programs and user-specific data. This storage area defaults to the internal flash memory, however if a Compact Flash card is integrated into the system the user storage area is copied to the CF card and stored there.

22.1.2 System storage

The system storage area is where the Falcon device stores the core Operating System files as well as any bundled "in-ROM" software. This includes the Documents To Go suite, the email client and the web browser.

This storage area is always kept on the internal flash memory and is not user customizable.

22.2 Compact Flash card

A Compact Flash card can be integrated into the system in order to upgrade the size default user storage area. Currently the Falcon device supports Compact Flash cards up to 4.0GB in size.

Unlike other storage options, the Compact Flash card is meant to serve as a permanent storage location. It is NOT meant to be used as a removeable drive. Once installed the Compact Flash card becomes an integral part of the system and serves

as the default user storage area. It is best to consider the Compact Flash card as the Falcon equivalent of an internal hard drive.

The Compact Flash card slot is located internally, inside the battery compartment.

22.2.1 Integrating the Compact Flash card

To integrate a Compact Flash card into the Falcon perform the following steps:

1. Turn off the Falcon.
2. Remove the battery.
3. Insert the Compact Flash card.
4. Replace the battery.
5. Turn on the Falcon.
6. Wait for the “Integrate Compact Flash Card?” dialogue to appear.
7. Click “Yes.”

After you click “Yes” the Falcon copies the contents of the internal flash memory onto the card and reboots the system. Once the system reboots it is running off the Compact Flash card and can be used as normal.

IMPORTANT The Compact Flash card is NOT removeable storage. Integrating the card will erase any existing data and prepare it for use within the Falcon.

22.2.2 Format the Compact Flash card

To format a Compact Flash card that is installed in the Falcon you must open a terminal prompt and then issue the following commands:

```
sume
/sbin/format_cf
```

Once this process finishes, the Compact Flash card is erased. However it cannot be re-integrated into the system until the old mount point is removed.

22.2.3 Prepare to re-integrate a Compact Flash card

Once a Compact Flash card has been integrated into the Falcon, there is no simple way of removing it. In order to re-use a formatted Compact Flash card or install a new one, you must remove the old mount point.

You can remove the old mount point by opening a termainal prompt and issuing the following commands:

```
sume
rm /mnt/hda1/.holly_fs
```

The system is now back to its pre-Compact Flash state. You can integrate a Compact Flash card as normal via the Preferences panel.

22.2.4 Tested cards

The following Compact Flash cards have been tested as working on the Falcon device.

22.2.4.1 Compact Flash Type I

Transcend 8GB

Lexar 8GB

SanDisk 8GB

SimpleTech 8GB

Lexar 4GB

SanDisk 4GB

Pretec 4GB

Kingston 4GB

Canon 4GB

Sandisk512MB

Lexar 1GB 80X

Sandisk1GB

Kodak512MB

Lexar 256MB 12X

Sandisk512MB Ultra II

Lexar 2GB 80X

Kodak256MB CompactFlash Picture

PNY 512MB

Lexar 512MB 40X

Kingston 1GB

Sandisk256MB

Kodak1GB Memory

PNY 256MB

Sandisk2GB Ultra II

Lexar 1GB 4X

Kingston 512MB SLC

PNY 1GB

Lexar 512MB 4X

Lexar 512MB 80X

Lexar 256MB 4X

Kingston 2GB Elite Pro

Lexar 256MB 40X
Sandisk1GB Ultra
Lexar 4GB 80X WA Pro II
Sandisk128MB
Lexar 8GB Pro 40X
Simple 256MB Type I
Kingston 512MB
Dane Elec256MB 12X
Lexar 1GB 40X
Sandisk1GB Extreme
Lexar 128MB 12X
Sandisk256MB Ultra
Lexar 512MB 16X
Sandisk32MB Memory
Kingston 256MB
PNY 128MB 3.3V
Dane Elec512MB DA-CF512-R
Lexar 128MB 4X
Delkin 1GB
Microtech Intl256MB CompactFlash Digital FlashFilm
Lexar 1GB 40X
Lexar 512MB 12X
Delkin 512MB
Kodak512MB
PNY 64MB Memory
Simple 1GB
Sandisk2GB Extreme III
Sandisk2GB
Delkin 2GB Pro High Speed
Microtech Intl128MB Film
Sandisk4GB Extreme III
Sandisk512MB Extreme
Fujifilm128MB
Delkin 4GB
Viking256MB CompactFlash

Kingston 4GB Elite Pro

Promaster64MB ProMaster Memory

Sandisk8GB Ultra II

Viking512MB

Sandisk4GB Ultra II

Viking1GB

Sandisk64MB

Delkin 256MB

Viking128MB Memory

Delkin 1GB Pro High Speed

Atp 1GB High Speed 60X

Atp 512MB High Speed 45X

Sandisk2GB Extreme

Sandisk4GB

Delkin 128MB

Simple 512MB

Memorex128MB

Lexar 1GB

Lexar 32MB Memory

Viking32MB

Kodak128MB CompactFlash Picture

X Digital 512MB 70X PROformance

Memorex512MB

Transcend512MB

Fujifilm64MB

Fujifilm256MB

Kingston 128MB

Lexar 64MB 12X

Sandisk8MB Memory

Memory Upgrades1GB CompactFlash Storage

Kodak256MB

Peripheral Enhancements512MB

PQI1GB 40X

Canon128MB

Viking16MB

22.2.4.2 Compact Flash Type II

Lexar 4MB, 8GB

Seagate 5, 8GB

Western Digital 6GB

GS Magic 4, 6, 8GB

Sony 4, 6GB

Hitachi/IBM 512MB, 1, 4, 6, 8GB

SimpleTech 160MB, 192MB, 224MB, 512MB/ 1GB

22.3 SD/MMC card

The SD/MMC card slot serves as the primary method of removeable storage on the device. Located on the right hand of the Falcon, it supports both SD (Secure Digital) and MMC (MultiMedia Card) devices.

SD/MMC cards are automatically mounted upon insertion. The mount point for a SD/MMC card is */Storage/0*.

22.3.1 Format

To ensure maximum compatibility, the Falcon formats the SD/MMC card with a FAT16 filesystem. This is the default filesystem used by MS-DOS as well as a number of embedded devices. The Treo line of smartphones can read and write to FAT16 filesystems, so any card formatted by the Falcon can be used interchangeably with a Treo as well as Linux, Windows or OS X.

If you choose to programatically format a SD/MMC card ensure that the FAT16 filesystem is used.

22.3.2 Tested cards

The following SD/MMC cards have been tested as working on the Falcon device.

22.3.2.1 SD

eXpansys Memory 4GB

Transcend 4GB

SanDisk 4GB

Pretec 4GB

ATP 4 GB

Sandisk 2GB Ultra II

Atp 2GB

Lexar 2 GB

Pretec 2GB

Kingston 2GB

Transcend 2GB

Kodak 2GB

PQI 2GB

Sandisk 1GB

Sandisk 512MB

Sandisk 256MB

Sandisk 1GB Ultra II

Sandisk 512MB Ultra II

Sandisk 128MB SD

Sandisk 1GB Ultra II

Sandisk 2GB Ultra II

Sandisk 1GB Ultra II Plus w/USB

Sandisk 512MB Gaming Green

Sandisk 256MB Ultra

Sandisk 256MB Gaming Orange

Sandisk 128MB Gaming Pink

Sandisk 32MB

Sandisk 512MB Extreme

Sandisk 512MB Ultra II Plus w/USB

PNY 512MB

Kodak 512MB

Lexar512MB

PNY 256MB

Lexar 256MB

Kodak256MB

Lexar 128MB

Kingston 1GB

Lexar 1GB 32X HS

PNY 1GB

Kodak 1GB

PNY 128MB

Kingston 512MB

Dane 256MB

Promaster 128MB ProMaster

Lexar 1GB

Dane 128MB

Dane 512MB

Kingston 128MB Hi-Speed

Kodak128MB

Kingston 256MB

Kodak256MB

Lexar 512MB 40X

Lexar 256MB 40X

Delkin Devices1GB

Lexar 1GB 40X

Viking512MB

Simple 128MB

Viking256MB

Lexar 1GB 40X Platinum

Lexar 256MB Platinum

Lexar 512MB 32X HS

Lexar 512MB Platinum

Delkin 512MB

PNY 512MB w/256MB SD

Lexar 256MB 32X

Delkin 256MB

Atp 1GB 60X

Panasonic 256MB RP-SD256BU1A SD

Kingston 512MB Elite Pro

Kodak256MB

Fujifilm 256MB

Atp 256MB 60X

Atp 512MB 60X

Viking1GB

Viking128MB

Panasonic 512MB

Peripheral 512MB

Delkin 128MB

Atp 2GB 60X

Atp 128MB

Sandisk 64MB

Kodak 128MB

Upgrades 1GB

Kingston 128MB

Atp 512MB 60X Mini

Simple 256MB

Palm 256MB Expansion for PalmOne

Lexar 64MB

Kodak 64MB

Fujifilm 128MB

Lexar 32MB

Delkin 512MB Pro

Upgrades 512MB

Palm 128MB

Lexar 1GB 60X

Kingston 256MB Hi-Speed

Fujifilm 64MB

Simple 1GB

PQI 1GB

Kingston 64MB

Upgrades 256MB Mobile Mini

PNY 64MB

Sandisk 1GB Extreme III

Upgrades 128MB

PQI 256MB

Panasonic 64MB

Peripheral 256MB

Palm 64MB

22.3.2.2 MMC

Super Talent Flash 2GB

Kingston 2GB

Transcend 2GB

Super Digital 2GB

PNY 2GB

Ritek Ridata 2GB

Hitachi 2GB

Transcend 4GB

Pretec 4GB 4.0

Kingston 256MB

Kingston 128MB

Kingston 1GB

Sandisk128MB

Memorex128MB

Lexar 32MB

Nokia64MB MMC for 9290 Communicator

Palm1GB for Tungsten/Zire/Treo

Palm64MB

Atp 1GB High Speed MMC Card

Memory Upgrades256MB MMC

Memory Upgrades512MB Mobile

Simple 128MB

Kingston 512MB MMCplus

Simple 32MB

Atp 512MB High Speed MMC Flash Card

Lexar 64MB MMC

Peripheral Enhancements128MB EDGE Digital Media Multi-Media Card

Simple 256MB

Sandisk64MB

Lexar Media128MB MMC

Memory Upgrades64MB Mobile Multimedia Flash Card

Simple 64MB

PQI64MB

Memory Upgrades128MB MMC

22.4 USB drive

The USB port on the Falcon supports the standard mass storage profile. Any flash drive or standard hard drive that conforms to the mass storage profile will be automatically mounted upon connection to the Falcon.

NOTE The Falcon does NOT currently support the NTFS filesystem so any large hard drives that have been formatted as NTFS under Windows XP or Windows Vista will not be usable in the Falcon.

22.4.1 Optical drive

<HAVE TO TEST>

22.5 Filesystems

The following filesystems are supported on the Falcon:

<IS THIS SOMETHING WE WANT TO EXPOSE?>

22.6 Default storage location

22.6.1 Applications

All third party applications should install to the following location:

```
/Home/User/Applications/
```

This folder is designated as the default storage location for all user-installed software. As part of the user storage area it will be automatically copied to a Compact Flash card should the user choose to integrate one into the system. It will also remain intact during a system ROM upgrade if the user chooses to preserve the user storage area.

22.6.2 Data

Any user information, such as documents, pictures, default images or application configuration files should be stored in the following location:

All third party applications should install to the following location:

```
/Home/User/Data/
```

This folder is designated as the default storage location for all user-created data. As part of the user storage area it will be automatically copied to a Compact Flash card should the user choose to integrate one into the system. It will also remain intact during a system ROM upgrade if the user chooses to preserve the user storage area.

CHAPTER 24

System Support

This chapter provides an overview of the Falcon's auxillary functions.

24.1 Power

The

PART V

Open Source Tools

This part of the guide reviews many of the open source tools that are used with the Falcon.

CHAPTER 25

Support Libraries

25.1 Applications

25.1.1 vi

The vi file editor is a powerful tool in the hands of an experienced user, however the custom command set can present a bit of a learning curve for those unfamiliar with the program. A staple among Linux distributions, vi behaves the same on nearly every system.

Key to using vi is understanding the different modes of the editor. As you switch between modes the same keys take on different commands. Once mastered this allows for extremely efficient editing, though it is often the source of initial frustration.

25.1.1.1 vi Command Primer

Covering the inner workings of vi is beyond the scope of this document, but there are many resources available to learn the basic commands of the editor. Such a tutorial is available on the University of Illinois's Unix 101 web site.

http://www.uic.edu/depts/accc/software/unixgeneral/vi101.html

25.1.1.2 Limitations

The version of vi included on the Foleo will close when the command <Menu>-<c> is issued. Issuing an equivalent command on a standard distribution of vi does not cause the editor to close.

- Hx App Manager H) - The Application Launcher presents the system UI to the user. It handles the App Launcher and System Bar.
- Falcon Versamail (Hp)
- Browser UI Layer (H)
- DataViz Word To Go (L)
- DataViz Sheet To Go (L)
- DataViz Slideshow To Go (L)
- Hx Preferences (H)
- Hx App Launcher (H)
- VM Contacts module (H)

- Setup Wizard (H)
- Hx System Bar (H)

25.2 Daemons

- Power d (H)
- BlueZ (Oc)
- WiFi Supplicant (Oc)
- Connection Manager (H)

25.3 App Frameworks

- UNO (Hp): iData, Background Service, and SyncSAM
- Octane (Hp)

25.4 App libraries

- libHx (H) - Falcon also provides non-GUI system support with libhx such as database, ipc, systempaths, and sound.
- HxSync (Hp) - Syncing is key to our company, handhelds and Falcon, so Palm is keeping HxSync proprietary.
- SyncMLlib (O) - Our syncing strategy follows the SyncML standard.
- glib (O) - Although Falcon does not use GTK, the glib utility library is provided and useful (although gobjects, pthreads, and messages are not included).
- glibc (O) - the standard library from Gnu.
- Access HTML Core (L) - to render HTML.
- Palm HTML Container (L) - to access the Internet.
- Open SSL (O)
- Zlib (O) - ZLib is a well-used cross-platform lossless data-compression library. For example, the gzip tool uses this library for compression and decompression. Various other libraries and graphics compression formats use this library as part of their system, hence this library is included in the Falcon system. For more information, see **http://www.gzip.org/zlib/**.
- gettext (O) - gettext is a set of utilities and libraries for building applications and libraries where multiple language strings could be used. gettext is a de facto standard concerning Linux. For more information, see **http://www.gnu.org/software/gettext/**. Falcon uses gettext to achieve language-independent string handling for localization.
- expat (O) - Expat is an XML parser – a stream-oriented parser where an application registers handlers for things that the parser might want to find in an XML document. As such, expat operates more like a state machine. Expat is used in various parts of the other packages used for parsing XML. The Falcon Application Manager is also using Expat for parsing info.hxi files and other Falcon-specific configuration files. For more information, see **http://expat.sourceforge.net/**.

- ReadLine (O)
- SQLite (O) - SQLite is part of the base Falcon system for database support. It is a small C library that implements a self-contained, embeddable SQL database engine. The preferences system and parts of the synchronization libraries are currently using SQLite. Developers can also make use of SQLite in their applications. For more information see:
  - **http://www.sqlite.org**
  - **http://www.sqlite.org/docs.html**
- libHxUI (H) - built on LiTE and provides all of the Falcon GUI.

25.5 Utility libraries

- util-linux (O)
- ProcPS (O)
- coreutils (O)
- busybox (O) - provides some of the same Linux file and shell utilities as available from Gnu, but is less full-featured to appropriately fit into Embedded Linux.

25.6 LiTE UI toolbox framework

LiTE is a LightWeight Toolbox Enabler. It is built on top of DirectFB. The main job of LiTE is to abstract out the DirectFB surface and subsurface structures as well as the internal event handling into easily understandable components that could be used for building toolbox widgets. LiteBox is the building block for building clickable buttons, labels, editable text, and so on. For more information, see the LiTE Architecture Tech Note.

25.7 Graphics library

Falcon uses DirectFB for the underlying graphics primitives, toolbox support in combination with LiTE, event handling, clipboard support, the window manager and application launching, and so on.

DirectFB is a thin library that provides hardware graphics acceleration or SW emulation, input device handling, windowing support including translucent windows. It operates on top of the Linux framebuffer device. Each hardware abstraction layer has a corresponding software fallback that is not supported by hardware.

DirectFB also has an IPC implementation called fusion, which Falcon uses for rapid between-process messaging.

For more information on DirectFB, see:

- **http://www.directfb.org**
- **http://www.directfb.org/docs/DirectFB_Reference**

25.8 Font libraries

- Fonts (L)
- fontconfig (O) - manage fonts
- FreeType (O) - FreeType is the main font engine inside DirectFB.

  FreeType is an open source font engine that understands Truetype, Postscript and various bitmap format fonts. It handles both standard ASCII as well as various Unicode formats. For more information, see **http://www.freetype.org**.

25.9 Graphics Codecs

Graphics libraries, notably PNG, jpeg and non-animated gif.

- GIF (O)
- libJPG (O)
- libPNG (O)

25.10 File Systems

A ton of file system support that now includes unionfs to tie them together. Much of it is open source and documentation is available online.

- EXT2/3 (O)
- FAT32 (O)
- JFFS (O)
- RAM FS (O)

25.11 Kernel modules

- Fusion (O) - DirectFB provides only one drawing surface. For multiple surfaces, Falcon uses the Fusion module. Using tempFS RAM Disk for IPC, Falcon also uses it for system messaging. Linux has a number of IPC mechanisms, but DirectFB invented their own to get large chunks of data back and forth. This was chosen for Falcon's IPC.
- ppp (O)

25.12 Kernel

- Linux Kernel 2.4.21 (Oc) - The basis of our OS is Linux 2.4.21.

25.13 Hardware drivers

- CF (Oc)
- SD (Oc)
- NAND (Oc)

- Power (Oc)
- Battery (Oc)
- BT (Oc)
- WiFi (Oc)
- Keypad (Oc)
- LED (Oc)
- Audio (Oc)
- USB (Oc)
- LCD (Oc)
- Frame Buffer (Oc)

25.14 Boot time only

- BLOB (Oc)

25.15 Non-ROM components

- binutils (O)
- DFB Term (L)
- NAND Updater (L)
- strace (O)
- modutils (O)
- powerctl (H)
- BlueZ bluefw (Oc)
- BlueZ hcidump (Oc)
- gcc (O)

PART VI

Style Guide

This part of the guide describes development “best practices” when creating applications for the Falcon.

CHAPTER 26

Style Guide

This chapter provides guidelines developers should follow for the *look and feel* of apps that use software components in the SDK.

26.1 Applications

26.1.1 Description

Generally speaking, apps take over the entire screen. Multiple apps may be running under the covers briefly. Background processes such as mail delivery are supported within this model.

26.1.2 Goals

The user can launch any Falcon applications instantly.

Switching between applications happens instantly.

There is no perceptible load time for an application.

Do not expose the user to the concept of an app running vs. not running, other than whether it is on screen.

All app functionality should be possible without a hard drive (flash memory should not be a limiting factor).

26.1.3 Startup modes

Falcon should support apps which want to differentiate between being launched via the launch menu and being launched using next/previous app.

A browser is an example of an app that might differentiate.

- If the browser is launched via the launch menu, it might want to go to a home page.
- If the browser is launched via next/previous app, if might want to go back to its previous state instead (to facilitate copy/paste and the like).

A word processor is an example of an app that might not differentiate. It would always start in the same state as when last run., with the same open files and the same view.

26.1.4 Launching applications

Applications can be launched in the following ways:

- Application launch menu, a system-wide function that allows access to a menu of installed applications.
- Next/Previous App key combo.
- Hyperlinks from other applications (for example, clicking on a URL link in a mail message launches the browser)

26.2 File management

26.2.1 Description

- The underlying OS supports a full-blown file structure, but Falcon application conventions simplify the model.
- Expert users require some method of accessing and modifying file locations, however. This functionality is provided in the File Manager application.
- The design is optimized for files to be accessed through the application. In other words, you launch the application and select open rather than navigating a directory tree and clicking on a file.
- Files are saved in place. There is no explicit Save command (although there is a Save As command so you can rename or move files).

26.2.2 Goals

Falcon application file management should minimize user frustration with:

- Spending time figuring out where files are located
- Inefficiency of navigating through file structures to find files
- Misfiling and losing files

26.2.3 Functional details and UI flow

26.2.3.1 File types

- Falcon applications have a default MIME type or equivalent identifier. This requires support for some kind of registration handler.
- Extensions are shown on all files that have them in their file name.
- Determining the type of a given file should be done via information in the file first. The extension should only be used as a last resort to determine file type.

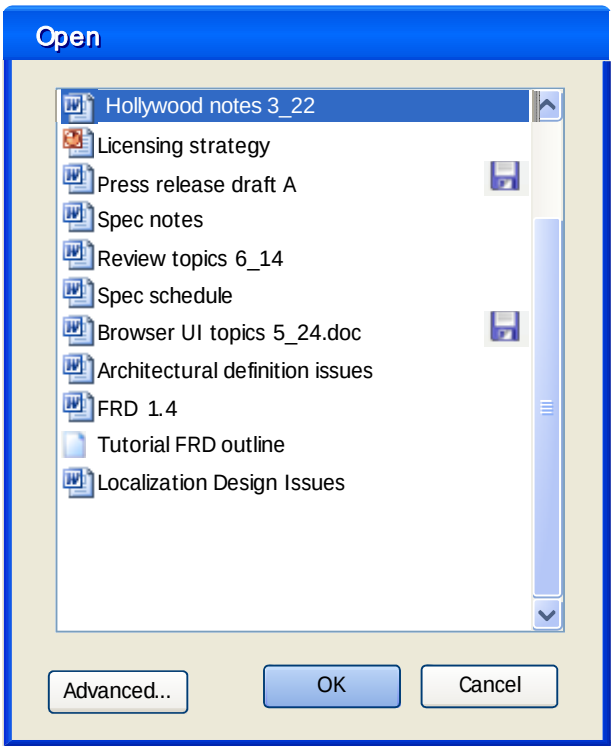
26.2.3.2 Opening a file

Falcon applications should present the following to commands for opening files:

- Recent Files
- Open

26.2.3.3 Recent Files dialog

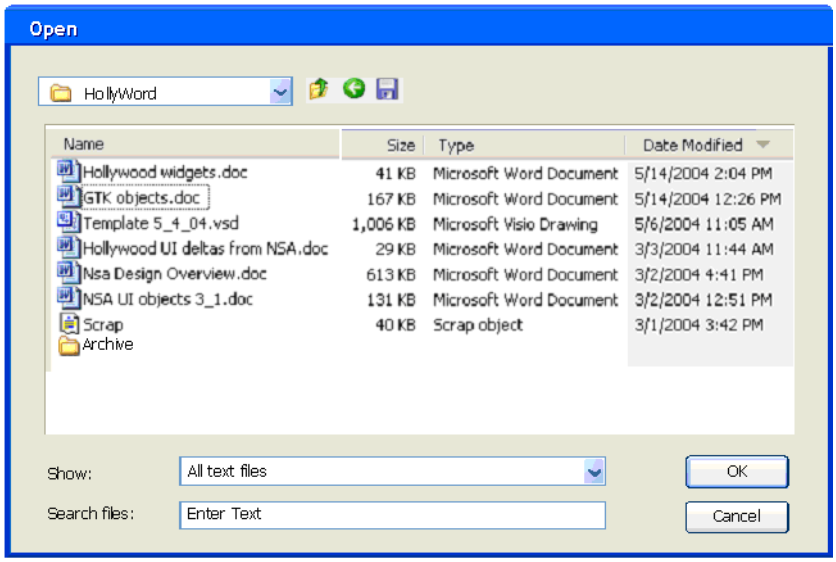
- The Recent Files UI will be a dialog rather than a menu. Dialogs allow a longer list than a menu, and a clean checkpoint for missing or moved files. The list in a dialog provides faster access with scroll point and with command shortcuts.



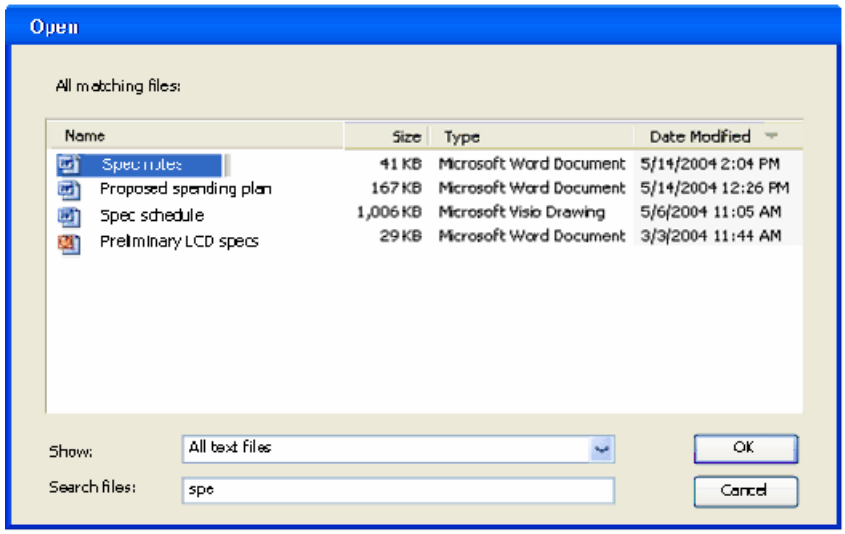
- Title: “Recent Files” (not shown)
- Extensive list of recent files (as many as 50)
  - Flat list, regardless of location
  - If on card, display icon (floppy disk icon in screenshot below is a placeholder)
  - No Find option
  - The Advanced button should be changed to Browse. Clicking the Browse button should close the Recent Files Dialog and open the Open dialog.

26.2.3.4 Open dialog

- The Open dialog should be a standard file hierarchy browsing interface.



- The search files field is a simple Find function.
- The results of the find should be displayed in the dialog below.



26.2.4 Saving changes to files

- File saving behavior is determined by the app, but the system convention is to save in place as you go.
- No Save menu is required.
- A Save As menu allows you to change the name or create a copy at any time.

NOTE The downside of this approach is that it puts a greater burden on the Undo functionality of the application.

- User should not have to think about saving files.
- User should not lose data when the application crashes.

26.3 Navigation

26.3.1 Goals: keyboard vs. trackpoint

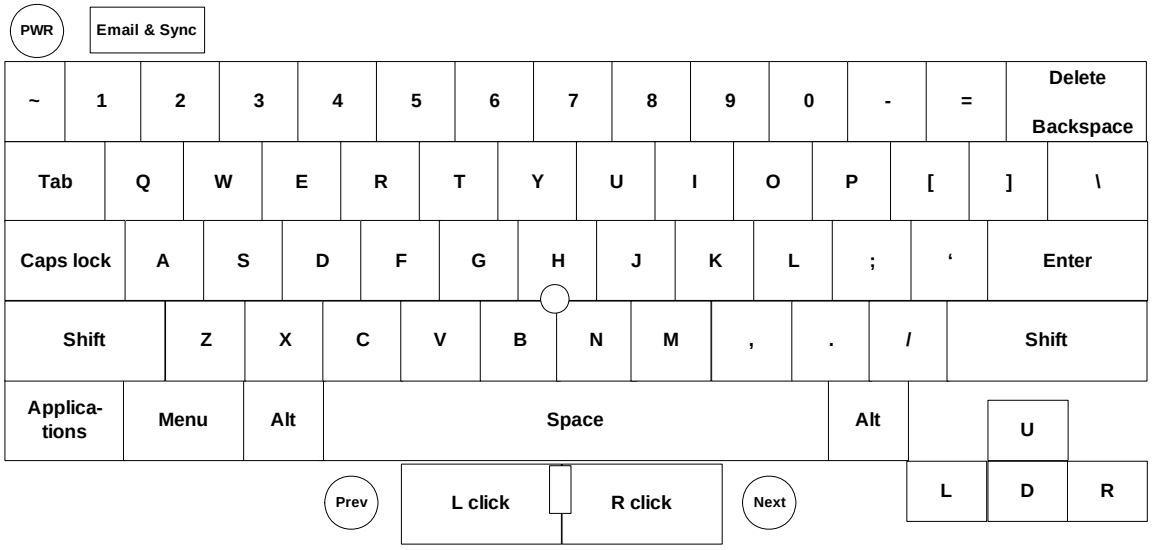
- Don't rely on keyboard navigation to replace trackpoint totally.
- Eliminate worst situations, for example, window management and scrolling.
- Implications
  - May still use keyboard shortcuts
  - Look at ways to fine tune trackpoint hardware/software.

26.3.2 Description

The following tools can be used to navigate within applications:

- Trackpoint
  - Generic pointing device to target any item on screen
  - Only method to access drawing of polygons or freeform shapes
  - Multiple select by dragging
  - Useful for editing or quick navigation between objects on a screen (although there may be keyboard shortcuts for these operations)
  - Redundant with keyboard navigation for scrolling and simple modal dialog navigation
- Scroll keys
  - Select items in list
  - Move cursor in text objects
- Scroll wheel
  - Line or page scroll items in a list (keyboard equivalent of scrollbar, and implemented consistently across the system)
  - Single click of the wheel generally: line scroll
  - Fast spin of the wheel ('ing) generally: page scroll.
- Tab key
  - Navigate between items in the focus queue
  - Shift + tab: select previous item
- Shift key
  - Multiple select (text selection or items in list)

- Alt key: auxiliary navigation key
  - Alt + Click: discontiguous select
  - Alt + scroll: can be used for power navigation
- Enter key: OK or keydown
- Left-click: mouse click
- Right-click: context-sensitive menu
- Backspace key
  - Backspace: backspace or delete a record
  - Shift + Backspace: forward delete or delete a record



26.3.3 Common functions

The following functions are not yet implemented systemwide, however all applications should implement the stated behaviour as follows:

- Menu + Backspace: delete word before cursor
- Shift + Menu + Backspace: delete word after cursor

26.3.4 Reserved functions

- The menu + number key combinations are reserved for use for system-wide functions like volume, brightness, etc. Developers may not use a reserved key combination for application shortcuts.

26.4 Wireless connectivity

Falcon supports two types of wireless connectivity:

- Bluetooth

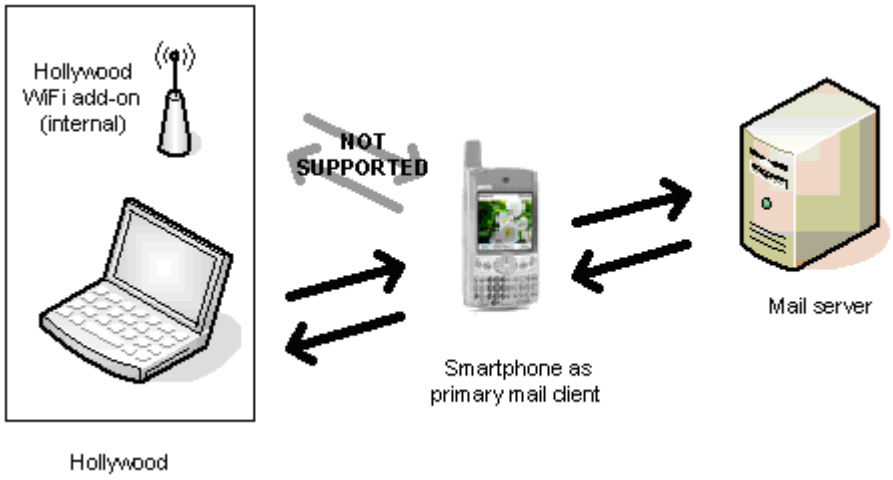
  - Synchronization of mail and PIM data
  - Support for wireless mouse peripherals
- WiFi
  - Web browsing
  - Direct access to POP3 mail

26.4.1 Wireless goals

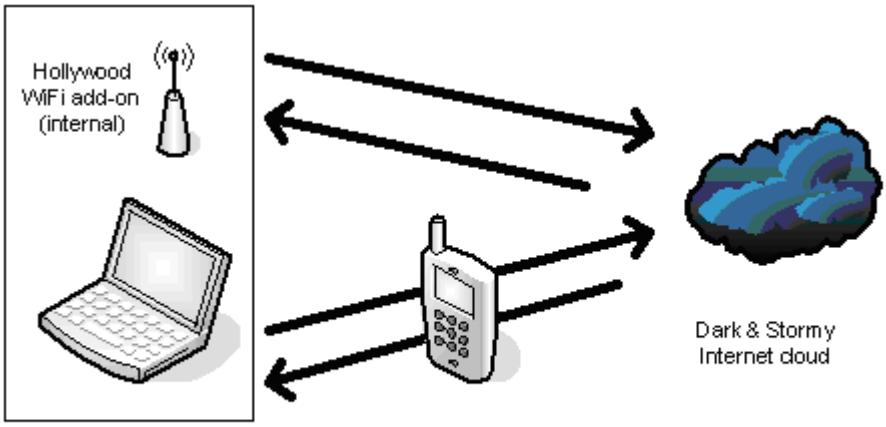
- Minimize the need for a user to be aware of or deal with differences between WiFi and Bluetooth.
- Be smart about providing the best connection option.
- Provide flexibility for power users but don't expose UI at the top level (“provide but hide” power features).

The following diagrams capture the primary data transfer cases (excluding the mouse option), in descending order of priority.

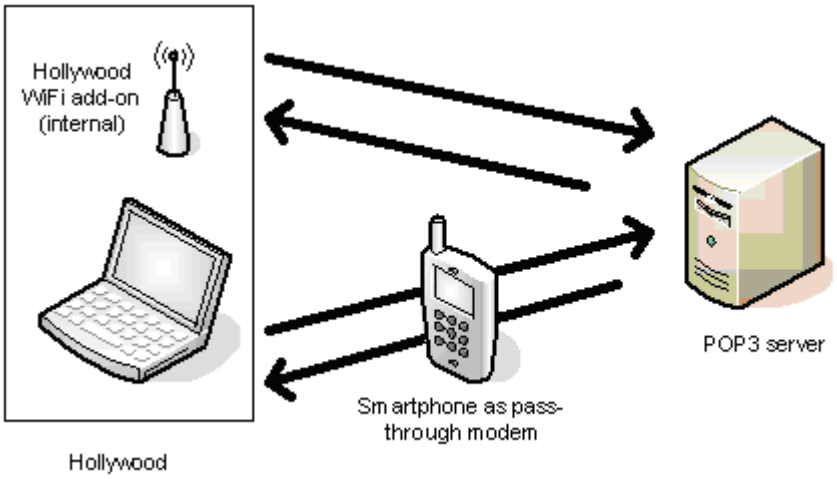
Synchronization with smartphone (primary usage case)



Smartphone as web access modem (secondary usage case)



Smartphone as pass-through modem for POP3 mail (power user case). This is basically the same as web access.



26.5 Security

Security should strikes a balance between protection and usability.

26.5.1 Protection

- Prevent data from being accessed by unauthorized individuals
- Prevent installation of malicious apps or viruses
- Prevent the user from inadvertently damaging the system by locking access privileges

26.5.2 Usability

- Hide or minimize complexity of security from the user

- User should not need to know about root-level security (for example, don't require an administrator password for basic functions like installing apps)
- Tie different security components to one password if possible

NOTE Usability goals are not simply for convenience. Poor UI security usability often results in the user bypassing the security or being unable to access important functionality.

26.5.3 Locking Falcon

- To prevent a stranger from picking up a Falcon and accessing the owner's data, the owner can set a password to access the device.
- When password lock-out is enabled, the idle screen (with password field in the title bar) should be shown whenever the device wakes up or is rebooted.
- Entering the correct password should make the device proceed with whatever it would have done if lock-out was disabled.

26.5.4 Preventing malicious damage by apps

- To prevent malicious apps from installing themselves without the user's knowledge, a confirmation prompt is required when apps are installed.
- To prevent malicious system damage, potentially dangerous operations should require confirmation prompts.

26.6 Error handling

26.6.1 Reboot

Holding down the power button for 5 seconds should shut down and reboot the device. On reboot, the File Manager app should be launched automatically.

26.6.2 Force quit an application

If an app hangs, a keystroke combination allows the user to force quit the foreground application.

- Proposal: App key + Delete.
- Displays confirmation dialog
- If closed, the System Preferences application automaticaly restarts

NOTE Falcon only supports force quitting the foreground application.

26.6.2.1 Force Quit confirmation dialog

- Standard Falcon dialog window
- Text stating "Are you sure you want to force the application '*<Application Name>*' to quit? All unsaved data will be lost."
- Button with text "*Yes*"

  - Clicking Yes will force quit the foreground application.
- Button with text "*No*"
  - Clicking No will cancel the force quit and return the user to the foreground application.

26.6.3 Crash and recover behavior

- When an application crashes, it should not hang the system.
- If the system does hangs, the screen freezes (no BSOD).
- When the system reboots after a forced quit or system hang, the system simply reboots and the System Preferences application is launched.

26.7 Media support

26.7.1 Graphics

Supported formats are:

- JPG
- GIF
- PNG

NOTE BMP and TIFF are not supported.

26.7.2 Video

Falcon does not currently ship with any support for video playback or embedded codec support.

26.7.3 Audio

- Supported formats: PCM audio
- The first generation of Falcon supports a simple output volume control.
- No input.
- No option to change select sound drivers.
- No option to select synth engine.
- No UI to change system sounds.

26.8 System services

26.8.1 Clipboard

The clipboard is described in **Clipboard**.

26.8.2 Windows

Falcon is architected to support multiple windows, but the system convention is to present applications in a PDA-like model of one window at a time.

Windows are displayed for the following cases:

- Utility dialogs (Open, Save As, etc.)
- Confirmation dialogs
- Subviews of an application, for example a record view of contacts
- Alerts
- Popups in the browser

26.8.2.1 Dialog box UI

- Title bar
  - Dialog title
  - Toolbar (optional)
  - Does NOT have a close or minimize box
- Frame (can contain widgets, including horizontal and vertical scrollbars)

  NOTE Panes do not automatically have a scrollbar. Apps can provide the scrollbar if the content of the pane exceeds the amount of the pane that is displayed.

- Pane widgets (`GtkHpaned`, `GtkVpaned`)
- NO dragging handles in v. 1.0 (`GtkHandleBox`). These are handles to drag something out of a window, e.g., toolbars.
- NO window resizing gadgets.
- Horizontal scrollbar.

  NOTE Modal dialogs with widgets can be scrolled. This is not standard in most desktop environments, but is designed as an overflow mechanism for localization.

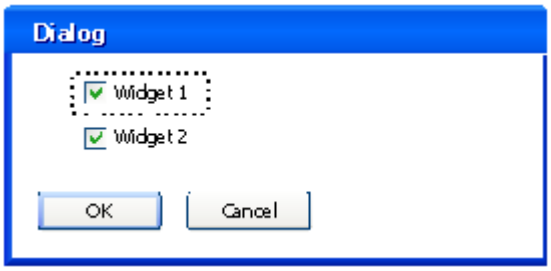
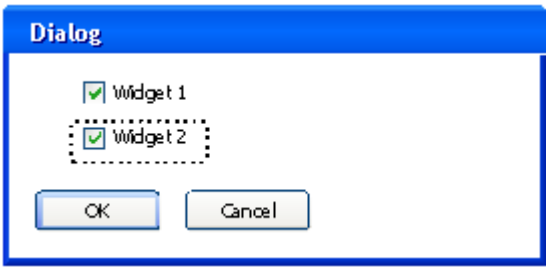
- Bottom scrollbar
  - Optional controls
  - Status text (`GtkStatusbar`?)
- Command buttons (modal dialogs). At a minimum a close button is required.

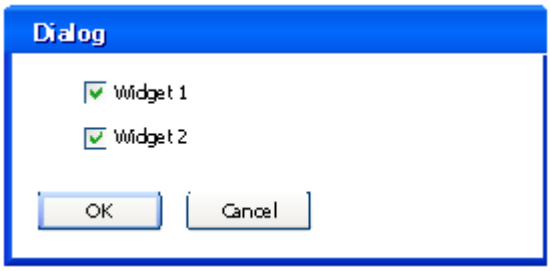
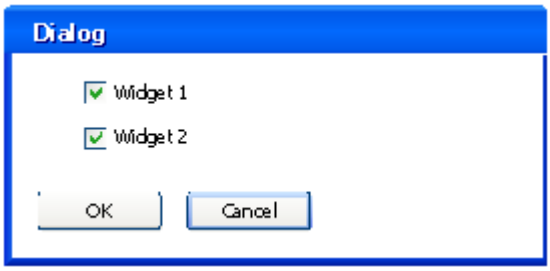
26.8.2.2 Navigation

- All widgets on the screen are part of the tab order and can receive focus.
- There is always at least one item with focus. Graphic treatment TBD.
- To move forward in the tab order, press Tab.
- To move backward in the tab order, press shift + Tab.

- Focus cycles at beginning and end of tab order.
- There is no concept of a Treo-like navigation and interaction mode. As a result, it may be possible for the tab key to get *stuck* in a text field if the text field accepts tabs.
- Either one or two items always have focus.
  - One command button always has focus.
  - If a widget is given focus, one of the command buttons also has focus. In this state, pressing Enter = OK, and Space = keydown on the widget with focus.
  - If the user goes through the tab order and gives focus to a button, then there are no widgets with focus, and pressing Enter or Space will both = keydown on the command button with focus.
- Default focus is typically the OK button and the first widget.
- Opening and closing windows:
  - Menu + Enter = OK
  - Menu + Delete = Cancel
  - No close box

Example of navigation

Action	State	Focus	Enter	Space
Open dialog		Widget 1, OK	OK	Toggle Widget 1
Press Tab		Widget 2, OK	OK	Toggle Widget 2

Action	State	Focus	Enter	Space
Press Tab		OK only	OK	OK
Press Tab		Cancel only	Cancel	Cancel

26.8.2.3 Moving and resizing windows

- Windows are opened by default centered horizontally and vertically in one size
- Moving windows
  - Windows can be moved by clicking and holding on the title bar and dragging
  - Windows can also be moved by clicking and dragging on the frame.
  - A window cannot be dragged entirely off the screen.
  - Window position is remembered when dragged.
- Resizing windows is supported by the OS but not provided as a general convention in our apps.

26.8.2.4 Windows and state saving

- While an app is running, if a window is closed and subsequently opened, the position and display of the window is maintained.
- When an app is quit, the state of the window is stored.
  - If the app is launched via the launch menu, i.e., launches in the default state, the state data is flushed.
  - If the app is launched via a back mechanism, i.e., app key + back, then the previous state of a modal dialog is restored.

26.8.2.5 Active windows

- Only one window can be active at a time, although it is possible to have one or more sub-dialogs open simultaneously. Multiple sub-dialogs are discouraged; as a UI convention, tabs or tree objects are preferred.

- "Click through" is not supported. To interact with another window, the top window must first be dismissed.
- Inactive windows have a different graphic treatment; they are dimmed behind the active window.

26.8.2.6 Types of windows

In addition to general modal dialogs, alert windows supported.

- Alerts can be displayed by any app or by the system.
- Alerts receive focus when they are initially displayed. This allows for easy dismissing.
- Alerts are modal and must be dismissed before action can be taken on the windows beneath them.

26.8.3 Title bar

Since the Falcon screen is only 400 pixels high, the title bar is designed to be as efficient as possible to maximize resolution of the main screen area. For example, at a 1024 x 768 resolution, a standard Word window in XP uses 253 pixels for the title and toolbar. This leaves about 515 pixels for displaying the content of the file. The Falcon title bar is designed to occupy 25-30 pixels, which would leave about 440 pixels for the content of the file. In other words, although the screen is 62.5% the height of a 1024 x 768 display, it can display 85% of the vertical content.

This efficiency is accomplished by:

- Moving the toolbar icons into the title bar
- Making the menu commands modal (press the menu button)

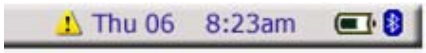


The title bar contains:

- Application name, flush left
- Optional toolbar icons (e.g., file open, cut, copy, paste)
- Optional widgets
  - Dropdown list and variants
  - Pushbuttons
  - Lookup field
- Background task icons, which may or may not be actionable (not shown)
- Alert icons
- A fixed number of pixels on the right side of the title bar are dedicated for system gadgets and status.
  - Date/time
  - Battery gauge
  - Bluetooth status

  - WiFi status
- Third-party applications have the ability to expand the title bar with additional rows in order to accommodate more icons. Graphic treatment TBD. Overuse of this is discouraged however, as the screen is already vertically challenged.

26.8.4 Alert icons



- Either the system or an app can display icons in the title bar.
- These icons can show multiple states and/or simple animation (several bitmap states for blinking, etc.).
- The icons appear in the right-hand section of the title bar, to the left of the system gadgets (time/date).
- If more than one alert icon appears, each new icon is displayed to the left of the previously displayed icon.
- If the icons overflow (take up more space than is available), then the icons are truncated (the app takes priority).
- Primary activation of the toolbar icons is via the track point.
- There is a debate about whether to give the toolbar icons focus; see design issues.
- Title bar icons should have hover text, as shown here.



26.9 System level functions

26.9.1 Launcher quick menu

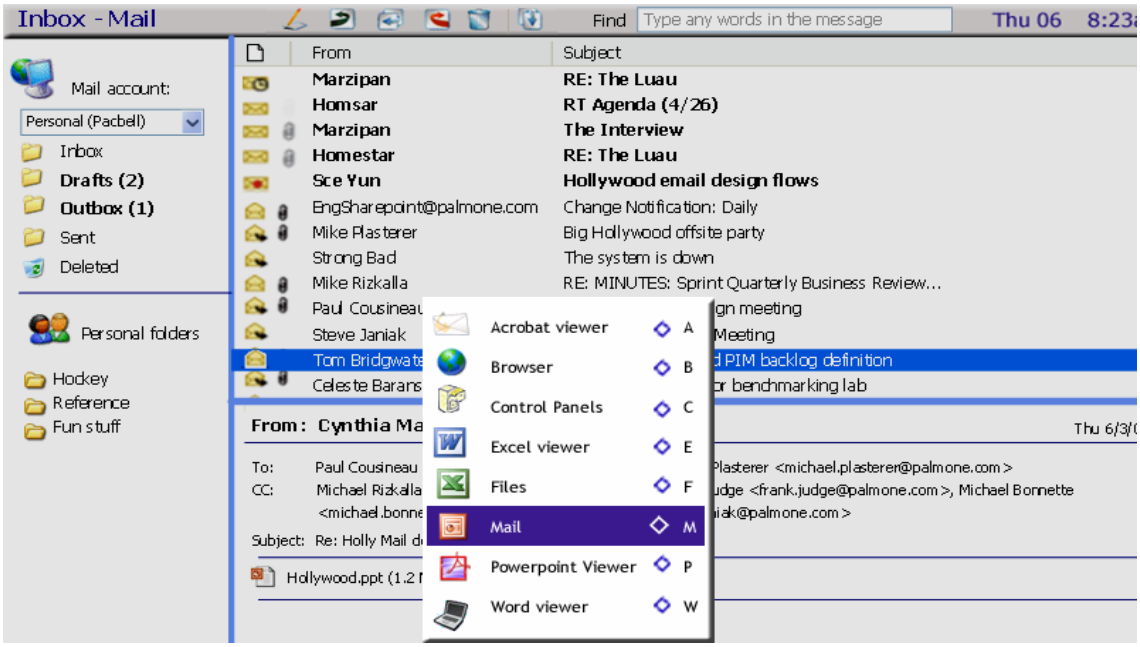
- Pressing the Apps button displays a list of all apps on the device. The list pops up from the bottom, not in the middle of the screen.
- Overflow goes to multiple columns.
- The list is sorted alphabetically only. There are no favorites, recent apps, or other sort options.
- Type-down lets you get to an app quickly.

26.9.2 UI objects

- UI objects pop up from the bottom rather than being centered on the screen.
- The list is alphabetical; there are no sorting options.
- Each entry in the list contains:
  - app icon

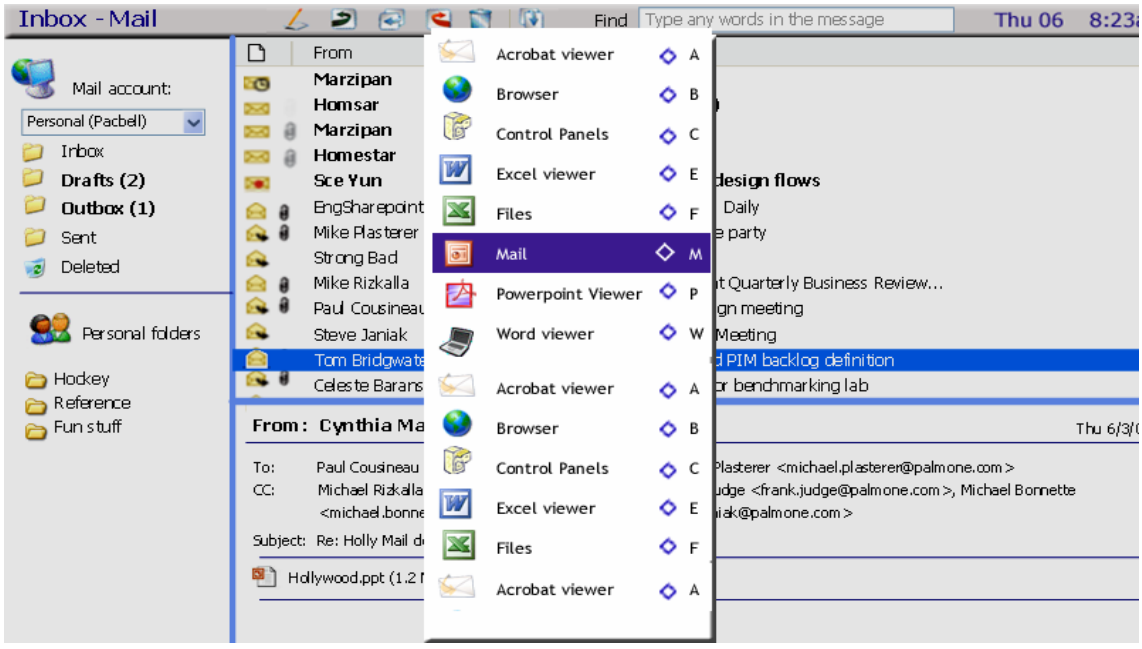
  - app name
  - app shortcut icon and letter (icon is a placeholder for symbol for apps button)
- The method for setting the letters is described below.
- Punt on drop shadow.
- Overflow goes to multiple columns.
- Graphics are placeholders. Note background treatment (dimming or changing of translucency).

Example of out-of-box treatment

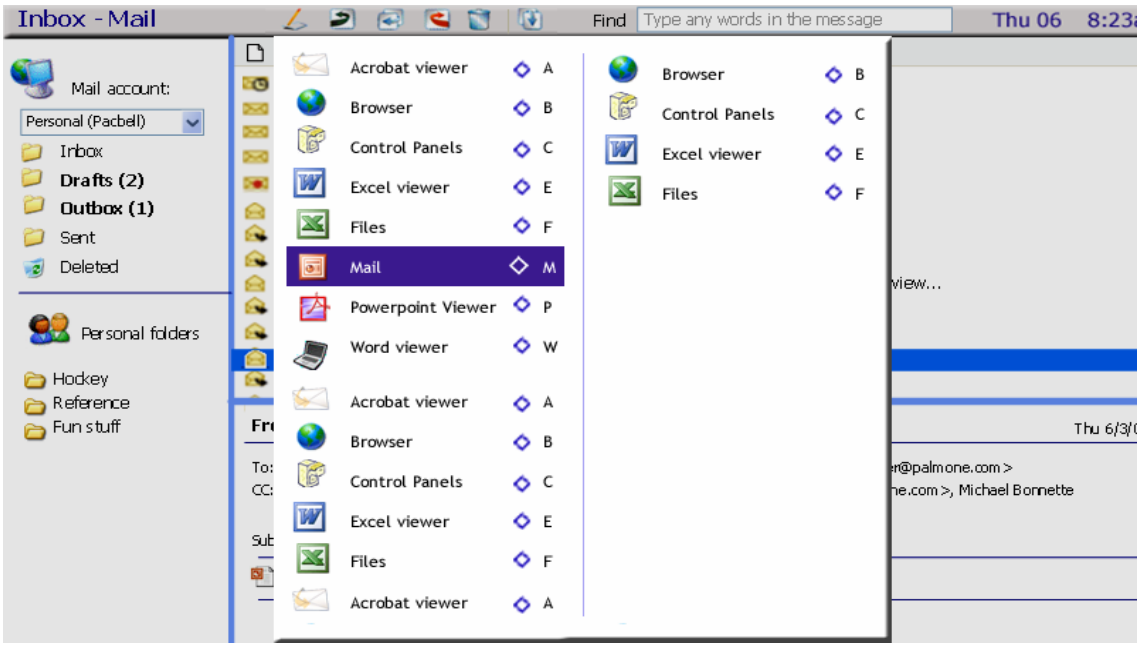


Expanded example

This example is expanded to fit more icons (ignore exact text, which is duplicated and not alphabetized):



Expanding beyond one column



26.9.3 File transfer with PCs

- Files can be copied to an SD card in a format that can be read by a PC when the SD card is inserted in a reader attached to the PC.
- Falcon will provide file transfer features over Bluetooth and WiFi.

26.9.4 Managing applications

- Applications and all of their supporting files should be installable, movable, and deleteable by manipulating a single entity.
  - No supporting files should be exposed to the user. The supporting files should, however, be viewable via command-prompt navigation.
  - Applications should not install files in system file areas.
  - No non-user data components should be left on the Falcon when an application is deleted.
- Falcon applications and supporting files are packaged in Falcon Packages, .hxp files.

26.10 Display Lock application

26.10.1 Goals

- Provide a password lock which secures the system
- Option to automatically disable based on the presence of a paired smartphone

26.10.2 Launching Display Lock

The Display Lock app is launched in one of two ways:

- Automatic launch via a timeout set in the System Preferences panel.
- Automatic launch during boot, reboot, or wake-up if the display locking feature is enabled in the System Preferences panel.

26.10.3 System password

Setting a password in the Display Lock application sets a user password on the system account. This is similar to setting the password with the command line passwd application, however the Display Lock application **DOES NOT** currently enable a weak password check.

Because the Display Lock application simply sets a user password, you can change the password at any time by using the passwd application from the terminal prompt. This is the same user password that is used to grant root access when *sume* is called.

NOTE It is possible to set a cryptographically weak password via the Display Lock application even though the same password would be rejected by the passwd application.

26.11 Localization

Every localization project for every app and every generation of OS software increases development time and costs. Common problems and challenges include:

- Resizing dialogs and debating abbreviations when strings do not fit.
- Extracting text in a format that can be easily managed with non-technical translators and/or reviewers.
- Tracking and managing text string changes during development of the base application.
- Debugging crashes, even though translating resources should not in theory introduce bugs.
- Running or porting a single-byte app to a double-byte system, or displaying double-byte data on a single-byte language device.

Falcon is designed to put effort up front to minimize these recurring problems.

26.12 Multiple-language ROM support

26.12.1 Requirements

- Ability to minimize SKU management by having multiple languages burned into a single ROM.
- Ability to switch between languages on a single device. If possible support the ability to switch this as a preference setting. This will create an easier factory process to reconfigure devices (although of course different languages have different keyboards).

- Ability to configure the keyboard independent of language.

26.12.2 Plan

- Support multiple languages in ROM. Currently, Falcon supports English and Spanish.
- Support run-time switching between languages.
- Map default data databases cleanly to the language switching. For example, if you switch to French, your help files switch to French as well.

PART VII

Appendix

APPENDIX A

HxUI XML DTD

A.1 HxUI XML Format Verification

The HxUI XML format was detailed in Chapter 18, which listed the many elements and attributes that could be used in the file. Verifying that the file was corrected created without syntax errors can be done with the following HxUI XML DTD file.

A.1.1 Configure HxUI XML File

You will need to reference hxui_1.0.dtd from your hxui file using the following line:

```
<!DOCTYPE hxui SYSTEM "hxui_1.0.dtd">
```

immediately after the xml declaration at the top of the file.

NOTE You'll need an XML verification tool, such as xmlstar (http://xmlstar.sourceforge.net), to parse the DTD file and validate your HxUI XML file.

A.1.2 DTD File

To create the DTD file copy all of the following core to hxui_1.0.dtd and save the file on your development PC.

```
<?xml version="1.0" encoding="UTF-8"?>

<!ELEMENT hxui (application_window | dialog | menu | root_menu |
  container)*>
<!ATTLIST hxui version CDATA #REQUIRED>
<!ATTLIST hxui default_locale CDATA "en_US">
<!ATTLIST hxui start_id CDATA #IMPLIED>

<!ELEMENT identifier EMPTY>
<!ATTLIST identifier name CDATA #REQUIRED>
<!ATTLIST identifier value CDATA #REQUIRED>

<!ELEMENT constant EMPTY>
<!ATTLIST constant name CDATA #REQUIRED>
<!ATTLIST constant value CDATA #REQUIRED>

<!ELEMENT pos EMPTY>
<!ATTLIST pos locale CDATA #IMPLIED>
<!ATTLIST pos x CDATA #REQUIRED>
<!ATTLIST pos y CDATA #REQUIRED>
```

```
<!ELEMENT size EMPTY>
<!ATTLIST size locale CDATA #IMPLIED>
<!ATTLIST size w CDATA #REQUIRED>
<!ATTLIST size h CDATA #REQUIRED>

<!ELEMENT font EMPTY>
<!ATTLIST font locale CDATA #IMPLIED>
<!ATTLIST font family (standard | monospaced | serif | proportional
|
  sans_serif) "standard">
<!ATTLIST font point_size CDATA "10">
<!ATTLIST font weight (plain | bold | italic | bold_italic) "plain">
<!ATTLIST font name CDATA "">
<!ATTLIST font is_kerned (yes | no) "yes">
<!ATTLIST font is_hinted (yes | no) "yes">
<!ATTLIST font is_antialiased (yes | no) "yes">

<!ELEMENT color EMPTY>
<!ATTLIST color locale CDATA #IMPLIED>
<!ATTLIST color value CDATA #REQUIRED>
<!ATTLIST color alpha CDATA #IMPLIED>

<!ELEMENT text (#PCDATA)>
<!ATTLIST text locale CDATA #IMPLIED>

<!ELEMENT image EMPTY>
<!ATTLIST image locale CDATA #IMPLIED>
<!ATTLIST image file CDATA #REQUIRED>

<!ELEMENT application_window (command_button | checkbox | combobox
|
  container | drowdownbox | field | gadget | label | listbox |
  radio_button_group | scrollbar | slider | spin_button |
  focus_array)*>
<!ATTLIST application_window locale CDATA #IMPLIED>
<!ATTLIST application_window id CDATA #REQUIRED>

<!ELEMENT window (pos | size | command_button | checkbox | combobox
|
  container | drowdownbox | field | gadget | label | listbox |
  radio_button_group | scrollbar | slider | spin_button |
  focus_array)*>
<!ATTLIST window locale CDATA #IMPLIED>
<!ATTLIST window id CDATA #REQUIRED>
<!ATTLIST window level (top | middle | bottom) #REQUIRED>

<!ELEMENT dialog (pos | size | text | command_button | checkbox |
```

```
  combobox | container | drowdownbox | field | gadget | label |
  listbox | radio_button_group | scrollbar | slider | spin_button |
  focus_array)*>
<!ATTLIST dialog locale CDATA #IMPLIED>
<!ATTLIST dialog id CDATA #REQUIRED>

<!ELEMENT command_button (pos | size | text)*>
<!ATTLIST command_button locale CDATA #IMPLIED>
<!ATTLIST command_button id CDATA #REQUIRED>
<!ATTLIST command_button enabled (yes | no) "yes">
<!ATTLIST command_button visible (yes | no) "yes">
<!ATTLIST command_button is_default (yes | no) "no">

<!ELEMENT checkbox (pos | text)*>
<!ATTLIST checkbox locale CDATA #IMPLIED>
<!ATTLIST checkbox id CDATA #REQUIRED>
<!ATTLIST checkbox enabled (yes | no) "yes">
<!ATTLIST checkbox visible (yes | no) "yes">
<!ATTLIST checkbox is_checked (yes | no) "no">

<!ELEMENT combobox (pos | size | listitem)*>
<!ATTLIST combobox locale CDATA #IMPLIED>
<!ATTLIST combobox id CDATA #REQUIRED>
<!ATTLIST combobox enabled (yes | no) "yes">
<!ATTLIST combobox visible (yes | no) "yes">

<!ELEMENT container (pos | size | color | command_button | checkbox
|
  combobox | container | drowdownbox | field | gadget | label |
  listbox | radio_button_group | scrollbar | slider | spin_button |
  tabbed_container)*>
<!ATTLIST container locale CDATA #IMPLIED>
<!ATTLIST container id CDATA #REQUIRED>
<!ATTLIST container enabled (yes | no) "yes">
<!ATTLIST container visible (yes | no) "yes">
<!ATTLIST container has_frame (yes | no) "no">
<!ATTLIST container has_horizontal_resizer (yes | no) "no">
<!ATTLIST container has_vertical_resizer (yes | no) "no">
<!ATTLIST container focusable (yes | no) "no">

<!ELEMENT dropdownbox (pos | text | listitem)*>
<!ATTLIST dropdownbox locale CDATA #IMPLIED>
<!ATTLIST dropdownbox id CDATA #REQUIRED>
<!ATTLIST dropdownbox enabled (yes | no) "yes">
<!ATTLIST dropdownbox visible (yes | no) "yes">

<!ELEMENT field (pos | size | text | font | color)*>
<!ATTLIST field locale CDATA #IMPLIED>
```

```
<!ATTLIST field id CDATA #REQUIRED>
<!ATTLIST field enabled (yes | no) "yes">
<!ATTLIST field visible (yes | no) "yes">
<!ATTLIST field is_editable (yes | no) "yes">
<!ATTLIST field is_multiline (yes | no) "no">
<!ATTLIST field is_scrollable (yes | no) "no">
<!ATTLIST field is_password (yes | no) "no">
<!ATTLIST field is_numeric (yes | no) "no">

<!ELEMENT gadget (pos | size | text_object | graphic_object)*>
<!ATTLIST gadget locale CDATA #IMPLIED>
<!ATTLIST gadget id CDATA #REQUIRED>
<!ATTLIST gadget enabled (yes | no) "yes">
<!ATTLIST gadget visible (yes | no) "yes">
<!ATTLIST gadget focusable (yes | no) "no">

<!ELEMENT text_object (pos | text | font)*>
<!ATTLIST text_object locale CDATA #IMPLIED>

<!ELEMENT graphic_object (pos | image)*>
<!ATTLIST graphic_object locale CDATA #IMPLIED>

<!ELEMENT label (pos | size | text | font | color)*>
<!ATTLIST label locale CDATA #IMPLIED>
<!ATTLIST label id CDATA #REQUIRED>
<!ATTLIST label enabled (yes | no) "yes">
<!ATTLIST label visible (yes | no) "yes">
<!ATTLIST label alignment (left | center | right) "left">
<!ATTLIST label origin (top | bottom) "top">
<!ATTLIST label is_underlined (yes | no) "no">

<!ELEMENT listbox (pos | size | listitem)*>
<!ATTLIST listbox locale CDATA #IMPLIED>
<!ATTLIST listbox id CDATA #REQUIRED>
<!ATTLIST listbox enabled (yes | no) "yes">
<!ATTLIST listbox visible (yes | no) "yes">
<!ATTLIST listbox allow_multiple_selection (yes | no) "no">

<!ELEMENT listitem (image | text)*>
<!ATTLIST listitem locale CDATA #IMPLIED>
<!ATTLIST listitem is_selected (yes | no) "no">

<!ELEMENT radio_button (pos | text)*>
<!ATTLIST radio_button locale CDATA #IMPLIED>
<!ATTLIST radio_button id CDATA #REQUIRED>
<!ATTLIST radio_button enabled (yes | no) "yes">
<!ATTLIST radio_button visible (yes | no) "yes">
<!ATTLIST radio_button state (on | off) "off">
```

```
<!ELEMENT radio_button_group (radio_button)*>
<!ATTLIST radio_button_group locale CDATA #IMPLIED>

<!ELEMENT scrollbar EMPTY>
<!ATTLIST scrollbar locale CDATA #IMPLIED>
<!ATTLIST scrollbar id CDATA #REQUIRED>
<!ATTLIST scrollbar enabled (yes | no) "yes">
<!ATTLIST scrollbar visible (yes | no) "yes">
<!ATTLIST scrollbar full_range CDATA #REQUIRED>
<!ATTLIST scrollbar visible_range CDATA #REQUIRED>
<!ATTLIST scrollbar initial_position CDATA "0">

<!ELEMENT slider (pos | size)*>
<!ATTLIST slider locale CDATA #IMPLIED>
<!ATTLIST slider id CDATA #REQUIRED>
<!ATTLIST slider enabled (yes | no) "yes">
<!ATTLIST slider visible (yes | no) "yes">
<!ATTLIST slider num_ticks CDATA "0">
<!ATTLIST slider orientation (horizontal | vertical) "horizontal">
<!ATTLIST slider initial_value CDATA #REQUIRED>
<!ATTLIST slider min_value CDATA #REQUIRED>
<!ATTLIST slider max_value CDATA #REQUIRED>
<!ATTLIST slider page_size CDATA #REQUIRED>

<!ELEMENT spin_button (pos)*>
<!ATTLIST spin_button locale CDATA #IMPLIED>
<!ATTLIST spin_button id CDATA #REQUIRED>
<!ATTLIST spin_button enabled (yes | no) "yes">
<!ATTLIST spin_button visible (yes | no) "yes">
<!ATTLIST spin_button initial_value CDATA #REQUIRED>
<!ATTLIST spin_button min_value CDATA #REQUIRED>
<!ATTLIST spin_button max_value CDATA #REQUIRED>
<!ATTLIST spin_button increment CDATA "1.0">
<!ATTLIST spin_button decimal_precision CDATA "0">

<!ELEMENT tab (text)*>
<!ATTLIST tab locale CDATA #IMPLIED>
<!ATTLIST tab id CDATA #REQUIRED>
<!ATTLIST tab is_selected (yes | no) "no">

<!ELEMENT tabbed_container (pos | size | tab | command_button |
  checkbox | combobox | container | drowdownbox | field | gadget |
  label | listbox | radio_button_group | scrollbar | slider |
  spin_button | tabbed_container)*>
<!ATTLIST tabbed_container locale CDATA #IMPLIED>
<!ATTLIST tabbed_container id CDATA #REQUIRED>
<!ATTLIST tabbed_container enabled (yes | no) "yes">
```

```
<!ATTLIST tabbed_container visible (yes | no) "yes">

<!ELEMENT focus_array (focus_item)*>
<!ATTLIST focus_array locale CDATA #IMPLIED>

<!ELEMENT focus_item EMPTY>
<!ATTLIST focus_item locale CDATA #IMPLIED>
<!ATTLIST focus_item id CDATA #REQUIRED>

<!ELEMENT menu (menu_item | menu_separator | submenu)*>
<!ATTLIST menu locale CDATA #IMPLIED>
<!ATTLIST menu id CDATA #REQUIRED>

<!ELEMENT root_menu (submenu)*>
<!ATTLIST root_menu locale CDATA #IMPLIED>
<!ATTLIST root_menu id CDATA #REQUIRED>

<!ELEMENT menu_item (image | text)*>
<!ATTLIST menu_item locale CDATA #IMPLIED>
<!ATTLIST menu_item id CDATA #REQUIRED>
<!ATTLIST menu_item shortcut CDATA #IMPLIED>
<!ATTLIST menu_item enabled (yes | no) "yes">

<!ELEMENT toggle_menu_item (image | text)*>
<!ATTLIST toggle_menu_item locale CDATA #IMPLIED>
<!ATTLIST toggle_menu_item id CDATA #REQUIRED>
<!ATTLIST toggle_menu_item shortcut CDATA #IMPLIED>
<!ATTLIST toggle_menu_item enabled (yes | no) "yes">
<!ATTLIST toggle_menu_item selected (yes | no) #REQUIRED>

<!ELEMENT menu_separator EMPTY>
<!ATTLIST menu_separator locale CDATA #IMPLIED>
<!ATTLIST menu_separator id CDATA #REQUIRED>

<!ELEMENT submenu (image | text | menu_item | menu_separator |
  submenu)*>
<!ATTLIST submenu locale CDATA #IMPLIED>
<!ATTLIST submenu id CDATA #REQUIRED>
<!ATTLIST submenu shortcut CDATA #IMPLIED>
<!ATTLIST submenu enabled (yes | no) "yes">
```

APPENDIX B

FAQs

B.1 Falcon

B.1.1 Hardware

Q: How do I determine which version of Falcon I have?

A: Look at the serial number that is etched into the device on the LCD bezel:

ME5 = Falcon EVT5 (no longer supported)
FE6 = Falcon EVT6
FD1 = Falcon DVT

To determine board revision from the kernel:

```
Cat /proc/cpuinfo
```

The number in the Revision field will display the specific model of the device: EVT5, EVT6 or DVT.

Q: I'm confused about connecting my development system to the Falcon?

A: After you boot the device into the Linux kernel, it will then be a USB host and will use the USB-to-Ethernet dongle as its NIC. It will be configured by our init scripts as 192.168.1.102 (this will be changed to dhcp later). It should be connected with a crossover cable to the 2nd NIC on your workstation which needs to be configured as 192.168.2.100 since that's hard-coded in the fstab and other scripts on the device.

Q: My Ethernet-usb adapter doesn't seem to be working.

A: Make sure that your adapter is a a Palm supported adapter. These are the only adapter with driver support on the Falcon. If the adapter is connected to a working Ethernet network, you will see green flashing lights on the adapter.

Q: Usually on a Linux system there is a /proc/bus/usb device, but not on Falcon. How is the USB setup for the Falcon device?

A: The /proc/bus/usb entries are available in the current build of the Falcon system ROM.

Q: How do I know what the name of my Falcon device is when running the Bluetooth stack as a server?

A: /etc/hostname file contains the device name that your Treo will pick up. If you edit this file and change the name, make sure to reboot your device.

Q: Can Falcon use MMC cards?

A: Yes. The Falcon can use both SD and MMC cards. Please note that low-quality cards may not be manufactured to proper tolerances. Palm recommends only using high-quality name-brand cards in the Falcon.

Q: How do you set the LCD brightness and flicker levels?

A: To get current LCD brightness and flicker levels while in the terminal:

```
Cat /proc/video/brightness
Cat /proc/video/flicker
```

To set current LCD brightness and flicker levels while in the terminal:

```
Echo "400" >> /proc/video/brightness
Echo "400" >> /proc/video/flicker
```

These values can be written in hex or decimal. If you enter a number out-of-range, the tool will warn you.

NOTE These values change on-the-fly, but they are reset to factory defaults upon reboot.

B.1.2 OS

Q: Can you have spaces in program names on Falcon?

A: The .hxp Format Specification document should contain the following line: Filenames embedded in .hxp files should *not* contain any spaces, as that makes it easier to process with standard UNIX tools.

That means no spaces in the executable name. However, it is acceptable to put spaces in the display name field of the .hxp file, which is used when the Apps key is pressed. So the name displayed when choosing the application may contain spaces.

If you must use spaces, use the underscore (_) in place of a space in the filename.

Q: Can a file path be larger than 256 characters? Given that documents will live in the /Home/User/Data folder, it would seem unlikely to surpass 256 characters.

A: The kernel header limits.h sets PATH_MAX to 4096, and NAME_MAX to 255. NAME_MAX is the limit of the filename. The pathname must be less than 256 characters, even with the /Home/User/Data convention.

Q: Are you currently building any ROMs with X-Windows?

A: Falcon does not and is not going to have an x11 world. (To simulate the fixed screen size on the Falcon, our simulator does use an SDL/X11 window. But there is no programmatic access to it. And if there were, it would not function on the Falcon itself).

Q: How about GTK?

A: In the Hollywood (pre-Falcon) days, Palm tried GTK but found it not responsive enough. Now Palm provides HxUI, a custom framework based on LiTE, which in turn is based on the DirectFB graphics library, which performs well.

Q: But I notice that you provide glib which is part of the GTK package?

A: Yes, when we used GTK, we also included glib. glib provides some useful utility functions so we continue to make most of them available. However there were some features of glib that we did not choose to support. We do not provide gobject, gmodule, or gthread. A partial list of those features we do support include garray, gettext, gdate, gstring, and gtree. There are many others.

Q: Are threads handled differently in the Falcon build compared to standard Linux?

A: Falcon uses default Linux pthreads. Usually it's tricky to run events from separate threads. Best to have the GUI in the main thread, and helper, non-GUI threads running for other tasks.

B.1.3 Applications

Q: How do you view .html files through the browser on Falcon?

A: You can view local html files using file:///path in HollyWeb. For example, if a xyz.html file is located in the Applications directory, in HollyWeb's URL field enter:

```
file:///Applications/xyz.html
```

B.2 SDK

B.2.1 Installing

Q: I can no longer run the simulator and I get a "Fusion/Init" error.

A: When launching your application in the simulator, you got this error message Fusion/Init: FUSION_ENTER failed.

This can occur if an updated version of the SDK is installed over a current version and a new version of the fustion module is included in the SDK. When the sdk-installer.sh script runs, it discovers fusion is already there and does not install the new version.

In order to correct the issue you must remove the old version of fusion. Issue the following command within the terminal:

```
sudo rmmod fusion
```

Then rerun the fusion-installer.sh step of the SDK install. More details can be found in the release notes and readme files. This might recur if DirectFB is upgraded in the future.

B.2.2 Programming

Q: What does it mean when make wants me to add to the PKG_CONFIG_PATH environment variable?

A: If, after typing `make` for a Falcon project, you see the error message `Package hx-1.0 was not found in the pkg-config search path`, you should add the directory containing hx-1.0.pc to the PKG_CONFIG_PATH environment variable.

This could be fixed by properly setting PKG_CONFIG_PATH. However, Palm provides a custom pkg-config tool that already has the proper path built in. The trick is that we actually adjust the PATH variable to point to our version of pkg-config.

```
export PATH= /opt/holly/toolroot/i386-linux/local/bin; \ /opt/holly/
toolroot/local/bin:$PATH
```

This has the advantage of ensuring that other tools provided with the Falcon SDK also work as expected.

Q: I forgot whether I built for arm or i386. How do I find out what my executable is?

A: Use `file`. For i386 it will return:

```
hxhello: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), for GNU/
Linux 2.0.0, dynamically linked (uses shared libs), not stripped.
```

For arm it will return:

```
file: error while loading shared libraries:
/opt/holly/sysroot/arm-linux/usr/lib/libz.so.1: ELF file OS ABI invalid
```

Or you could just try to execute your application. If it is the correct type of executable, it will run. If it is an arm file you are trying to execute on the simulator, Linux will return:

```
cannot execute binary file
```

Q: What does it mean if I got a PTHREAD_RECURSIVE_MUTEX_INITIALIZER_NP warning?

A: If in compiling you get the warning message #warning PTHREAD_RECURSIVE_MUTEX_INITIALIZER_NP is not defined, be aware of dead locks. That usually happens if there's suddenly a header that requires D_GNU_SOURCE to make sure the pthread libraries include the right defines, for example, if you write an application using the pthread libraries.

B.3 API

B.3.1 HxUI

Q: Are there any special key down characters that we should ignore? For editing, we are currently treating the key down key codes as Unicode characters. On the PalmOS, we'd get characters for low batteries that we wouldn't want to treat as text. Is there a range that we could check against?

A: The only character that you should ignore is the key symbol of 0xF03A, used by the email button.

Q: Is there a way to tell that the click was a double-click or a triple-click? We spoke about how in fields double clicking would select an entire word

and triple clicking would select an entire line. There didn't seem to be a parameter for it in the `HxWidgetOnButtonDownFnPtr` *callback.*

A: We have not yet implemented support for this. It may be implemented as an extension of the event, or as a message. In general, we prefer to use the messaging system over overloading the events.

Q: Is it okay to pass a NULL pointer to `HxWidgetSetCallback` *for both hOldCallback and pCallback? When we receive a mouse down message and it's in a position onscreen that we care about, we'd like to register for the mouse move notification. Then when the mouse up comes through, we'd like to unregister for the mouse move. This should keep our code cleaner because we won't have to keep track of whether the mouse was down or not. However, when we first set the callback, the old callback comes back as NULL. So when the mouse up occurs, if we reset the callback to the old callback, we'll be sending NULL to both parameters. Will this work or are we going about this in the wrong way?*

A: You are going about this in the right way. And NULL can be sent for both parameters.

Q: Mouse tracking doesn't seem to know when I have moved out of a button. If I mouse down on a CommandButton, and then move the cursor off the control while the mouse button is down, this is treated as a successful hit. This should in fact be a cancel. This is true for checkboxes and radio buttons as well.

A: This is a known bug which is about to be fixed.

Q: Why does an `HxWidgetGetBounds` *right after* `HxGadgetNew` *return wrong values?*

A: Originally we allowed a 4 pixel focus border that we would always want to appear onscreen. So after calling `HxGadgetNew` we might have adjusted size and location for that. Since the 1.4C branch, we've added a new `HxGadgetNew` API that takes a focusable parameter and assumes that if you set that to true, you've already accounted for the gadget boundary in your bounds and `HxWidgetGetBounds` will return the expected value.

Q: Why can't I set my `HxListBox` *to have no selection?*

A: This was a bug in pre-1.4C versions of the OS. Now you can use `HxListBoxSetSelection` to set the selection to HXLISTBOX_NOSELECTION.

Q: Why won't `HxWidgetSetEnabled` *enable/disable my command button?*

`HxWidgetSetEnabled` does disable widgets such as checkbox and list, etc.

A: `HxResult` is an enum, which can sometimes be treated as a type. Falcon types typically end with a "_t". However, `HxResult` has no _t for historical reasons and isn't likely to change.

Q: Am I really not supposed to ever use `HxDeleteMessage` *on non-recurring timers, even if I'm calling* `HxRemoveTimerMessage()` *to remove the timer prematurely?*

A: There are recurring timers and non-recurring. The recurring timers run and run until `HxRemoveTimerMessage()` is called. It will just remove the timer instance. `HxDeleteMessage` will delete the actual message instance itself. I think I need to rename the API to indicate that it removes a timer, and not the message.

The non-recurring timers fire once, and that's it, they are removed from the timer chain. There's no need to call `HxRemoveTimerMessage()`, if it's done you get the HX_RESOURCE_NOT_FOUND error as it's removed immediately after it has fired. But the actual message is still around. You could remove the message itself with `HxDeleteMessage()`, but it has nothing to do with the timer instances.

Q: Can we reuse strings for HxSendTimerMessage? I understand the string needs to be unique for each message that we want to post with `HxSendTimerMessage()`*. For how long does this need to be true? If our application exits completely and restarts, would it be bad if the strings were reused?*

A: Yes it's unique within the context of the application process, so you could reuse it as many times you want inside the application. This restriction is mostly to make sure user messages do not clash, i.e., someone else is using the same message ID.

Q: Why does HxResult not end with "_t"?

A: `HxWidgetSetEnabled` is indeed the function to enable and disable command buttons. It just had not been implemented as of the 1.4B branch.

B.3.2 LiTE

Q: Can I resize a surface?

A: The bad news is that you can't resize a surface that was created independently of a window. If it was created as a free-standing surface, it needs to be destroyed and recreated.

The good news is that DirectFB does allow you to change the size of a window. DirectFB will take care of the surface destruction, recreation, and copying over the video buffer for you.

Q: Was there a fix for corrupted HxWindow_t structs?

A: The problem was with the LiteWindow that is part of `HxWindow_t`. The LiteWindow was not always properly initialized. We added the lite_init_window API to LiTE, which separates the allocation and initialization of the LiteWindow. HxWindow internally calls lite_init_window to insure that it is properly set up.

B.4 Linux

Q: I've updated my PATH variable in my .bashrc file, but it doesn't seem to have any effect?

A: .bashrc is executed once when you start a terminal window. If you change something in .bashrc, you need to reload the values. For this reason, you usually don't want to be modifying your PATH variable in .bashrc (unless you like doing the special-casing of checking path before setting it all over the place). Path should be setup in .bash_profile. That will get executed once on login, and won't get re-executed on sourcing .bashrc. To keep it clean, create a separate file, like ".bash_hollypath" and add your Falcon-specific locations to your path there. You'll need to add the line to source .bash_hollypath inside of your ~/.bash_profile script.